دانشگاه صنعتی امیرکبیر

(پلی‌تکنیک تهران)

دانشکده مهندسی کامپیوتر

پایان‌نامه کارشناسی

مهندسی کامپیوتر

مقایسه مدل‌های بینایی ماشین مبتنی بر فریم و مبتنی بر رویداد در سامانه‌های نهفته بی‌درنگ

نگارش

سید سینا نگهبان

استاد راهنما

دکتر حامد فربه

شهریور ۱۴۰۵


بسم الله الرحمن الرحيم

چکیده

بینایی ماشین بخش جدایی‌ناپذیر بسیاری از سامانه‌های خودکار نهفته و بی‌درنگ است، اما شبکه‌های عصبی پیچشی متعارف صرف‌نظر از میزان تغییر واقعی صحنه، حجمی تقریبا ثابت از محاسبات را بر هر فریم تحمیل می‌کنند. دوربین‌های رویدادمحور و شبکه‌های عصبی اسپایکی (SNN)، با تولید داده و انجام محاسبه فقط در صورت وجود تغییر، از نظر مفهومی برای چنین کاربردهایی جذاب‌اند؛ اما پیشینه‌ی پژوهش نشان می‌دهد این کاهش نظری شرط لازم برای کارایی عملی است، نه شرط کافی، و نتیجه به‌شدت به سخت‌افزار زیرین وابسته است. این پروژه با هدف بررسی شکاف میان مزیت نظری و عملی شکل گرفته و می‌پرسد وقتی یک زنجیره‌ی بینایی ماشین رویدادمحور و اسپایکی، به‌جای زنجیره‌ی متعارف فریم‌محور، برای تشخیص اشیا روی سکوهای محاسباتی واقعی اجرا شود، این مزیت نظری چه مقدار صرفه‌جویی واقعی در حافظه، تاخیر و انرژی به همراه دارد. برای پاسخ، به‌جای مقایسه‌ی دقت تشخیص، ماتریسی از ۱۸ مدل همتا از دو خانواده‌ی معماری ST-VGG و ResCat، در سه مقیاس کانالی و سه نسخه‌ی ورودی (Event-SNN، Strict RGB-ANN و ANN-native RGB)، بر پایه‌ی اصل برابری ساختاری روی صحنه‌های یکسان از مجموعه‌داده‌ی DSEC پیاده‌سازی و از چهار زاویه‌ی مکمل شامل شمارش نظری محاسبات، اجرای اندازه‌گیری‌شده روی GPU، مدل تحلیلی انرژی Lemaire و برآورد نگاشت نورومورفیک با SANA-FE ارزیابی شده است. نتایج نشان می‌دهند این کاهش نظری در چارچوب تحلیلی به کاهش واقعی انرژی می‌انجامد، اما وقتی همان مدل‌ها به‌صورت متراکم روی یک GPU متعارف اجرا می‌شوند، به‌طور خودکار به کاهش تاخیر یا انرژی تبدیل نمی‌شود. نتیجه اصلی پروژه این است که کارایی عملی محاسبات رویدادمحور و اسپایکی، ویژگی ذاتی الگوریتم به‌تنهایی نیست، بلکه حاصل طراحی همزمان الگوریتم و سخت‌افزار است.

واژه‌های کلیدی:

بینایی ماشین رویدادمحور، شبکه‌های عصبی اسپایکی، سامانه‌های نهفته بی‌درنگ، کارایی انرژی، سخت‌افزار نورومورفیک

فهرست مطالب

۱ مقدمه ۱

۱-۱ انگیزه و طرح مسئله ۲

۱-۲ شکاف میان مزیت نظری و مزیت عملی ۲

۱-۳ هدف، دامنه و کار انجام‌شده در پروژه ۳

۱-۴ ساختار پایان‌نامه ۴

۲ مبانی و پیشینه بینایی ماشین مبتنی بر فریم و رویداد ۵

۲-۱ مقدمه ۶

۲-۲ بینایی ماشین و کاربردهای آن ۶

۲-۳ شبکه‌های عصبی برای بینایی ماشین: از ویژگی‌های دستی تا یادگیری عمیق ۸

۲-۴ از طبقه‌بندی تصویر تا تشخیص اشیا: مدل‌های مبتنی بر فریم ۹

۲-۵ محدودیت‌های بینایی مبتنی بر فریم برای پردازش بی‌درنگ ۱۰

۲-۶ دوربین‌های رویدادمحور: الهام زیستی و اصول کارکرد ۱۱

۲-۶-۱ الهام از شبکیه‌ی زیستی ۱۱

۲-۶-۲ اصول کارکرد حسگر رویدادمحور ۱۲

۲-۶-۳ کاربردهای بینایی رویدادمحور ۱۳

۲-۷ شبکه‌های عصبی اسپایکی: الهام از سیستم عصبی زیستی ۱۴

۲-۷-۱ نورون زیستی و نمایش اسپایکی ۱۴

۲-۷-۲ از نسل‌های شبکه‌ی عصبی تا شبکه‌های اسپایکی ۱۴

۲-۷-۳ مدل نورون LIF و پویایی زمانی ۱۵

۲-۷-۴ محاسبات رویدادمحور و تنکی فعالیت ۱۶

۲-۸ استنتاج زمانی: زمان به‌مثابه بعد محاسباتی ۱۶

۲-۹ بینایی ماشین در سامانه‌های نهفته بی‌درنگ ۱۷

۲-۹-۱ محدودیت‌های سامانه‌های نهفته ۱۷

۲-۹-۲ مزیت رویداد و اسپایک برای سامانه‌های نهفته ۱۷

۲-۹-۳ از وعده‌ی نظری تا عملکرد واقعی ۱۸

۲-۱۰ پیشینه‌ی پژوهش ۱۹
۲-۱۰-۱ مدل‌های تشخیص اشیا روی داده‌ی رویدادمحور ۱۹
۲-۱۰-۲ مقایسه‌ی کارایی انرژی شبکه‌های عصبی اسپایکی و مصنوعی ۲۰
۲-۱۰-۳ سخت‌افزار نورومورفیک و معیارسنجی آن ۲۱
۲-۱۱ جمع‌بندی ۲۳

۳ چارچوب مفهومی و طراحی مطالعه مقایسه‌ای ۲۴
۳-۱ مقدمه ۲۵
۳-۲ پرسش راهنمای این پروژه ۲۵
۳-۳ مرزبندی ارزیابی کیفیت تشخیص ۲۵
۳-۴ ابعاد مقایسه ۲۶
۳-۵ اصل برابری ساختاری در طراحی مقایسه ۲۷
۳-۶ مجموعه‌داده‌ی مشترک و نمایش دوگانه‌ی ورودی ۲۷
۳-۷ ضرورت استفاده از چند سطح شواهد در ارزیابی ۲۸
۳-۸ جمع‌بندی ۲۸

۴ روش‌شناسی، پیاده‌سازی و ارزیابی ۳۰
۴-۱ مقدمه ۳۱
۴-۲ طراحی آزمایش و ماتریس مدل‌ها ۳۱
۴-۳ مجموعه‌داده و ساخت ورودی‌ها ۳۴
۴-۴ مدل نورون اسپایکی PLIF و پردازش زمانی ۳۶
۴-۵ آموزش محدود Event-SNNها و دامنه‌ی ادعا ۳۸
۴-۶ محیط نرم‌افزاری و معیارهای ارزیابی ۳۹
۴-۶-۱ پارامتر، حافظه و شمارش محاسبات ۳۹
۴-۶-۲ فعالیت اسپایکی ۳۹
۴-۶-۳ تاخیر، throughput و حافظه‌ی GPU ۳۹
۴-۶-۴ اندازه‌گیری انرژی GPU ۴۱
۴-۶-۵ مدل تحلیلی انرژی ۴۲

۴-۶-۶ نگاشت نورومورفیک با SANA-FE ۴۴
۴-۷ نتایج ۴۶
۴-۷-۱ برابری ساختاری، اندازه‌ی مدل و هزینه‌ی حافظه ۴۶
۴-۷-۲ فعالیت اسپایکی و محاسبات موثر ۴۸
۴-۷-۳ تاخیر full-detector روی GPU ۵۲
۴-۷-۴ انرژی و توان اندازه‌گیری‌شده‌ی full-detector روی GPU ۵۴
۴-۷-۵ برآورد تحلیلی انرژی برای full-detector ۵۷
۴-۷-۶ نتایج نگاشت نورومورفیک SANA-FE ۶۱
۴-۷-۷ جمع‌بندی کمی نتایج در سطوح ارزیابی ۶۳
۴-۸ محدودیت‌های روش و نتایج ۶۵

۵ نتیجه‌گیری و پیشنهاد کارهای آینده ۶۶
۵-۱ نتیجه‌گیری ۶۷
۵-۲ پیشنهاد کارهای آینده ۶۸

منابع و مراجع ۷۰

فهرست اشکال

۴-۱ معماری ST-VGG+SSD ۳۲
۴-۲ معماری ResCat-SNN+SSD ۳۲
۴-۳ نمونه ورودی DSEC ۳۴
۴-۴ مدل نورون PLIF ۳۷
۴-۵ برابری ساختاری مدل‌ها ۴۷
۴-۶ حافظه مدل‌ها ۴۸
۴-۷ نرخ اسپایک لایه‌ها ۴۹
۴-۸ مقایسه محاسبات ۵۱
۴-۹ تاخیر full-detector ۵۳
۴-۱۰ انرژی GPU ۵۴
۴-۱۱ نسبت انرژی Event-SNN ۵۶
۴-۱۲ نمودار تاخیر-انرژی ۵۷
۴-۱۳ نسبت انرژی تحلیلی ۵۹
۴-۱۴ تفکیک انرژی تحلیلی ۶۱
۴-۱۵ انرژی SANA-FE ۶۲
۴-۱۶ تاخیر SANA-FE ۶۳
۴-۱۷ مقایسه سطوح ارزیابی انرژی ۶۴
۴-۱۸ خلاصه نتایج ۶۴

فهرست جداول

۲-۱ مقایسه دوربین فریم‌محور و رویدادمحور ۱۳
۲-۲ سه نسل شبکه عصبی ۱۵
۴-۱ ماتریس مدل‌ها ۳۳
۴-۲ تنظیمات آموزش ۳۸
۴-۳ منابع محاسباتی مدل‌ها ۴۷
۴-۴ فعالیت اسپایکی ۴۹
۴-۵ محاسبات موثر ۵۰
۴-۶ تاخیر GPU ۵۲
۴-۷ توان، انرژی و EDP روی GPU ۵۵
۴-۸ انرژی تحلیلی ۵۸
۴-۹ مولفه‌های انرژی تحلیلی ۵۹
۴-۱۰ نتایج SANA-FE ۶۲

فصل اول

مقدمه

۱-۱ انگیزه و طرح مسئله

بینایی ماشین امروزه بخش جدایی‌ناپذیر بسیاری از سامانه‌های خودکار[۱] است: از خودروهای خودران و ربات‌ها گرفته تا پهپادها و سامانه‌های بازرسی صنعتی [۱]. آنچه در بسیاری از این کاربردها مشترک است، نیاز به اجرای بی‌درنگ روی یک سامانه‌ی نهفته با بودجه‌ی محدود توان، حافظه و زمان است. شبکه‌های عصبی پیچشی[۲] که امروزه ستون فقرات اغلب زنجیره‌های تشخیص اشیا هستند، معمولا برای اجرا روی دوربین‌های متعارف با نرخ فریم ثابت طراحی شده‌اند؛ رویکردی که صرف‌نظر از میزان تغییر واقعی صحنه، حجمی تقریبا ثابت از محاسبات را برای پردازش هر فریم تحمیل می‌کند.

دوربین‌های رویدادمحور[۳] و شبکه‌های عصبی اسپایکی[۴]، با الهام از شبکیه‌ی چشم و نورون زیستی[۵]، رویکردی متفاوت پیشنهاد می‌کنند: تولید داده و انجام محاسبه فقط در صورت وجود تغییر یا فعالیت [۲]. این ویژگی، از نظر مفهومی، این دو فناوری را برای کاربردهای نهفته و بی‌درنگ که در بالا برشمرده شد بسیار جذاب می‌سازد، زیرا انتظار می‌رود کاهش تعداد عملیات مؤثر به کاهش مصرف انرژی و تأخیر اجرا منجر شود.

۱-۲ شکاف میان مزیت نظری و مزیت عملی

جذابیت مفهومی محاسبات رویدادمحور به‌تنهایی تضمین نمی‌کند که این مزیت در عمل و روی یک سکوی محاسباتی[۶] واقعی نیز محقق شود. چندین پژوهش پیشین به‌طور مستقل به این نکته اشاره کرده‌اند که کاهش تعداد عملیات موثر یک شبکه‌ی عصبی اسپایکی، شرط لازم برای کارایی انرژی بیشتر است، اما شرط کافی نیست؛ نتیجه‌ی نهایی به شدت به سخت‌افزار زیرین و توانایی آن برای بهره‌برداری واقعی از تنکی[۷] فعالیت وابسته است [۳، ۴، ۵]. به‌طور مشخص، اجرای یک شبکه‌ی عصبی اسپایکی به‌صورت متراکم روی یک GPU متعارف، که اساسا برای محاسبات موازی و متراکم بهینه شده است، لزوما به کاهش زمان اجرا یا انرژی مصرفی منجر نمی‌شود، حتی اگر همان شبکه از نظر تعداد عملیات

[۱] Autonomous Systems
[۲] Convolutional Neural Network (CNN)
[۳] Event Camera
[۴] Spiking Neural Network (SNN)
[۵] Biological Neuron
[۶] Computing Platform
[۷] Sparsity

فعال بسیار کاراتر از مدل مصنوعی متناظرش باشد.

این پروژه دقیقا با هدف بررسی همین شکاف شکل گرفته است: شکاف میان آنچه یک معماری رویدادمحور و اسپایکی روی کاغذ و از نظر شمارش عملیات وعده می‌دهد و آنچه همان معماری، وقتی روی سکوهای محاسباتی واقعی و در دسترس اجرا می‌شود، از خود نشان می‌دهد.

۱-۳ هدف، دامنه و کار انجام‌شده در پروژه

هدف این پروژه بررسی این موضوع است که مزیت نظری پردازش رویدادمحور و اسپایکی، که از کاهش فعالیت و محاسبات غیرضروری ناشی می‌شود، تا چه اندازه در اجرای واقعی به کاهش هزینه‌های محاسباتی منجر می‌شود. به طور مشخص، این پروژه بررسی می‌کند که وقتی یک زنجیره‌ی بینایی ماشین مبتنی بر داده‌ی رویدادمحور و شبکه‌ی عصبی اسپایکی، به‌جای یک زنجیره‌ی متعارف مبتنی بر فریم و شبکه‌ی عصبی مصنوعی[۸] برای یک کاربرد یکسان تشخیص اشیا[۹] به کار گرفته شود، چه تفاوتی از نظر حافظه، حجم محاسبات، تأخیر اجرا و انرژی مصرفی مشاهده خواهد شد.

برای پاسخ به این پرسش، مجموعه‌ای از ۱۸ مدل متناظر از دو خانواده‌ی معماری ST-VGG و ResCat طراحی و پیاده‌سازی شد. برای هر خانواده، سه مقیاس کانالی ۱۶، ۳۲ و ۶۴ و برای هر مقیاس سه نسخه‌ی Event-SNN، Strict RGB-ANN و ANN-native RGB در نظر گرفته شد. مدل‌های متناظر تا حد امکان از نظر ساختار، عمق و ظرفیت یکسان نگه داشته شدند تا تفاوت‌های مشاهده‌شده عمدتاً به نوع ورودی و ماهیت محاسبات اسپایکی یا متراکم نسبت داده شوند. ارزیابی روی صحنه‌های یکسان از مجموعه‌داده‌ی DSEC انجام شد و اجرای مستقیم مدل‌ها با PyTorch/CUDA روی پردازنده‌ی گرافیکی NVIDIA Tesla T4 مورد سنجش قرار گرفت. علاوه بر اندازه‌گیری مستقیم روی GPU، مدل تحلیلی انرژی Lemaire [۶] و برآورد نگاشت نورومورفیک[۱۰] با SANA-FE [۷] نیز برای بررسی رفتار مدل‌ها در سطوح مکمل ارزیابی به کار گرفته شدند.

تمرکز این پروژه بر چهار معیار اصلی شامل اندازه‌ی مدل و هزینه‌ی حافظه، حجم محاسبات مؤثر، تأخیر اجرا و انرژی مصرفی است. دقت تشخیص به‌صورت مقایسه‌ای میان زنجیره‌ی رویدادمحور و فریم‌محور ارزیابی نمی‌شود، زیرا یک مقایسه‌ی معتبر از این نظر به آموزش کامل مدل‌ها تا همگرایی،

[۸] Artificial Neural Network (ANN)
[۹] Object Detection
[۱۰] Neuromorphic Mapping

تنظیم ابرپارامترها و ارزیابی روی داده‌ی مستقل نیاز دارد که خارج از دامنه‌ی این پروژه است؛ این موضوع در بخش ۳-۳ با جزئیات بیشتری توضیح داده می‌شود.

نتایج نشان دادند که با در نظر گرفتن فعالیت واقعی نورون‌های اسپایکی، بار محاسباتی مؤثر مدل‌های Event-SNN نسبت به نسخه‌های متراکم متناظر حدود ۷۴ تا ۹۳ درصد کاهش می‌یابد. با این حال، این کاهش روی NVIDIA Tesla T4 به کاهش تأخیر یا انرژی منجر نشد و مدل‌های اسپایکی در اجرای متراکم CUDA/PyTorch عموماً کندتر و از نظر انرژی پرهزینه‌تر بودند. در مقابل، مدل تحلیلی انرژی و برآورد نورومورفیک نشان دادند که مزیت محاسبات اسپایکی زمانی بهتر آشکار می‌شود که بستر سخت‌افزاری بتواند به‌طور مستقیم از تنکی فعالیت و ماهیت رویدادمحور محاسبات بهره‌برداری کند. بنابراین، نتیجه‌ی اصلی پروژه آن است که کارایی عملی پردازش اسپایکی تنها به معماری الگوریتم وابسته نیست و به نحوه‌ی تطبیق الگوریتم با سخت‌افزار اجرا نیز بستگی دارد.

۱-۴ ساختار پایان‌نامه

ادامه‌ی این پایان‌نامه در چهار فصل سازمان‌دهی شده است. فصل ۲ مبانی و پیشینه‌ی بینایی ماشین فریم‌محور، رویدادمحور و شبکه‌های عصبی اسپایکی را مرور می‌کند. فصل ۳ چارچوب مفهومی و اصول طراحی مقایسه، شامل برابری ساختاری مدل‌ها و استفاده از داده‌های متناظر، را توضیح می‌دهد. فصل ۴ روش پیاده‌سازی، مجموعه‌داده، محیط ارزیابی، معیارهای اندازه‌گیری و نتایج تجربی و تحلیلی را ارائه می‌کند. در نهایت، فصل ۵ نتایج اصلی پروژه را جمع‌بندی کرده و پیشنهادهایی برای کارهای آینده ارائه می‌دهد.

فصل دوم

مبانی و پیشینه بینایی ماشین مبتنی بر فریم و رویداد

۲-۱ مقدمه

هدف این فصل، فراهم کردن زمینه‌ی مفهومی و فنی لازم برای درک مسئله و موضوع پروژه می‌باشد. برای این منظور، ابتدا بینایی ماشین به‌عنوان یک حوزه‌ی کاربردی معرفی می‌شود و کاربردهایی که پیشران توسعه‌ی آن بوده‌اند مرور می‌شوند. سپس، سیر تحول شبکه‌های عصبی در بینایی ماشین از روش‌های مبتنی بر ویژگی‌های دستی تا شبکه‌های پیچشی عمیق و مدل‌های تشخیص اشیای مبتنی بر فریم دنبال می‌شود. در ادامه، محدودیت‌های ذاتی رویکرد فریم‌محور در کاربردهای بی‌درنگ بررسی و زمینه برای معرفی دوربین‌های رویدادمحور فراهم می‌شود؛ دوربین‌هایی که الهام آن‌ها از شبکیه‌ی چشم و اصول کارکردشان به‌تفصیل شرح داده می‌شوند. در کنار حسگر رویدادمحور، شبکه‌های عصبی اسپایکی به‌عنوان مدل محاسباتی متناظر معرفی می‌شوند؛ مدل‌هایی که الهام آن‌ها از فیزیولوژی نورون زیستی است و ماهیت زمانی و رویدادمحور محاسبات را با حسگر رویدادمحور هم‌راستا می‌کنند. بخشی از این فصل به‌طور مشخص به مفهوم استنتاج زمانی اختصاص دارد؛ یعنی به این پرسش که چگونه زمان در این مدل‌ها، نه یک بعد جانبی، بلکه بخشی از خود محاسبات است. در پایان، بینایی ماشین در سامانه‌های نهفته بی‌درنگ و محدودیت‌های محاسباتی، حافظه‌ای و انرژی این سامانه‌ها بررسی می‌شود و دلایل مطرح‌شده در پیشینه‌ی پژوهش برای مناسب بودن دوربین‌های رویدادمحور و شبکه‌های عصبی اسپایکی برای این‌گونه سامانه‌ها ارائه می‌شود. سرانجام، پیش از جمع‌بندی فصل، پیشینه‌ی پژوهشی که مستقیما با موضوع این پروژه در ارتباط است در سه محور مرور می‌شود: مدل‌های تشخیص اشیا روی داده‌ی رویدادمحور، مطالعات مقایسه‌ی کارایی انرژی شبکه‌های عصبی اسپایکی و مصنوعی و سخت‌افزار نورومورفیک[۱] به همراه تلاش‌های اخیر برای معیارسنجی آن.

۲-۲ بینایی ماشین و کاربردهای آن

بینایی ماشین به مجموعه‌ای از روش‌ها گفته می‌شود که به یک سامانه‌ی محاسباتی امکان می‌دهند از روی داده‌ی تصویری، اطلاعات معنادار درباره‌ی صحنه، اشیا یا رویدادهای موجود در آن استخراج کند [۱]. این حوزه از دهه‌های آغازین علوم رایانه با تلاش برای بازسازی هندسی صحنه از تصویر آغاز شد

[۱] Neuromorphic Hardware

و به‌تدریج به سمت مسائل ادراکی سطح بالاتر مانند طبقه‌بندی[۲]، تشخیص اشیا، دنبال‌کردن[۳] و درک صحنه[۴] گسترش یافت. آنچه بینایی ماشین را نسبت به بسیاری از دیگر حوزه‌های پردازش داده متمایز می‌کند، حجم بالا و ابعاد زیاد داده‌ی ورودی است: حتی یک تصویر معمولی شامل صدها هزار پیکسل است که بخش زیادی از اطلاعات آن برای یک کاربرد مشخص زائد یا تکراری است.

کاربردهای بینایی ماشین در طول زمان از آزمایشگاه به محیط واقعی منتقل شده‌اند و امروزه بخشی جدایی‌ناپذیر از بسیاری از سامانه‌های خودکار هستند. در خودروهای خودران و سامانه‌های کمک‌راننده، بینایی ماشین برای تشخیص خودروها، عابران پیاده، علائم راهنمایی و خطوط جاده به‌کار می‌رود و مستقیما بر ایمنی سرنشینان اثر دارد. در رباتیک، تشخیص و مکان‌یابی اشیا، ناوبری و دستکاری اشیا وابسته به بینایی ماشین است. در حوزه‌ی صنعتی، بازرسی بصری خط تولید و تشخیص عیوب سطحی با دقت و سرعتی انجام می‌شود که برای بازرسی انسانی دشوار است. در حوزه‌ی پزشکی، تحلیل تصاویر پزشکی به تشخیص زودهنگام بیماری‌ها کمک می‌کند. در امنیت و نظارت، تشخیص و ردیابی اشیا و افراد در تصاویر دوربین‌های نظارتی انجام می‌شود. در واقعیت افزوده و واقعیت مجازی، بینایی ماشین مبنای درک محیط و ردیابی حرکت کاربر است. در پهپادها و سامانه‌های پرنده‌ی خودکار نیز بینایی ماشین برای اجتناب از مانع و ناوبری به‌کار می‌رود.

آنچه در بسیاری از این کاربردها مشترک است، نیاز به پردازش بی‌درنگ یا نزدیک به بی‌درنگ روی سکوی محاسباتی محدود است. برخلاف کاربردهایی مانند طبقه‌بندی آفلاین یک مجموعه‌داده‌ی ثابت، در کاربردهایی مانند خودروی خودران یا پهپاد، تاخیر میان دریافت تصویر و تصمیم‌گیری مستقیما بر عملکرد و ایمنی سامانه اثر می‌گذارد و سکوی اجرا به جای یک مرکز داده با منابع تقریبا نامحدود، اغلب یک سامانه‌ی نهفته با محدودیت توان، حافظه و گرما است.

---

[۲] Classification

[۳] Tracking

[۴] Scene Understanding

۲-۳ شبکه‌های عصبی برای بینایی ماشین: از ویژگی‌های دستی تا یادگیری عمیق

پیش از فراگیر شدن یادگیری عمیق[۵]، بیشتر روش‌های بینایی ماشین بر استخراج ویژگی‌های دستی[۶] استوار بودند؛ ویژگی‌هایی مانند لبه‌ها، گوشه‌ها و بافت که با الگوریتم‌های ثابت از تصویر استخراج و سپس به یک مدل طبقه‌بندی سنتی داده می‌شدند. این رویکرد در بسیاری از مسائل ساده عملکرد قابل قبولی داشت، اما در برابر تغییرات روشنایی، زاویه‌ی دید و شلوغی صحنه شکننده بود، زیرا ویژگی‌های دستی نمی‌توانستند تمام تنوع داده‌ی واقعی را پوشش دهند.

نقطه‌ی عطف این حوزه، معرفی شبکه‌های عصبی پیچشی بود. LeCun و همکاران نشان دادند که یک شبکه‌ی عصبی پیچشی می‌تواند مستقیما از روی پیکسل‌های خام، ویژگی‌های لازم برای تشخیص را یاد بگیرد، بدون آنکه این ویژگی‌ها از پیش به‌صورت دستی طراحی شوند [۸]. ایده‌ی اصلی کانولوشن، اشتراک وزن میان موقعیت‌های مختلف تصویر و بهره‌گیری از ساختار مکانی محلی داده‌ی تصویری است؛ به این ترتیب، تعداد پارامترهای شبکه به‌شدت کاهش می‌یابد و شبکه نسبت به جابه‌جایی الگوها در تصویر تغییرناپذیر می‌شود. با این حال، به دلیل محدودیت داده و توان محاسباتی در دهه‌های بعد، این ایده تا مدتی در مقیاس کوچک باقی ماند.

جهش بعدی با در دسترس قرار گرفتن مجموعه‌داده‌های با مقیاس بزرگ و شتاب‌دهنده‌های محاسباتی گرافیکی رخ داد. Krizhevsky و همکاران با معرفی شبکه‌ای موسوم به AlexNet نشان دادند که یک شبکه‌ی پیچشی عمیق، با آموزش روی میلیون‌ها تصویر و اجرا روی GPU، می‌تواند به‌طور چشمگیری از روش‌های مبتنی بر ویژگی دستی در طبقه‌بندی تصویر پیشی بگیرد [۹]. این نتیجه، آغازگر موجی از پژوهش در معماری‌های عمیق‌تر بود که در سال‌های بعد با سرعت زیادی ادامه یافت. Simonyan و Zisserman با معرفی خانواده‌ی VGG نشان دادند که ساده نگه داشتن بلوک‌های پیچشی (فیلترهای کوچک ۳ × ۳) و افزایش تدریجی عمق شبکه می‌تواند دقت را به‌طور پیوسته بهبود دهد [۱۰]. تقریبا همزمان، معماری‌هایی مانند GoogLeNet/Inception با ترکیب فیلترهایی در چند مقیاس مختلف درون یک بلوک واحد، مسیر دیگری برای افزایش ظرفیت شبکه بدون رشد بی‌رویه‌ی تعداد پارامترها ارائه دادند

[۵] Deep Learning
[۶] Hand-crafted Features

[۱۱]. با عمیق‌تر شدن هرچه بیشتر شبکه‌ها، مسئله‌ی محو گرادیان[۷] به مانعی جدی برای آموزش تبدیل شد؛ He و همکاران با معرفی اتصال‌های باقی‌مانده[۸] در معماری ResNet نشان دادند که با عبور دادن یک مسیر میان‌بر از ورودی هر بلوک به خروجی آن، می‌توان شبکه‌هایی با ده‌ها یا حتی صدها لایه را بدون افت عملکرد آموزش داد [۱۲]. در کنار این خط اصلی، رویکردهای دیگری مانند اتصال چگال میان لایه‌ها یا الحاق کانالی مسیرهای ویژگی نیز برای بهبود جریان اطلاعات و گرادیان در شبکه‌های عمیق پیشنهاد شده‌اند.

آنچه این خانواده از مدل‌ها را برای بینایی ماشین مناسب می‌کند، سه ویژگی ساختاری مشترک است: اشتراک وزن[۹] که تعداد پارامترها را نسبت به یک شبکه‌ی کاملا متصل[۱۰] به‌شدت کاهش می‌دهد، میدان دریافتی[۱۱] محلی که با عمیق‌تر شدن شبکه به‌تدریج بزرگ‌تر می‌شود و امکان یادگیری ویژگی‌های سلسله‌مراتبی، از لبه‌ها و بافت‌های ساده در لایه‌های ابتدایی تا الگوهای معنایی پیچیده‌تر در لایه‌های عمیق‌تر، را فراهم می‌کند. این اصول ساختاری، صرف‌نظر از جزئیات هر معماری خاص، مبنای تقریبا تمام شبکه‌های پیچشی مورد استفاده در بینایی ماشین امروزی است.

۲-۴ از طبقه‌بندی تصویر تا تشخیص اشیا: مدل‌های مبتنی بر فریم

طبقه‌بندی تصویر، که در آن یک برچسب کلی به کل تصویر نسبت داده می‌شود، تنها گام نخست بود. بسیاری از کاربردهای واقعی بینایی ماشین، از جمله همان کاربردهایی که در بخش ۲-۲ برشمرده شد، به تشخیص اشیا نیاز دارند؛ یعنی یافتن موقعیت مکانی هر شیء در تصویر (معمولا به‌صورت یک کادر محصورکننده[۱۲]) و طبقه‌بندی همزمان آن.

نخستین نسل مدل‌های یادگیری عمیق برای تشخیص اشیا، مانند R-CNN، مسئله را به دو مرحله تقسیم می‌کردند: نخست، تولید تعدادی ناحیه‌ی پیشنهادی[۱۳] که احتمال وجود شیء در آن‌ها بالاست و

[۷] Vanishing Gradient
[۸] Residual Connections
[۹] Weight Sharing
[۱۰] Fully Connected
[۱۱] Receptive Field
[۱۲] Bounding Box
[۱۳] Region Proposal

سپس طبقه‌بندی هر ناحیه با یک شبکه‌ی پیچشی جداگانه [۱۳]. این رویکرد دقت قابل قبولی داشت، اما به دلیل پردازش جداگانه‌ی هر ناحیه‌ی پیشنهادی، از نظر سرعت برای کاربردهای بی‌درنگ مناسب نبود.

نسل بعدی مدل‌ها با هدف اجرای بی‌درنگ طراحی شدند و تشخیص را در یک گذر واحد از شبکه انجام می‌دادند. Redmon و همکاران با معرفی YOLO (You Only Look Once) نشان دادند که می‌توان تشخیص اشیا را به‌صورت یک مسئله‌ی رگرسیون واحد صورت‌بندی کرد که به‌طور مستقیم مختصات کادرها و برچسب کلاس را از روی کل تصویر پیش‌بینی می‌کند [۱۴]. در همین راستا، Liu و همکاران معماری SSD (Single Shot MultiBox Detector) را معرفی کردند [۱۵]. ایده‌ی اصلی SSD استفاده از مجموعه‌ای از کادرهای پیش‌فرض[۱۴] در مقیاس‌ها و نسبت‌های ابعاد مختلف است که روی چند نقشه‌ی ویژگی[۱۵] با تفکیک‌پذیری متفاوت اعمال می‌شوند: نقشه‌های ویژگی با تفکیک‌پذیری بالاتر برای تشخیص اشیای کوچک و نقشه‌های عمیق‌تر با تفکیک‌پذیری پایین‌تر برای اشیای بزرگ‌تر مناسب‌اند. برای هر کادر پیش‌فرض، شبکه به‌طور همزمان احتمال تعلق به هر کلاس و تصحیح مختصات کادر را پیش‌بینی می‌کند و در نتیجه نیازی به مرحله‌ی جداگانه‌ی تولید ناحیه‌ی پیشنهادی نیست.

اشتراک تمام این مدل‌ها آن است که ورودی، یک فریم ثابت با ابعاد و کانال‌های مشخص (معمولا سه کانال RGB) است و کل تصویر، مستقل از فریم‌های پیشین، در یک گذر رو به جلو پردازش می‌شود. این فرض، یعنی استقلال هر فریم از فریم‌های قبلی، مبنای اغلب معماری‌های تشخیص اشیای مبتنی بر فریم است و در بخش ۲-۸ با رویکرد رویدادمحور و اسپایکی مقایسه خواهد شد.

۲-۵ محدودیت‌های بینایی مبتنی بر فریم برای پردازش بی‌درنگ

با وجود پیشرفت چشمگیر مدل‌های فریم‌محور، ماهیت خود دوربین‌های متعارف چند محدودیت ساختاری برای کاربردهای بی‌درنگ و نهفته ایجاد می‌کند. نخست، دوربین‌های متعارف تصویر را با نرخ فریم[۱۶] ثابت (برای مثال ۳۰ یا ۶۰ فریم بر ثانیه) ثبت می‌کنند، صرف‌نظر از آنکه صحنه در حال تغییر باشد یا کاملا ساکن. در بسیاری از صحنه‌های واقعی، بخش بزرگی از پیکسل‌های دو فریم متوالی تغییر معناداری نمی‌کنند و پردازش کامل هر فریم به معنای انجام محاسبات برای اطلاعات تکراری است.

[۱۴] Default/Anchor Boxes
[۱۵] Feature Map
[۱۶] Frame Rate

دوم، فاصله‌ی زمانی ثابت میان دو فریم متوالی یک کران پایین برای تاخیر ادراکی سامانه ایجاد می‌کند: رویدادی که بلافاصله پس از ثبت یک فریم رخ دهد، تا فریم بعدی از دید سامانه پنهان می‌ماند. در صحنه‌های با حرکت سریع، همین فاصله‌ی زمانی می‌تواند به تاری حرکتی[۱۷] نیز منجر شود، زیرا در طول زمان باز بودن شاتر دوربین، شیء در حال حرکت جابه‌جا می‌شود. سوم، دوربین‌های متعارف در شرایط نوری چالش‌برانگیز، مانند تغییر ناگهانی روشنایی یا صحنه‌های با دامنه‌ی دینامیکی[۱۸] بالا (به‌طور همزمان دارای نواحی بسیار روشن و بسیار تاریک)، کیفیت کافی ارائه نمی‌دهند، زیرا دامنه‌ی دینامیکی حسگرهای متعارف معمولا محدود به چند ده دسی‌بل است.

چهارم، پردازش هر فریم کامل با یک شبکه‌ی عصبی مصنوعی متعارف، صرف‌نظر از میزان تغییر واقعی صحنه، به مقدار تقریبا ثابتی از عملیات ضرب‌وجمع نیاز دارد. این حجم محاسبات برای اجرا روی سکوی محاسباتی با منابع نامحدود مسئله‌ساز نیست، اما در سامانه‌های نهفته با بودجه‌ی توان محدود، هزینه‌ی انرژی و تاخیر قابل توجهی تحمیل می‌کند. این محدودیت‌ها، به‌ویژه ترکیب تاخیر ذاتی نرخ فریم ثابت و هزینه‌ی محاسباتی مستقل از میزان تغییر صحنه، انگیزه‌ی اصلی توسعه‌ی حسگرهای رویدادمحور بوده است که در بخش بعدی معرفی می‌شوند.

۲-۶ دوربین‌های رویدادمحور: الهام زیستی و اصول کارکرد

۲-۶-۱ الهام از شبکیه‌ی زیستی

طراحی دوربین‌های رویدادمحور مستقیما از نحوه‌ی کارکرد شبکیه‌ی چشم زیستی الهام گرفته است. برخلاف یک حسگر تصویر متعارف که در بازه‌های زمانی منظم از تمام پیکسل‌ها نمونه‌برداری می‌کند، شبکیه‌ی زیستی عمدتا به تغییرات روشنایی حساس است: گیرنده‌های نوری و مدارهای عصبی شبکیه به‌جای گزارش مقدار مطلق روشنایی در هر لحظه، به تغییرات نسبی آن پاسخ می‌دهند و این پاسخ به‌صورت ناهمگام و مستقل برای هر ناحیه از میدان دید اتفاق می‌افتد. حسگرهای رویدادمحور، که گاه با عنوان شبکیه‌ی سیلیکونی نیز شناخته می‌شوند، همین اصل را در سخت‌افزار پیاده‌سازی می‌کنند [۲].

[۱۷] Motion Blur
[۱۸] Dynamic Range

۲-۶-۲ اصول کارکرد حسگر رویدادمحور

در متداول‌ترین نوع حسگر رویدادمحور، موسوم به حسگر بینایی پویا[۱۹]، هر پیکسل به‌طور مستقل و ناهمگام لگاریتم شدت روشنایی برخوردی به خود را زیر نظر دارد. هرگاه تغییر این مقدار لگاریتمی از یک آستانه‌ی از پیش تعیین‌شده فراتر رود، همان پیکسل بلافاصله یک رویداد تولید می‌کند، بدون آنکه منتظر یک سیگنال ساعت مرکزی یا نمونه‌برداری همزمان با سایر پیکسل‌ها باشد. Lichtsteiner، Posch و Delbruck یکی از نخستین پیاده‌سازی‌های عملی و پرکاربرد این اصل را ارائه کردند و نشان دادند که چنین حسگری می‌تواند تاخیری در حد میکروثانیه و دامنه‌ی دینامیکی بیش از ۱۲۰ دسی‌بل ارائه دهد، در حالی که فقط برای نواحی در حال تغییر صحنه داده تولید می‌کند [۱۶].

هر رویداد را می‌توان به‌صورت یک چهارتایی

$$e = (x, y, t, p) \tag{۱-۲}$$

نمایش داد که در آن x و y مختصات مکانی پیکسل، t زمان دقیق وقوع تغییر روشنایی و p قطبیت تغییر (افزایش یا کاهش روشنایی) را نشان می‌دهند. این نمایش با نام نمایش رویداد-آدرسی[۲۰] نیز شناخته می‌شود، زیرا هر رویداد را می‌توان یک بسته‌ی داده حاوی آدرس مکانی و برچسب زمانی در نظر گرفت.

سه ویژگی، حسگرهای رویدادمحور را از دوربین‌های متعارف متمایز می‌کند و مبنای اغلب کاربردهای آن‌ها است. نخست، تفکیک‌پذیری زمانی[۲۱] بسیار بالا؛ از آنجا که هر پیکسل مستقل از سایرین و بلافاصله پس از عبور از آستانه رویداد تولید می‌کند، محدودیت نرخ فریم ثابت وجود ندارد. دوم، دامنه‌ی دینامیکی بسیار بالاتر از دوربین متعارف، که امکان کارکرد مناسب در نور بسیار کم یا بسیار زیاد را فراهم می‌کند. سوم، تنکی ذاتی خروجی: در نواحی ساکن صحنه، هیچ رویدادی تولید نمی‌شود و حجم داده‌ی خروجی مستقیما با میزان تغییر صحنه متناسب است، برخلاف دوربین متعارف که حجم داده‌ی آن مستقل از محتوای صحنه و تنها تابع ابعاد تصویر است.

جدول ۲-۱ این سه ویژگی را به‌صورت خلاصه در کنار محدودیت‌های دوربین متعارف که پیش‌تر

[۱۹] Dynamic Vision Sensor (DVS)
[۲۰] Address-Event Representation
[۲۱] Temporal Resolution

در بخش ۲-۵ بررسی شد، قرار می‌دهد. همان‌طور که از این مقایسه پیداست، تفاوت اصلی دو حسگر تنها در نحوه‌ی نمونه‌برداری از صحنه نیست، بلکه در نوع رابطه‌ی میان محتوای صحنه و حجم داده‌ی تولیدشده نهفته است: در دوربین متعارف این رابطه وجود ندارد و حجم داده تنها تابع ابعاد تصویر و نرخ فریم است، در حالی که در دوربین رویدادمحور حجم داده مستقیما با میزان تغییر صحنه متناسب می‌شود.

جدول ۲-۱: مقایسه‌ی مفهومی ویژگی‌های دوربین فریم‌محور متعارف و دوربین رویدادمحور

ویژگی	دوربین فریم‌محور	دوربین رویدادمحور
واحد خروجی	تصویر کامل با نرخ ثابت	رویداد ناهمگام در سطح پیکسل
تفکیک‌پذیری زمانی	محدود به نرخ فریم (ms)	در حد میکروثانیه
دامنه‌ی دینامیکی	معمولا حدود ۶۰ dB	می‌تواند به ۱۲۰ dB برسد [۱۶]
حجم داده در صحنه‌ی ساکن	ثابت و مستقل از تغییر صحنه	نزدیک به صفر
تاری حرکتی	محتمل در حرکت سریع	به دلیل ماهیت ناهمگام، کمتر رخ می‌دهد

۲-۶-۳ کاربردهای بینایی رویدادمحور

تفکیک‌پذیری زمانی بسیار بالا، دامنه‌ی دینامیکی گسترده و تنکی ذاتی داده‌های خروجی باعث شده‌اند دوربین‌های رویدادمحور در کاربردهایی با نیاز به سرعت بالا و منابع محدود مورد توجه قرار گیرند: ردیابی اشیای سریع در رباتیک، تخمین جریان نوری[۲۲] با تاخیر پایین، ناوبری بصری در رباتیک هوایی و به‌ویژه رانندگی خودکار، جایی که ترکیب سرعت بالای صحنه و نیاز به واکنش سریع اهمیت دارد. مجموعه‌داده‌های بسیاری برای پژوهش در زمینه‌ی دوربین رویدادمحور طراحی شده‌اند و نشان می‌دهند که این حسگر می‌تواند در کاربردهای بی‌درنگ با تاخیر بسیار پایین به‌کار رود [۱۷].

با این حال، بهره‌برداری کامل از تنکی و ماهیت ناهمگام رویدادها به یک مدل محاسباتی نیاز دارد که خود نیز به‌طور طبیعی رویدادمحور باشد. شبکه‌های عصبی مصنوعی متعارف، که برای ورودی‌های فریم‌محور و متراکم طراحی شده‌اند، برای بهره‌گیری از این ویژگی مناسب نیستند؛ زیرا این شبکه‌ها صرف‌نظر از تعداد رویدادهای فعال، حجم ثابتی از محاسبات را روی کل ورودی انجام می‌دهند. این عدم تطابق، انگیزه‌ی معرفی شبکه‌های عصبی اسپایکی به‌عنوان مدل محاسباتی متناظر با حسگر رویدادمحور است.

[۲۲] Optical Flow

۲-۷ شبکه‌های عصبی اسپایکی: الهام از سیستم عصبی زیستی

۲-۷-۱ نورون زیستی و نمایش اسپایکی

در سیستم عصبی زیستی، نورون‌ها اطلاعات را عمدتا از طریق پتانسیل عمل[۲۳]، یا اسپایک، مبادله می‌کنند: یک نوسان کوتاه و گذرا در ولتاژ غشای سلول عصبی که در یک لحظه‌ی مشخص رخ می‌دهد و در طول آکسون به نورون‌های بعدی منتقل می‌شود. تفاوت بنیادین این نمایش با نمایش عددی پیوسته‌ای که در شبکه‌های عصبی مصنوعی متعارف به‌کار می‌رود این است که اطلاعات، به‌جای یک مقدار پیوسته، عمدتا در زمان وقوع و فراوانی این رویدادهای گسسته نهفته است. نورون‌های زیستی تنها در صورت رسیدن به یک شرایط مشخص اسپایک تولید می‌کنند و در سایر لحظات هیچ خروجی محاسباتی یا مخابراتی ندارند؛ همین ویژگی، مبنای کارایی انرژی چشمگیر مغز زیستی نسبت به سخت‌افزار محاسباتی متعارف دانسته می‌شود.

۲-۷-۲ از نسل‌های شبکه‌ی عصبی تا شبکه‌های اسپایکی

Maass در دسته‌بندی مشهور خود، تکامل مدل‌های شبکه‌ی عصبی مصنوعی را در سه نسل توصیف کرده است [۱۸]. نسل نخست، شبکه‌های مبتنی بر واحدهای آستانه‌ای مانند پرسپترون[۲۴] هستند که خروجی دودویی و ساده‌ای تولید می‌کنند. نسل دوم، شبکه‌های عصبی مصنوعی متعارفی هستند که در آن‌ها از توابع فعال‌سازی[۲۵] پیوسته و مشتق‌پذیر مانند sigmoid یا ReLU استفاده می‌شود؛ همین نسل، مبنای موفقیت‌های یادگیری عمیق است که در بخش ۲-۳ مرور شد. نسل سوم، شبکه‌های عصبی اسپایکی هستند که واحدهای محاسباتی آن‌ها اطلاعات را با اسپایک‌های گسسته و متمرکز در زمان مبادله می‌کنند و از این نظر به رفتار محاسباتی نورون زیستی نزدیک‌تر هستند. Maass نشان داد که این نسل از لحاظ نظری قدرت محاسباتی معادل یا بیشتر از نسل‌های پیشین دارد، به‌ویژه به این دلیل که می‌تواند از زمان دقیق وقوع اسپایک، و نه صرفا نرخ آن، برای نمایش اطلاعات بهره ببرد [۱۸].

[۲۳] Action Potential
[۲۴] Perceptron
[۲۵] Activation Function

جدول ۲-۲: سه نسل مدل شبکه عصبی بر اساس دسته‌بندی Maass [۱۸]

نسل	واحد محاسباتی	نمونه
اول	آستانه‌ی دودویی	پرسپترون
دوم	فعال‌سازی پیوسته و مشتق‌پذیر	CNNهای متعارف
سوم	نورون اسپایکی با حالت زمانی	SNN

۲-۷-۳ مدل نورون LIF و پویایی زمانی

رایج‌ترین مدل ریاضی برای نورون اسپایکی، مدل Leaky Integrate-and-Fire (LIF) است. در این مدل، هر نورون یک پتانسیل غشایی[۲۶] داخلی دارد که در طول زمان جریان ورودی سیناپسی[۲۷] را در خود انباشت می‌کند؛ همزمان، بخشی از این پتانسیل، مشابه رفتار یک خازن در حال تخلیه، به‌مرور نشت می‌کند و به مقدار استراحت باز می‌گردد. هرگاه پتانسیل غشایی از یک آستانه‌ی شلیک[۲۸] عبور کند، نورون یک اسپایک تولید می‌کند و بلافاصله پس از آن، پتانسیل غشایی بازنشانی می‌شود.

آنچه در این‌جا اهمیت دارد، تفاوت مفهومی این مدل با یک نورون مصنوعی متعارف است. در یک نورون ReLU متعارف، خروجی در هر گام محاسباتی تنها تابعی از ورودی همان گام است و هیچ حافظه‌ای از گذشته حفظ نمی‌شود. در مقابل، پتانسیل غشایی نورون LIF به‌طور طبیعی اطلاعات ورودی‌های پیشین را در خود انباشت می‌کند و بنابراین نورون اسپایکی، حتی بدون داشتن اتصال بازگشتی صریح، نوعی حافظه‌ی زمانی کوتاه‌مدت درونی دارد.

از منظر آموزش، تابع تولید اسپایک (مقایسه با آستانه) در نقطه‌ی آستانه ناپیوسته و غیرقابل مشتق‌گیری است؛ همین ویژگی، به‌کارگیری مستقیم الگوریتم پس‌انتشار[۲۹] متعارف را دشوار می‌کند. Neftci و همکاران راه‌حل رایج این مسئله، یعنی استفاده از گرادیان جانشین[۳۰]، را معرفی و تحلیل کرده‌اند؛ در این روش، در گذر رو به جلو از تابع پله‌ای واقعی استفاده می‌شود، اما در گذر رو به عقب، مشتق یک تابع پیوسته‌ی تقریب‌زننده جایگزین می‌شود تا آموزش مبتنی بر گرادیان امکان‌پذیر شود [۱۹].

[۲۶] Membrane Potential
[۲۷] Synaptic Input
[۲۸] Firing Threshold
[۲۹] Backpropagation
[۳۰] Surrogate Gradient

۲-۷-۴ محاسبات رویدادمحور و تنکی فعالیت

مهم‌ترین ویژگی محاسباتی شبکه‌ی عصبی اسپایکی، از دید کارایی انرژی، تنکی فعالیت آن است. در یک شبکه‌ی عصبی مصنوعی متعارف، خروجی هر نورون در هر گام یک عدد ممیز شناور غیرصفر است و باید در محاسبه‌ی لایه‌ی بعد شرکت کند. در شبکه‌ی عصبی اسپایکی، خروجی هر نورون در هر گام زمانی صفر یا یک است؛ نورونی که اسپایک تولید نمی‌کند، عملا در محاسبات آن گام زمانی مشارکتی ندارد. اگر نرخ متوسط تولید اسپایک در شبکه پایین باشد، که در بسیاری از داده‌های رویدادمحور واقعی چنین است، تعداد عملیات موثر شبکه می‌تواند به‌طور قابل توجهی کمتر از یک شبکه‌ی مصنوعی متعارف با همان توپولوژی باشد. این ویژگی، دلیل اصلی هم‌راستایی مفهومی شبکه‌ی عصبی اسپایکی با حسگر رویدادمحور است: هر دو، در غیاب تغییر یا فعالیت، عملا هیچ کاری انجام نمی‌دهند.

۲-۸ استنتاج زمانی: زمان به‌مثابه بعد محاسباتی

یکی از تفاوت‌های مفهومی مهم میان زنجیره‌ی فریم‌محور و زنجیره‌ی رویدادمحور، نحوه‌ی برخورد با بعد زمان است. در مدل‌های تشخیص اشیای فریم‌محور که در بخش ۲-۴ معرفی شدند، هر فریم به‌صورت یک نمونه‌ی مستقل پردازش می‌شود؛ حتی اگر ورودی از یک توالی ویدیویی گرفته شده باشد، شبکه به‌خودی‌خود اطلاعاتی از فریم‌های پیشین در خود ذخیره نمی‌کند، مگر آنکه به‌صراحت یک مکانیزم بازگشتی یا حافظه‌ی زمانی جداگانه به معماری اضافه شود. به این ترتیب، استنتاج در این مدل‌ها اساسا مکانی است و بعد زمان صرفا به‌عنوان توالی‌ای از استنتاج‌های مستقل مکانی ظاهر می‌شود.

در مقابل، همان‌طور که در بخش ۲-۷ توضیح داده شد، پتانسیل غشایی نورون‌های LIF به‌طور طبیعی در طول زمان انباشته می‌شود و میان گام‌های زمانی متوالی حفظ می‌شود. این بدان معناست که خروجی شبکه‌ی عصبی اسپایکی در هر گام، نه‌تنها به ورودی همان گام، بلکه به تاریخچه‌ی ورودی‌های پیشین (میراشده با ثابت زمانی[۳۱] τ) نیز وابسته است. به بیان دیگر، زمان در این مدل‌ها یک بعد جانبی نیست که پس از استنتاج مکانی به آن پرداخته شود، بلکه بخشی درونی از خود مکانیزم محاسباتی نورون است. این ویژگی با ماهیت داده‌ی ورودی، یعنی جریان پیوسته‌ی رویدادهای دارای برچسب زمانی دقیق، هم‌خوانی طبیعی دارد؛ در حالی که تحمیل این جریان پیوسته بر یک مدل مکانی صرف، نیازمند گسسته‌سازی مصنوعی زمان به فریم‌های جداگانه است.

[۳۱] Time Constant

این گسسته‌سازی در عمل نیز اجتناب‌ناپذیر است: حتی در یک شبکه‌ی عصبی اسپایکی، رویدادهای پیوسته باید در پنجره‌های زمانی کوچک تجمیع شوند تا هر پنجره یک گام زمانی شبکه را تشکیل دهد.

۲-۹ بینایی ماشین در سامانه‌های نهفته بی‌درنگ

۲-۹-۱ محدودیت‌های سامانه‌های نهفته

بسیاری از کاربردهایی که در بخش ۲-۲ برشمرده شدند از جمله، پهپادها، ربات‌های سیار، خودروهای خودران، پوشیدنی‌ها و حسگرهای هوشمند، باید بینایی ماشین را نه روی یک مرکز داده، بلکه روی یک سامانه‌ی نهفته و در نزدیکی خود حسگر اجرا کنند. این سامانه‌ها معمولا با سه محدودیت همزمان مواجه‌اند: بودجه‌ی توان محدود (اغلب باتری‌محور)، حافظه‌ی محدود و نیاز به پاسخ‌گویی در مهلت‌های زمانی مشخص. Sze و همکاران در بررسی جامع خود از روش‌های کارای پردازش شبکه‌های عصبی عمیق نشان می‌دهند که اجرای مستقیم شبکه‌های عصبی مصنوعی متعارف، به‌ویژه مدل‌های عمیق و پرحجم مانند خانواده‌های معرفی‌شده در بخش ۲-۳، می‌تواند از نظر انرژی و حافظه هزینه‌بر باشد و اجرای کارا در سامانه‌ی نهفته را به یک مسئله‌ی طراحی جدی تبدیل می‌کند [۲۰]. بخش قابل توجهی از این هزینه نه از خود عملیات محاسباتی، بلکه از جابه‌جایی داده میان حافظه و واحد پردازش ناشی می‌شود؛ به همین دلیل، هر رویکردی که بتواند حجم داده‌ی فعال و تعداد عملیات موثر را کاهش دهد، پتانسیل صرفه‌جویی چشمگیر در انرژی مصرفی دارد.

۲-۹-۲ مزیت رویداد و اسپایک برای سامانه‌های نهفته

دوربین رویدادمحور و شبکه‌ی عصبی اسپایکی، همان‌طور که در بخش ۲-۶ و بخش ۲-۷ شرح داده شد، هر دو بر پایه‌ی یک اصل مشترک بنا شده‌اند: تولید داده یا انجام محاسبه تنها در صورت وجود تغییر یا فعالیت. این اصل، از دید طراحی سامانه‌ی نهفته، جذاب است، زیرا در بسیاری از کاربردهای واقعی (برای مثال، یک دوربین نظارتی ثابت یا یک ربات ساکن) بخش بزرگی از زمان، صحنه تغییر چندانی نمی‌کند و پردازش کامل و متراکم آن، اتلاف منابع محسوب می‌شود.

همین اصل، صرفا در سطح نرم‌افزار باقی نمانده و مبنای طراحی نسلی از سخت‌افزارهای موسوم به نورومورفیک نیز بوده است. برخلاف یک پردازنده‌ی گرافیکی متعارف، که در هر گام محاسباتی تمام

واحدهای پردازشی خود را به‌صورت همزمان و صرف‌نظر از فعال یا غیرفعال بودن نورون‌های متناظر به‌کار می‌گیرد، یک سخت‌افزار نورومورفیک می‌تواند محاسبه و انتقال داده را تنها برای نورونی که واقعا اسپایک تولید کرده انجام دهد و از صرف انرژی برای نورون‌های خاموش در همان گام زمانی اجتناب کند. به بیان دیگر، تنکی فعالیت اسپایکی، که در بخش ۲-۷ توضیح داده شد، در چنین سخت‌افزاری مستقیما به کاهش مصرف انرژی ترجمه می‌شود، در حالی که در یک سکوی محاسباتی همگام و متراکم لزوما چنین نیست. همین ویژگی، محاسبات مبتنی بر اسپایک روی سخت‌افزار نورومورفیک را مسیری امیدبخش برای رسیدن به هوش مصنوعی کاراتر از نظر انرژی ساخته است که به‌طور خاص برای سامانه‌های محدود از نظر منابع، از جمله سامانه‌های نهفته مناسب است [۲۱].

۲-۹-۳ از وعده‌ی نظری تا عملکرد واقعی

با این حال، باید میان دو سطح متفاوت تمایز قائل شد: کاهش نظری تعداد عملیات موثر در سطح مدل، و کاهش واقعی تاخیر یا انرژی در سطح اجرای سامانه. تنکی فعالیت اسپایکی، که در بخش ۲-۷ توضیح داده شد، تضمین می‌کند که تعداد عملیات ضرب‌وجمع موثر یک شبکه‌ی اسپایکی می‌تواند کمتر از یک شبکه‌ی مصنوعی متناظر باشد. اما این کاهش تنها زمانی به کاهش واقعی زمان اجرا یا انرژی مصرفی تبدیل می‌شود که سخت‌افزار زیرین بتواند از این تنکی و ماهیت ناهمگام بهره‌برداری کند؛ یعنی بتواند از انجام محاسبه و انتقال داده برای نورون‌های غیرفعال صرف‌نظر کند. یک پردازنده‌ی گرافیکی متعارف، که برای عملیات ماتریسی متراکم و همگام بهینه شده است، لزوما چنین قابلیتی ندارد و ممکن است شبکه‌ی اسپایکی را نیز به‌صورت متراکم اجرا کند، حتی اگر بخش بزرگی از نورون‌های آن در یک گام زمانی مشخص اسپایک تولید نکرده باشند. سخت‌افزار نورومورفیک، مانند TrueNorth [۲۲] یا Loihi [۲۳]، دقیقا با هدف رفع همین شکاف طراحی شده است، اما در دسترس بودن محدود و بلوغ کمتر ابزارهای توسعه‌ی نرم‌افزاری برای این سخت‌افزارها، مانع از ارزیابی مستقیم و گسترده‌ی آن در بسیاری از پژوهش‌ها می‌شود. به همین دلیل، تمایز میان مزیت نظری یک رویکرد در سطح مدل و مزیت عملی آن در سطح اجرای واقعی، نکته‌ای است که در هر بررسی مقایسه‌ای میان رویکرد فریم‌محور و رویدادمحور باید به‌طور جدی مورد توجه قرار گیرد.

۲-۱۰ پیشینه‌ی پژوهش

بخش‌های پیشین این فصل، مبانی مفهومی لازم برای درک موضوعات این پروژه را معرفی کردند. در این بخش، پیشینه‌ی پژوهشی مرتبط با این مبانی در سه محور مرور می‌شود: نخست، مدل‌هایی که به‌طور خاص برای تشخیص اشیا روی داده‌ی رویدادمحور طراحی شده‌اند؛ دوم، پژوهش‌هایی که از زوایای مختلف، پرسش کارایی انرژی شبکه‌ی عصبی اسپایکی در برابر شبکه‌ی عصبی مصنوعی متعارف را روی سخت‌افزار واقعی بررسی کرده‌اند؛ و سوم، سخت‌افزار نورومورفیک و تلاش‌های اخیر جامعه‌ی پژوهشی برای معیارسنجی و استانداردسازی ارزیابی آن.

۲-۱۰-۱ مدل‌های تشخیص اشیا روی داده‌ی رویدادمحور

Cordone و همکاران یک شبکه‌ی عصبی اسپایکی کامل مبتنی بر SSD برای تشخیص اشیا روی داده‌ی خام رویدادمحور طراحی کردند و نشان دادند چنین شبکه‌ای می‌تواند به دقتی قابل مقایسه با شبکه‌های مصنوعی متعارف با پیچیدگی مشابه دست یابد [۲۴، ۲۵]. این پروژه، اما، تنها یکی از چند رویکرد رقیب برای تشخیص اشیا روی داده‌ی رویدادمحور است. Perot و همکاران، همزمان با انتشار یکی از بزرگ‌ترین مجموعه‌داده‌های عمومی تشخیص اشیای رویدادمحور در مقیاس یک‌مگاپیکسل، معماری‌ای موسوم به RED را معرفی کردند که به‌جای واحدهای اسپایکی، از بلوک‌های بازگشتی[۳۲] ConvLSTM برای تجمیع اطلاعات زمانی رویدادها استفاده می‌کند و نشان دادند که این رویکرد بازگشتی، از مدل‌های رویدادمحور پیشین که رویدادها را به‌صورت گذر رو به جلوی صرف پردازش می‌کردند، به‌مراتب دقیق‌تر عمل می‌کند [۲۶]. در ادامه‌ی همین خط پژوهشی، Gehrig و Scaramuzza با معرفی Recurrent Vision Transformers (RVT) نشان دادند که ترکیب یک ستون[۳۳] پیچشی سبک با توجه خودکار[۳۴] محلی و سراسری و یک مکانیزم بازگشتی زمانی، می‌تواند به دقت پیشرفته روی مجموعه‌داده‌ی Gen1 برسد و در همان حال، تاخیر استنتاج را نسبت به کارهای پیشین تا شش برابر کاهش دهد؛ به‌طوری‌که این مدل روی یک NVIDIA Tesla T4 در کمتر از ۱۲ میلی‌ثانیه اجرا می‌شود [۲۷].

این سه خط پژوهش، سه راهبرد متفاوت برای رویارویی با ماهیت رویدادمحور داده را نمایندگی می‌کنند: پردازش اسپایکی (Cordone)، تجمیع بازگشتی متعارف (Perot)، و ترکیب توجه و بازگشت

[۳۲] Recurrent
[۳۳] Backbone
[۳۴] Self-Attention

(Gehrig و Scaramuzza). هدف اصلی این پژوهش‌ها عموما بهینه‌سازی دقت و سرعت تشخیص روی مجموعه‌داده‌های تخصصی رویدادمحور بوده است.

در امتداد همین خط پژوهشی، دو کار دیگر به‌طور مشخص میان پرسش دقت تشخیص و پرسش کارایی انرژی، که در بخش ۲-۱۰-۲ به آن پرداخته می‌شود، پل می‌زنند. Kugele و همکاران معماری‌ای هیبریدی پیشنهاد کردند که در آن یک ستون اسپایکی برای استخراج ویژگی از رویدادها با یک سر شبکه‌ی مصنوعی متعارف برای طبقه‌بندی و مکان‌یابی ترکیب می‌شود و کارایی انرژی به‌صراحت هدف طراحی معماری قرار گرفته است [۲۸]. Su و همکاران نیز با معرفی EMS-YOLO، نخستین شبکه‌ی عصبی اسپایکی عمیقی را ارائه کردند که به‌جای تبدیل از یک شبکه‌ی مصنوعی از پیش آموزش‌دیده، مستقیما و با گرادیان جانشین برای تشخیص اشیا آموزش داده شده است؛ آن‌ها نشان دادند این مدل روی مجموعه‌داده‌ی رویدادمحور Gen1 و مجموعه‌داده‌ی فریم‌محور COCO، در تنها چهار گام زمانی، به دقتی نزدیک به مدل مصنوعی متناظر دست می‌یابد و در همان حال انرژی کمتری مصرف می‌کند [۲۹]. با این حال، در هر دو کار، عدد انرژی گزارش‌شده از شمارش نظری عملیات به‌دست آمده است، نه از اندازه‌گیری مستقیم روی یک سکوی سخت‌افزاری واقعی؛ همین نکته، زمینه را برای مرور خط دوم پژوهش‌های پیشینه در بخش بعد فراهم می‌کند.

۲-۱۰-۲ مقایسه‌ی کارایی انرژی شبکه‌های عصبی اسپایکی و مصنوعی

مدل تحلیلی Lemaire و همکاران [۶]، در میان خطی طولانی‌تر از پژوهش‌های تحلیلی درباره‌ی کارایی انرژی شبکه‌ی عصبی اسپایکی جای می‌گیرد. Davidson و Furber با مقایسه‌ی پیاده‌سازی رقمی گام‌های محاسباتی یک شبکه‌ی مصنوعی متعارف با گام‌های معادل آن در یک شبکه‌ی اسپایکی نرخ‌کدشده[۳۵]، نشان دادند که با فرض یک فناوری زیرین یکسان، اغلب پیاده‌سازی‌های اسپایکی نرخ‌کدشده کاراتر از شبکه‌ی مصنوعی اصلی نیستند، مگر آنکه از کدگذاری اسپایکی[۳۶] خلاقانه‌تری بهره گرفته شود [۳]. Dampfhoffer و همکاران نیز با مقایسه‌ی پیاده‌سازی رقمی متعارف شبکه‌های مصنوعی با پیاده‌سازی رویدادمحور شبکه‌های اسپایکی مبتنی بر نورون Integrate-and-Fire، با استفاده از شتاب‌دهنده‌ی Eyeriss به‌عنوان مطالعه‌ی موردی، نشان دادند شبکه‌ی اسپایکی تنها در تنکی اسپایک بسیار بالا می‌تواند با پیاده‌سازی‌های کارای شبکه‌ی مصنوعی رقابت کند [۴]. در پژوهشی جدیدتر، Yan، Bai و Wong با

[۳۵] Rate-Coded
[۳۶] Spike Encoding

تعریف یک خط مبنای[۳۷] منصفانه که هر گام زمانی شبکه‌ی اسپایکی را به تعداد بیت معادل آن در یک شبکه‌ی مصنوعی کوانتیزه[۳۸] نگاشت می‌دهد، فرمول‌های دقیقی برای انرژی روی چند معماری سخت‌افزاری مختلف ارائه کردند و نشان دادند که شبکه‌ی اسپایکی تنها با رعایت آستانه‌های مشخصی از تنکی و طول پنجره‌ی زمانی می‌تواند از شبکه‌ی مصنوعی کوانتیزه‌ی معادل کاراتر باشد [۵].

نکته‌ی مشترک این سه پژوهش این است که کاهش تعداد عملیات موثر به‌تنهایی کافی نیست؛ در اغلب معماری‌های سخت‌افزاری متعارف، هزینه‌ی دسترسی و جابه‌جایی حافظه بر هزینه‌ی محاسبه‌ی صرف غالب است. با این حال، این پژوهش‌ها عمدتا در سطح یک لایه‌ی منفرد، یک شبکه‌ی طبقه‌بندی کوچک یا یک بار کاری مصنوعی انجام شده‌اند.

در کنار این مطالعات تحلیلی، شاخه‌ی دیگری از پژوهش‌ها کارایی انرژی را نه از راه شمارش عملیات، بلکه با اندازه‌گیری مستقیم توان مصرفی روی سخت‌افزار واقعی بررسی کرده‌اند. Patel و همکاران با تبدیل معماری U-Net به یک شبکه‌ی اسپایکی و اجرای آن روی تراشه‌ی Loihi، توان پویای مصرفی (پس از کسر توان بی‌کاری از توان کل) را در برابر همان معماری روی CPU و GPU اندازه‌گیری کردند [۳۰]. Henkes، Eshraghian و Wessels نیز با استفاده از ابزار نمایه‌برداری KerasSpiking، انرژی هر گذر رو به جلوی یک شبکه‌ی اسپایکی روی Loihi را در برابر شبکه‌ی مصنوعی متناظر روی یک V100 GPU مقایسه کردند و ضریب کاهش انرژی مشخصی گزارش دادند [۳۱]. در پژوهشی جدیدتر و با کاربردی متفاوت (سامانه‌های فضایی)، Lunghi و همکاران، در کنار ارائه‌ی یک معیار انرژی مستقل از سخت‌افزار، به‌صراحت هشدار دادند که توانایی واقعی شبکه‌ی اسپایکی برای رسیدن به کارایی نظری‌اش هنوز در عمل اثبات نشده است [۳۲]؛ همین نکته، در چارچوبی متفاوت و برای داده‌ی رویدادمحور، توسط Dalgaty و همکاران نیز به‌عنوان یک دوگانگی بنیادین میان دو رویکرد پیچشی و اسپایکی صورت‌بندی شده است [۳۳]. آنچه این مطالعات اندازه‌گیری‌شده را از سه پژوهش پیشین متمایز می‌کند، اجرای واقعی روی سخت‌افزار متعارف یا نورومورفیک به‌جای شمارش نظری عملیات است.

۲-۱۰-۳ سخت‌افزار نورومورفیک و معیارسنجی آن

همان‌طور که در بخش ۲-۹ اشاره شد، سخت‌افزارهایی مانند TrueNorth [۲۲] و Loihi [۲۳] برای بهره‌برداری مستقیم از تنکی فعالیت اسپایکی طراحی شده‌اند. در کنار این دو، SpiNNaker یکی دیگر از

[۳۷] Baseline
[۳۸] Quantized

سکوهای با مقیاس بزرگ نورومورفیک است؛ سامانه‌ای میلیون‌هسته‌ای مبتنی بر پردازنده‌های ARM با یک زیرساخت ارتباطی ناهمگام و اختصاصی برای انتقال بسته‌های اسپایک، که برای شبیه‌سازی شبکه‌های عصبی زیستی در مقیاس بزرگ و بی‌درنگ طراحی شده است [۳۴]. نسل جدیدتر این سکو، موسوم به SpiNNaker2، در پژوهشی اخیر توسط Arfa و همکاران برای اجرای یک عامل یادگیری تقویتی اسپایکی به‌کار گرفته شد و در برابر یک GPU متعارف تا ۳۲ برابر کاهش انرژی مصرفی نشان داد، در حالی که تاخیر آن با اجرای مبتنی بر GPU قابل مقایسه باقی ماند [۳۵]؛ این نتیجه نشان می‌دهد سکوهای نورومورفیک با مقیاس بزرگ همچنان در حال تکامل‌اند و مقایسه‌ی آن‌ها با سکوهای متعارف، موضوعی فعال در پژوهش‌های اخیر است. نمونه‌ای از استقرار کامل چنین رویکردی، یعنی جفت‌شدن مستقیم یک حسگر رویدادمحور با یک تراشه‌ی نورومورفیک، در کار Amir و همکاران دیده می‌شود؛ آن‌ها یک دوربین رویدادمحور را مستقیما به یک شبکه‌ی اسپایکی نگاشته‌شده روی تراشه‌ی TrueNorth متصل کردند و یک سامانه‌ی تشخیص حرکت دست را با توان مصرفی بسیار پایین و به‌صورت کاملا بی‌درنگ پیاده‌سازی کردند [۳۶]. این کار، یکی از معدود نمونه‌های منتشرشده‌ای است که جفت‌شدن مفهومی حسگر رویدادمحور و تراشه‌ی اسپایکی نورومورفیک را برای یک تراشه‌ی واقعی و یک کاربرد تشخیص حرکت دست نشان می‌دهد.

با رشد تنوع این سخت‌افزارها و روش‌های ارزیابی آن‌ها، نبود معیاری استاندارد برای مقایسه‌ی منصفانه‌ی نتایج میان پژوهش‌های مختلف، خود به یک مسئله تبدیل شده است. Yik و همکاران، با معرفی چارچوب NeuroBench، به این مسئله پاسخ می‌دهند: آن‌ها مجموعه‌ای از کاربردها و ابزارهای مشترک، هم مستقل از سخت‌افزار و هم وابسته به سخت‌افزار، پیشنهاد می‌کنند تا رویکردهای نورومورفیک به‌صورت یکسان و قابل‌تکرار سنجیده شوند [۳۷]. این تلاش، از منظر روش‌شناسی، این نکته را تایید می‌کند که یک عدد کارایی به‌تنهایی، بدون مشخص کردن اینکه در کدام سطح (نظری، اندازه‌گیری‌شده یا نگاشت‌شده) به‌دست آمده، به‌سادگی می‌تواند گمراه‌کننده باشد.

در مجموع، سه خط پژوهش مرورشده در این بخش، طیف گسترده‌ای از رویکردهای موجود در ادبیات پژوهشی را در سه حوزه‌ی مرتبط نشان می‌دهند: مدل‌های تشخیص اشیای رویدادمحور (شامل رویکردهای بازگشتی، ترانسفورمری، هیبریدی و مستقیما آموزش‌دیده‌ی اسپایکی)، مقایسه‌های تحلیلی و اندازه‌گیری‌شده‌ی کارایی انرژی شبکه‌ی اسپایکی در برابر شبکه‌ی مصنوعی روی سخت‌افزار متعارف و نورومورفیک و تلاش‌های اخیر برای استانداردسازی معیارسنجی سخت‌افزار نورومورفیک، از جمله نسل‌های جدیدتر آن.

۲-۱۱ جمع‌بندی

در این فصل، مسیر مفهومی از بینایی ماشین به‌عنوان یک حوزه‌ی کاربردی، تا شبکه‌های عصبی پیچشی و مدل‌های تشخیص اشیای فریم‌محور و در ادامه تا دوربین‌های رویدادمحور و شبکه‌های عصبی اسپایکی به‌عنوان جایگزین الهام‌گرفته از زیست‌شناسی، دنبال شد. نشان داده شد که حسگر رویدادمحور و شبکه‌ی عصبی اسپایکی، هر دو بر پایه‌ی اصل مشترک واکنش به تغییر ساخته شده‌اند و از این نظر برای کاربردهای بی‌درنگ و سامانه‌های نهفته، که در آن‌ها توان، حافظه و زمان منابعی محدود هستند، از نظر مفهومی جذاب‌اند. همچنین نشان داده شد که استنتاج زمانی در این مدل‌ها، برخلاف مدل‌های فریم‌محور متعارف، بخشی درونی و ذاتی از مکانیزم محاسباتی نورون است. در نهایت، تمایز میان مزیت نظری کاهش عملیات موثر در سطح مدل و مزیت عملی آن در سطح اجرای سخت‌افزاری واقعی مرور شد. پیش از پایان فصل، پیشینه‌ی پژوهشی مرتبط با موضوعات این فصل در سه محور مدل‌های تشخیص اشیای رویدادمحور، مقایسه‌های کارایی انرژی شبکه‌ی اسپایکی و مصنوعی و سخت‌افزار نورومورفیک مرور شد.

فصل سوم

چارچوب مفهومی و طراحی مطالعه مقایسه‌ای

۳-۱ مقدمه

همانطور که در فصل دوم گفته شد، کاهش نظری حجم محاسبات موثر که رویکرد رویدادمحور نوید می‌دهد، لزوما به کاهش واقعی تاخیر یا انرژی مصرفی روی یک سکوی محاسباتی واقعی ترجمه نمی‌شود و این تمایز میان مزیت نظری و مزیت عملی، نکته‌ای است که هر مطالعه‌ی مقایسه‌ای در این حوزه باید صریحا به آن بپردازد. هدف این فصل، پیش از ورود به جزئیات فنی مدل‌ها، مجموعه‌داده و سکوهای اجرایی که در فصل چهارم ارائه می‌شوند، روشن کردن این نکته است که این پروژه دقیقا به دنبال پاسخ به چه چیزی است، چرا مقایسه به همین شکل طراحی شده و اینکه این مطالعه صراحتا به دنبال چه چیزی نیست.

۳-۲ پرسش راهنمای این پروژه

همان‌طور که در فصل دوم بحث شد، حسگر رویدادمحور و شبکه‌ی عصبی اسپایکی از نظر مفهومی برای کاربردهای نهفته و بی‌درنگ جذاب‌اند، زیرا هر دو تنها در صورت وجود تغییر یا فعالیت، داده تولید می‌کنند یا محاسبه انجام می‌دهند. این جذابیت مفهومی صرفا یک دلیل نظری برای این انتظار است که چنین رویکردی ممکن است در عمل کاراتر باشد. پرسشی که این پروژه به دنبال آن است دقیقا همین گذر از نظر به عمل را هدف می‌گیرد: وقتی یک زنجیره‌ی بینایی ماشین مبتنی بر رویداد و محاسبات اسپایکی، به‌جای زنجیره‌ی متعارف مبتنی بر فریم و شبکه‌ی عصبی مصنوعی، برای انجام یک کاربرد یکسان تشخیص اشیا پیاده‌سازی و روی سکوهای محاسباتی واقعی اجرا شود، این مزیت نظری چه مقدار از خود را در قالب صرفه‌جویی واقعی در حافظه، زمان اجرا و انرژی مصرفی نشان می‌دهد؟ تشخیص اشیا برای این منظور انتخاب شده است، زیرا یکی از رایج‌ترین و پرکاربردترین کاربردهای بینایی ماشین در سامانه‌های نهفته و بی‌درنگی است که در فصل دوم بررسی شدند و بستری مناسب برای سنجش رفتار محاسباتی هر دو رویکرد فراهم می‌کند.

۳-۳ مرزبندی ارزیابی کیفیت تشخیص

نکته‌ای که باید از همان ابتدا روشن شود این است که این پروژه دقت تشخیص و معیارهایی که کیفیت طبقه‌بندی و مکان‌یابی اشیا را می‌سنجد، میان زنجیره‌ی رویدادمحور و زنجیره‌ی فریم‌محور مقایسه

نمی‌کند. این تصمیم از سر بی‌اهمیت دانستن دقت گرفته نشده، بلکه دو دلیل مشخص دارد.

نخست، این پرسش پیش‌تر و به‌طور مستقل در پیشینه‌ی پژوهش بررسی شده است. مطالعاتی که مستقیما به ساخت شبکه‌های عصبی اسپایکی برای تشخیص اشیا روی داده‌ی رویدادمحور پرداخته‌اند، از جمله همان خط پژوهشی که معماری‌های این پروژه از آن الهام گرفته‌اند، نشان داده‌اند که چنین مدل‌هایی، پس از آموزش کامل و ارزیابی مناسب، می‌توانند به دقت تشخیصی قابل مقایسه با مدل‌های مصنوعی متعارف با پیچیدگی مشابه دست یابند [۲۴، ۲۵]. به بیان دیگر، این سوال که «آیا یک شبکه‌ی عصبی اسپایکی رویدادمحور اصولا می‌تواند به‌خوبی یک شبکه‌ی مصنوعی متعارف تشخیص دهد؟» پیش از این با پاسخ مثبت روبه‌رو شده است. تکرار این مقایسه در مقیاس کوچک‌تر این پروژه، بدون منابع لازم برای انجام درست آن، ارزش افزوده‌ی چندانی به این پیشینه اضافه نمی‌کند.

دوم، ساختن یک ادعای معتبر درباره‌ی دقت تشخیص، به آموزش کامل هر یک از مدل‌های مقایسه‌شده تا همگرایی، تنظیم دقیق ابرپارامترها و ارزیابی روی داده‌ی نگه‌داشته‌شده‌ی مستقل نیاز دارد؛ فرایندی که به زمان محاسباتی، توان پردازشی و بودجه‌ی پژوهشی قابل توجهی وابسته است. این منابع در مقیاس یک پروژه‌ی کارشناسی در دسترس نبودند. در چنین شرایطی، تلاش برای گزارش یک عدد دقت که از آموزشی ناقص یا غیرمنصفانه به‌دست آمده، نه‌تنها کمکی به فهم موضوع نمی‌کند، بلکه می‌تواند گمراه‌کننده نیز باشد؛ زیرا یک عدد دقت پایین ممکن است صرفا نتیجه‌ی آموزش ناکافی باشد و ربطی به شایستگی ذاتی رویکرد رویدادمحور نداشته باشد.

به همین دو دلیل، این پروژه با آگاهی کامل، مرز مطالعه را در نقطه‌ای متفاوت رسم می‌کند: به‌جای تلاش برای پاسخ به پرسشی که پیش‌تر و بهتر پاسخ داده شده، تمرکز بر پرسشی گذاشته می‌شود که در پیشینه‌ی پژوهش کمتر به آن پرداخته شده است، یعنی رفتار محاسباتی و اجرایی این دو رویکرد وقتی روی سکوهای محاسباتی واقعی پیاده می‌شوند.

۳-۴ ابعاد مقایسه

با کنار گذاشتن دقت تشخیص، این مطالعه بر چهار بعد محاسباتی متمرکز می‌شود که مستقیما با قابلیت استقرار یک مدل بینایی ماشین روی یک سامانه‌ی نهفته مرتبط‌اند و در فصل دوم به‌عنوان محدودیت‌های اصلی چنین سامانه‌هایی معرفی شدند: اندازه‌ی مدل و هزینه‌ی حافظه‌ی لازم برای نگه‌داری پارامترها، حجم واقعی محاسباتی که هر زنجیره برای پردازش یک ورودی انجام می‌دهد، زمان لازم برای رسیدن از

ورودی به خروجی و انرژی مصرفی در طول این فرایند. این چهار بعد، صرف‌نظر از آنکه خروجی نهایی مدل چقدر دقیق است، تعیین می‌کنند که آیا یک مدل اصولا در بودجه‌ی توان، حافظه و زمان یک سامانه‌ی نهفته جا می‌شود یا خیر. از این رو، حتی بدون مقایسه‌ی دقت، این ابعاد به‌تنهایی برای پاسخ به پرسش این پروژه کافی و مرتبط هستند.

۳-۵ اصل برابری ساختاری در طراحی مقایسه

برای آنکه هر تفاوتی که در این ابعاد مشاهده می‌شود، واقعا قابل انتساب به نوع ورودی (رویداد در برابر فریم) و نوع محاسبه (اسپایکی در برابر متعارف) باشد و نه به یک تفاوت اتفاقی و بی‌ربط در طراحی مدل، این پروژه بر اصل برابری ساختاری[۱] استوار است. به‌جای انتخاب یک شبکه‌ی اسپایکی دلخواه از یک سو و یک شبکه‌ی مصنوعی دلخواه از سوی دیگر، برای هر معماری پایه چند نسخه‌ی متناظر ساخته می‌شود که تا حد امکان در همه‌ی جنبه‌های ساختاری مثل عمق، ترتیب لایه‌ها و ظرفیت، یکسان نگه داشته می‌شوند و تنها در نوع ورودی و نوع واحد محاسباتی با یکدیگر تفاوت دارند. اگر این کنترل انجام نشود، هر تفاوتی که در حافظه، زمان یا انرژی مشاهده شود ممکن است صرفا بازتاب یک انتخاب طراحی تصادفی باشد.

۳-۶ مجموعه‌داده‌ی مشترک و نمایش دوگانه‌ی ورودی

بعد دیگر برابری در طراحی این مطالعه، به داده‌ی ورودی مربوط می‌شود. برای آنکه تفاوت در نتایج به تفاوت در نحوه‌ی پردازش نسبت داده شود و نه به تفاوت در محتوای صحنه‌های مشاهده‌شده، هر دو زنجیره باید دقیقا همان صحنه‌های واقعی را پردازش کنند. اگر زنجیره‌ی رویدادمحور روی یک مجموعه‌داده و زنجیره‌ی فریم‌محور روی مجموعه‌داده‌ای دیگر ارزیابی شود، هرگونه تفاوت مشاهده‌شده می‌تواند ناشی از تفاوت میان دو مجموعه‌داده باشد. به همین دلیل، این پروژه از یک مجموعه‌داده‌ی واحد استفاده می‌کند که برای هر صحنه، هم ثبت رویدادمحور و هم ثبت فریم‌محور را به‌طور هم‌زمان در اختیار قرار می‌دهد؛ به این ترتیب، محتوای صحنه به‌عنوان یک متغیر مزاحم[۲] کنار گذاشته می‌شود و صحنه‌ی زیربنایی برای هر دو زنجیره کاملا یکسان است. حوزه‌ی رانندگی خودکار به‌طور مشخص

[۱] Structural Parity
[۲] Confounding Variable

برای این منظور مناسب است، زیرا هم یکی از زمینه‌هایی است که نیاز واقعی به تشخیص بی‌درنگ و سریع اشیایی مانند خودروها و عابران پیاده در صحنه دارد و هم یکی از معدود حوزه‌هایی است که در آن چنین مجموعه‌داده‌های هم‌زمان و دومنظوره، با ثبت همزمان رویداد و فریم از یک صحنه‌ی واحد، در دسترس‌اند.

۳-۷ ضرورت استفاده از چند سطح شواهد در ارزیابی

نکته‌ی دیگری که از بحث فصل دوم درباره‌ی شکاف میان مزیت نظری و مزیت عملی نتیجه می‌شود این است که سنجیدن این چهار بعد تنها از یک زاویه، برای پاسخ‌گویی به پرسش راهنمای این پروژه کافی نیست. صرفا شمردن حجم نظری محاسبات یک مدل روی کاغذ نشان نمی‌دهد که این مزیت روی یک سکوی اجرایی واقعی چه سرنوشتی پیدا می‌کند، زیرا همان‌طور که در فصل دوم بحث شد، این تبدیل به سخت‌افزار زیرین وابسته است. برای پرهیز از این محدودیت، این پروژه همان مجموعه مدل‌ها را از چند زاویه‌ی مکمل بررسی می‌کند: نخست، ویژگی‌های ساختاری و حجم محاسباتی ذاتی هر مدل، مستقل از هر سخت‌افزار خاص؛ دوم، رفتار واقعی و اندازه‌گیری‌شده‌ی مدل‌ها هنگام اجرا روی یک سکوی محاسباتی متعارف و در دسترس؛ و سوم، یک برآورد تحلیلی و تخمینی از رفتار مورد انتظار روی سخت‌افزار تخصصی‌تری که مشخصا برای بهره‌برداری از محاسبات رویدادمحور و اسپایکی طراحی شده است، حتی اگر دسترسی مستقیم و کامل به چنین سخت‌افزاری در مقیاس این پروژه ممکن نباشد. ترکیب این سه زاویه، به‌جای تکیه بر یک نوع سنجش، امکان می‌دهد که مشخص شود شکاف میان مزیت نظری و مزیت عملی در کجا واقعا ظاهر می‌شود، در کجا ناپدید می‌شود و به چه چیزی وابسته است.

۳-۸ جمع‌بندی

این فصل چارچوب مفهومی این پروژه را، پیش از ورود به جزئیات فنی، ترسیم کرد. پرسش راهنما این است که آیا کاهش نظری حجم محاسبات موثر در رویکرد رویدادمحور، در عمل به کاهش واقعی حافظه، زمان اجرا و انرژی مصرفی منجر می‌شود. برای پاسخ به این پرسش، دقت تشخیص عمدا از دامنه‌ی مقایسه کنار گذاشته شد؛ به این دلیل که پیشینه‌ی پژوهش پیش‌تر رقابت‌پذیری آن را نشان داده است و اینکه منابع لازم برای آموزش و ارزیابی معتبر آن دارای محدودیت می‌باشد. در عوض، مقایسه

حول چهار بعد محاسباتی متمرکز شد که مستقیما با قابلیت اجرا روی یک سامانه‌ی نهفته مرتبط‌اند، با رعایت اصل برابری ساختاری میان نسخه‌های متناظر رویدادمحور و فریم‌محور، روی صحنه‌های کاملا یکسان گرفته‌شده از یک مجموعه‌داده‌ی مشترک و از چند زاویه‌ی مکمل سنجیده شد. فصل چهارم، این چارچوب را با معماری‌ها، مجموعه‌داده، سکوهای اجرایی و نتایج مشخص پیاده‌سازی می‌کند.

فصل چهارم

روش‌شناسی، پیاده‌سازی و ارزیابی

۴-۱ مقدمه

در این فصل، روش اجرایی پروژه و نتایج حاصل از آن به‌صورت یک زنجیره منسجم ارائه می‌شود. هدف اصلی آزمایش‌ها، مقایسه‌ی دو رویکرد بینایی ماشین برای کاربردهای نهفته‌ی بی‌درنگ است: زنجیره‌ی مبتنی بر دوربین رویدادمحور و شبکه‌ی عصبی اسپایکی و زنجیره‌ی مبتنی بر تصویر RGB و شبکه‌ی عصبی مصنوعی متعارف. بنابراین، در این فصل بررسی می‌شود که با ثابت نگه داشتن معماری تا حد امکان، تفاوت در نوع ورودی، ماهیت محاسبات اسپایکی و بستر سخت‌افزاری چه اثری بر حجم محاسبات، حافظه، تاخیر و انرژی دارد.

برای جلوگیری از اختلاط کمیت‌هایی که مبنای یکسان ندارند، ارزیابی در چهار گروه شواهد تفکیک شد. نخست، پیچیدگی معماری و فعالیت واقعی شبکه مستقل از سخت‌افزار بررسی شد. دوم، رفتار هر ۱۸ مدل به‌صورت full-detector[۱] هنگام اجرای PyTorch/CUDA روی یک NVIDIA Tesla T4 اندازه‌گیری شد. سوم، انرژی هر ۱۸ مدل به‌صورت full-detector در چارچوب واحد مدل تحلیلی Lemaire و همکاران برآورد شد [۶]. چهارم، فقط زیرشبکه‌ی اسپایکی شش Event-SNN با SANA-FE و یک نگاشت تجمیع‌شده‌ی Loihi-1-like بررسی شد [۷]. این تفکیک ضروری است، زیرا اجرای یک شبکه‌ی عصبی اسپایکی در PyTorch روی GPU لزوما به این معنا نیست که سخت‌افزار بتواند از تنکی فعالیت شبکه و ماهیت رویدادمحور آن بهره‌برداری کند.

۴-۲ طراحی آزمایش و ماتریس مدل‌ها

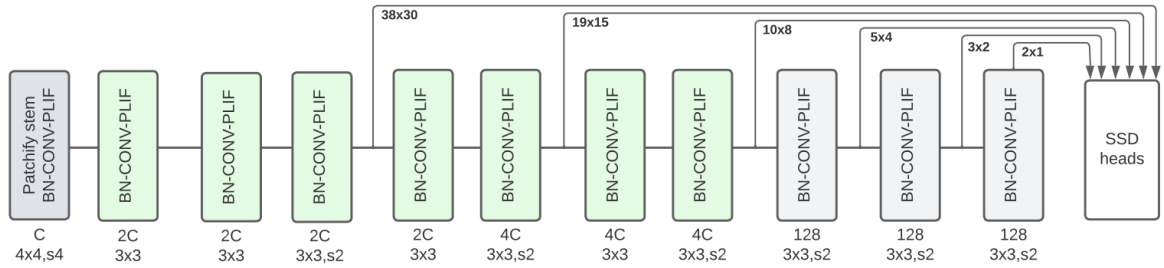
معماری‌های مورد مطالعه از خانواده‌های ST-VGG و ResCat در کار Cordone الهام گرفته شده‌اند [۲۴، ۲۵]. انتخاب این معماری‌ها به دلیل کاربرد آن‌ها در تشخیص اشیای رویدادمحور و تناسب آن‌ها با بررسی سناریوهای تعبیه‌شده و محدودیت‌های محاسباتی انجام شد. در کار اصلی Cordone و همکاران، این معماری‌ها برای توسعه‌ی شبکه‌های عصبی اسپایکی با قابلیت انجام تشخیص اشیا روی داده‌های رویدادمحور و تمرکز بر کاربردهای تعبیه‌شده و محدودیت‌های مربوط به سرعت و مصرف منابع مورد استفاده قرار گرفتند [۲۴، ۲۵]. از آنجا که این معماری‌ها بر پایه‌ی ساختارهای شناخته‌شده‌ی استخراج ویژگی و تشخیص SSD طراحی شده‌اند و مقیاس‌های مختلفی از پیچیدگی شبکه را فراهم می‌کنند،

[۱] در این پروژه، منظور از full-detector اجرای کامل زنجیره‌ی تشخیص شامل استخراج ویژگی، لایه‌های اضافی SSD و خروجی‌های طبقه‌بندی و رگرسیون است.

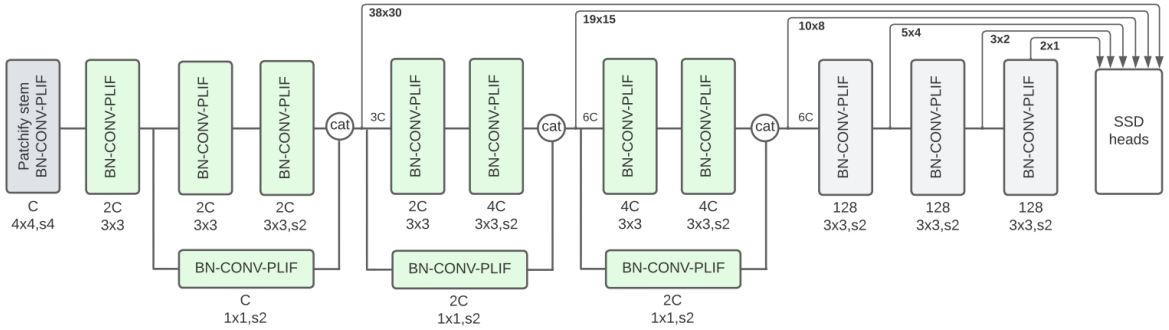
بستر مناسبی برای بررسی رابطه‌ی میان ظرفیت مدل، فعالیت اسپایکی و هزینه‌ی اجرایی در این پروژه فراهم می‌کنند.

در این پروژه، پیاده‌سازی این معماری‌ها به‌صورت مستقل بازسازی شد و برای هر خانواده سه مقیاس کانالی ۱۶، ۳۲ و ۶۴ در نظر گرفته شد. بدین ترتیب شش پیکربندی پایه تشکیل شد: ST-VGG-16، ST-VGG-32، ST-VGG-64، ResCat-16، ResCat-32 و ResCat-64.

ساختارهای اصلی ST-VGG+SSD و ResCat-SNN+SSD که مبنای بازسازی مدل‌های این پروژه قرار گرفته‌اند، به‌ترتیب در شکل ۴-۱ و شکل ۴-۲ نشان داده شده‌اند. این شکل‌ها مستقیماً از معماری‌های ارائه‌شده توسط Cordone در مرجع [۲۵] برگرفته شده‌اند.



شکل ۴-۱: معماری اصلی ST-VGG+SSD ارائه‌شده توسط Cordone [۲۵]. این معماری مبنای بازسازی خانواده‌ی ST-VGG در پروژه حاضر قرار گرفته است.



شکل ۴-۲: معماری اصلی ResCat-SNN+SSD ارائه‌شده توسط Cordone [۲۵]. این معماری مبنای بازسازی خانواده‌ی ResCat در پروژه حاضر قرار گرفته است.

در بازسازی انجام‌شده در این پروژه، منطق اصلی این دو خانواده حفظ شد، اما هدف تنها بازتولید نسخه‌ی اسپایکی نبود. برای ایجاد یک مقایسه‌ی کنترل‌شده میان پردازش مبتنی بر رویداد و پردازش مبتنی بر فریم، برای هر پیکربندی پایه سه نسخه‌ی متناظر ساخته شد:

۱. **Event-SNN**: ورودی دوکاناله‌ی رویدادی و بلوک‌های اصلی BN → Conv → PLIF؛

۲. **Strict RGB-ANN**: ورودی RGB سه‌کاناله و همان ترتیب ساختاری، با جایگزینی PLIF توسط ReLU؛

۳. **ANN-native RGB**: ورودی RGB و ترتیب متعارف‌تر $\text{Conv} \rightarrow \text{BN} \rightarrow \text{ReLU}$.

وجود نسخه‌ی Strict ANN باعث می‌شود مقایسه با Event-SNN از نظر ساختار تا حد امکان کنترل‌شده باشد. نسخه‌ی ANN-native نیز یک مرجع دوم فراهم می‌کند تا تفاوت احتمالی ناشی از ترتیب نرمال‌سازی و کانولوشن با اثر خود محاسبات اسپایکی اشتباه نشود. در مجموع، ماتریس آزمایش شامل ۱۸ مدل است که ترکیب آن‌ها در جدول ۴-۱ خلاصه شده است.

جدول ۴-۱: ماتریس مدل‌های مورد ارزیابی

خانواده	مقیاس	ورودی SNN	دو مرجع ANN
ST-VGG	۱۶	رویداد، ۲ کانال	Strict RGB / ANN-native RGB
ST-VGG	۳۲	رویداد، ۲ کانال	Strict RGB / ANN-native RGB
ST-VGG	۶۴	رویداد، ۲ کانال	Strict RGB / ANN-native RGB
ResCat	۱۶	رویداد، ۲ کانال	Strict RGB / ANN-native RGB
ResCat	۳۲	رویداد، ۲ کانال	Strict RGB / ANN-native RGB
ResCat	۶۴	رویداد، ۲ کانال	Strict RGB / ANN-native RGB

در خانواده‌ی ST-VGG، کاهش ابعاد مکانی با کانولوشن‌های stride دار و یک stem اولیه‌ی patchifying انجام می‌شود و از چند نقشه‌ی ویژگی در مقیاس‌های مختلف برای تشخیص استفاده می‌شود. در ResCat، مسیرهای ویژگی به‌جای جمع residual به‌صورت الحاق کانالی ترکیب می‌شوند؛ نسخه‌های ANN نیز همین توپولوژی را حفظ می‌کنند.

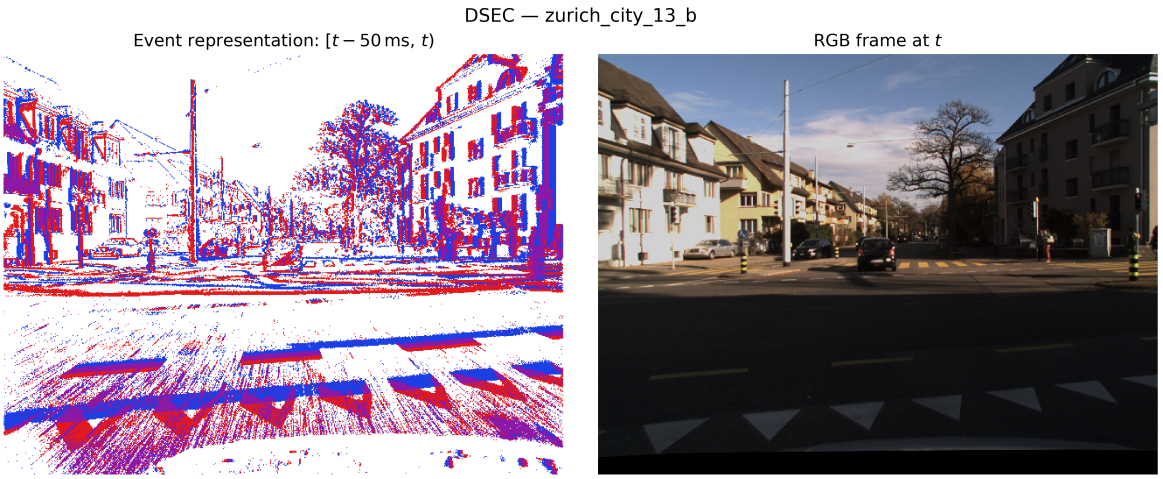
بخش خروجی تشخیص در هر سه زنجیره بر مفهوم Single Shot MultiBox Detector (SSD) استوار است [۱۵]. لایه‌های نهایی طبقه‌بندی و رگرسیون کادرهای محصورکننده به صورت متراکم باقی مانده‌اند، در حالی که در Event-SNN ستون اصلی و لایه‌های اضافی از پردازش اسپایکی و نورون‌های دارای حالت زمانی استفاده می‌شود و مدل‌های ANN از توابع فعال‌سازی متعارف بهره می‌برند.

۴-۳ مجموعه‌داده و ساخت ورودی‌ها

برای ایجاد ورودی‌های متناظر از مجموعه‌داده‌ی DSEC استفاده شد. این مجموعه‌داده شامل دوربین‌های رنگی فریم‌محور و دوربین‌های رویدادمحور همگام‌شده در سناریوهای رانندگی است [۳۸]. برچسب‌های تشخیص مورد استفاده در قطعه‌ی آزمایشی از توسعه‌ی DSEC برای تشخیص اشیا و داده‌های همراه پروژه‌ی DAGr فراهم شده‌اند [۱۷]. در این پروژه، از مخزن DAGr صرفا برای دریافت قطعه‌ی عمومی نمونه از مجموعه‌داده استفاده شد و هیچ‌یک از اجزای الگوریتمی DAGr در مدل‌های مورد ارزیابی به‌کار گرفته نشد.

قطعه‌ی مورد استفاده در آزمایش‌ها توالی zurich_city_13_b بود. پس از کنار گذاشتن نخستین تصویر، که برای آن بازه‌ی گذشته‌ی کامل ۵۰ میلی‌ثانیه‌ای در دسترس نبود، ۳۹ مشاهده‌ی زمانی متوالی برای ارزیابی نهایی باقی ماند. فاصله‌ی زمانی تصاویر تقریبا ۵۰ میلی‌ثانیه بود و برای هر زمان مرجع t، رویدادهای ثبت‌شده در بازه‌ی ۵۰ میلی‌ثانیه‌ای پیش از آن استخراج شدند.

نمونه‌ای واقعی از دو ورودی متناظر استخراج‌شده از همین توالی در شکل ۴-۳ نشان داده شده است. در سمت چپ، بازنمایی رویدادهای ثبت‌شده در پنجره‌ی زمانی $[t - ۵۰\ \mathrm{ms}, t)$ و در سمت راست، تصویر RGB متناظر با زمان مرجع t مشاهده می‌شود.



شکل ۴-۳: نمونه‌ای واقعی از ورودی‌های متناظر استخراج‌شده از توالی zurich_city_13_b مجموعه‌داده‌ی DSEC [۳۸]. سمت چپ، بازنمایی رویدادهای مثبت و منفی تجمیع‌شده در بازه‌ی $[t - ۵۰\ \mathrm{ms}, t)$ و سمت راست، تصویر RGB متناظر با زمان مرجع t نشان داده شده است.

هر رویداد به‌صورت

$$e_i = (x_i, y_i, t_i, p_i) \quad (۱-۴)$$

در نظر گرفته شد که در آن x_i و y_i مختصات مکانی رویداد، t_i زمان وقوع آن و p_i قطبیت تغییر روشنایی را نشان می‌دهد.

برای تشکیل ورودی مدل Event-SNN، رویدادهای هر پنجره‌ی زمانی به یک آرایه‌ی دودویی دوکاناله با ابعاد

$$۲ \times ۴۸۰ \times ۶۴۰ \quad (۲-۴)$$

تبدیل شدند. دو کانال ورودی به‌طور جداگانه اطلاعات مربوط به قطبیت‌های مثبت و منفی را نگه می‌دارند. اگر در یک پیکسل، طی پنجره‌ی زمانی مورد نظر حداقل یک رویداد از یک قطبیت مشخص رخ داده باشد، مقدار متناظر آن پیکسل در کانال مربوطه برابر یک و در غیر این صورت برابر صفر قرار می‌گیرد.

بنابراین، ورودی مدل Event-SNN در این پروژه حاصل پردازش کاملا ناهمگام و تک‌رویدادی نیست؛ بلکه رویدادها ابتدا در پنجره‌های زمانی ۵۰ میلی‌ثانیه‌ای تجمیع می‌شوند و هر پنجره یک گام زمانی شبکه را تشکیل می‌دهد. با این حال، حالت غشایی[۲] نورون‌های PLIF میان پنجره‌های متوالی حفظ می‌شود و در نتیجه شبکه اطلاعات زمانی پنجره‌های پیشین را در حالت داخلی خود نگه می‌دارد.

در ۳۹ پنجره‌ی مورد استفاده در ارزیابی، میانگین اشغال ورودی رویدادی برابر ۰٫۳۰۰۱ و انحراف معیار آن برابر ۰٫۰۴۷۷ بود. کمینه و بیشینه‌ی اشغال نیز به‌ترتیب ۰٫۲۲۱۱ و ۰٫۳۷۶۴ ثبت شد.

برای مدل‌های ANN، تصویر RGB متناظر با همان زمان مرجع t با ابعاد

$$۳ \times ۴۸۰ \times ۶۴۰ \quad (۳-۴)$$

به شبکه داده شد. به این ترتیب، برای هر مشاهده‌ی زمانی، ورودی رویدادی Event-SNN و ورودی فریم‌محور مدل‌های ANN از یک بازه‌ی زمانی متناظر در همان توالی استخراج شدند.

[۲] Membrane State

۴-۴ مدل نورون اسپایکی PLIF و پردازش زمانی

در مدل‌های Event-SNN استفاده‌شده در این پروژه، نورون‌های Parametric Leaky Integrate-and-Fire (PLIF) به‌عنوان واحد پردازشی اصلی به‌کار گرفته شدند. مدل PLIF نسخه‌ی قابل یادگیری نورون Leaky Integrate-and-Fire (LIF) است که در آن ثابت زمانی غشایی[۳] به‌عنوان یک پارامتر قابل آموزش در نظر گرفته می‌شود [۳۹]. این ویژگی اجازه می‌دهد که دینامیک زمانی نورون‌ها متناسب با ویژگی‌های داده‌ی ورودی تنظیم شود.

در نورون‌های LIF و PLIF، ورودی سیناپسی ابتدا باعث تغییر پتانسیل غشایی نورون می‌شود. پویایی غشا در حالت گسسته‌ی زمانی به‌صورت زیر بیان می‌شود:

$$H_t = V_{t-1} + \frac{1}{\tau}\left(X_t - (V_{t-1} - V_{reset})\right) \tag{۴-۴}$$

که در آن H_t پتانسیل غشایی قبل از بررسی آستانه، X_t جریان ورودی سیناپسی، V_{t-1} حالت غشایی قبلی، V_{reset} مقدار بازنشانی و τ ثابت زمانی غشایی است. در مدل PLIF پارامتر τ قابل یادگیری بوده و در طول آموزش بهینه می‌شود.

پس از محاسبه‌ی پتانسیل غشایی، تولید اسپایک با مقایسه‌ی مقدار فعلی با آستانه‌ی شلیک است که در آن ثابت زمانی غشایی انجام می‌شود:

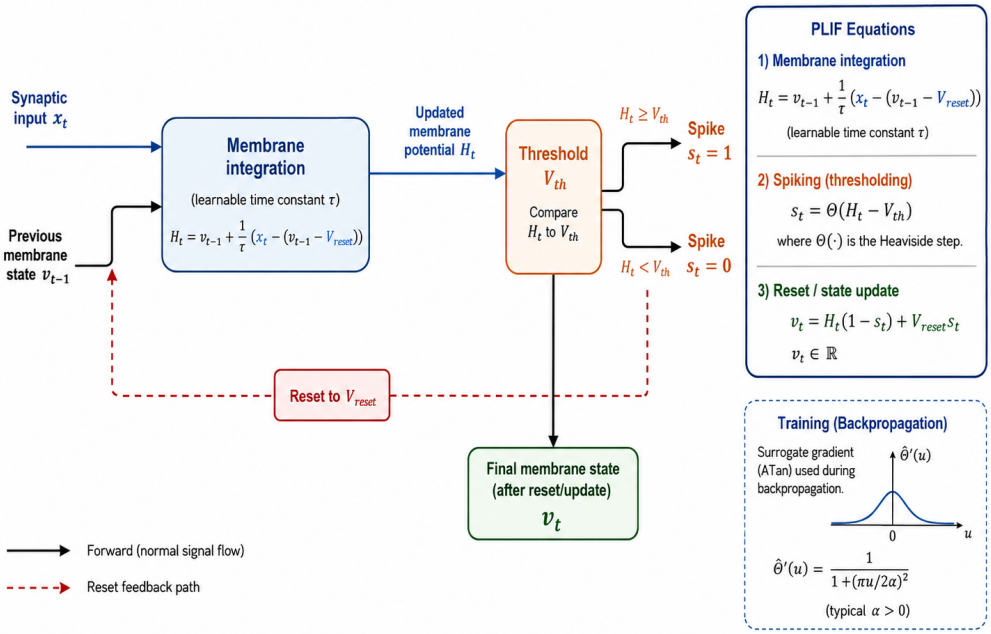
$$S_t = \Theta(H_t - V_{th}) \tag{۵-۴}$$

که در آن S_t خروجی اسپایک نورون و V_{th} آستانه‌ی شلیک است. در صورت عبور پتانسیل غشا از آستانه، یک اسپایک تولید شده و حالت نورون بازنشانی می‌شود:

$$V_t = H_t(1 - S_t) + V_{reset}S_t \tag{۶-۴}$$

نمای کلی فرآیند تجمع جریان، مقایسه با آستانه و بازنشانی حالت نورون در شکل ۴-۴ نمایش داده شده است.

[۳] Membrane Time Constant



شکل ۴-۴: نمایش مفهومی نورون Parametric Leaky Integrate-and-Fire (PLIF). ورودی سیناپسی باعث تغییر پتانسیل غشایی می‌شود و ثابت زمانی τ به‌عنوان پارامتر قابل یادگیری، رفتار زمانی نورون را کنترل می‌کند. پس از عبور پتانسیل غشا از آستانه‌ی شلیک، اسپایک تولید شده و حالت نورون بازنشانی می‌شود.

از آنجا که تابع تولید اسپایک در نقطه‌ی آستانه مشتق‌پذیر نیست، برای آموزش شبکه از روش گرادیان جانشین استفاده شد. این روش با جایگزین کردن مشتق واقعی تابع شلیک با یک تابع تقریب‌پذیر، امکان استفاده از الگوریتم پس‌انتشار را در شبکه‌های عصبی اسپایکی فراهم می‌کند [۱۹]. در این پروژه از تقریب ATan با پارامتر شیب ۲ استفاده شد.

در پیاده‌سازی انجام‌شده، نورون PLIF به‌صورت مستقل در محیط PyTorch پیاده‌سازی شد. مقدار اولیه‌ی ثابت زمانی غشایی $\tau = ۲$ و آستانه‌ی شلیک برابر با $V_{th} = ۱$ در نظر گرفته شد.

در مرحله‌ی استنتاج، شبکه به‌صورت جریانی اجرا شد. در ابتدای هر توالی، حالت غشایی تمامی نورون‌ها یک‌بار صفر شده و سپس در طول ۳۹ پنجره‌ی زمانی متوالی حفظ شد. این انتخاب به این دلیل انجام شد که هدف اندازه‌گیری رفتار یک پردازش جریانی پیوسته بود و اجرای مستقل هر پنجره به‌صورت یک نمونه‌ی جداگانه مد نظر نبود. بنابراین، حالت جریانی معیار اصلی ارزیابی مدل در این پروژه است.

۴-۵ آموزش محدود Event-SNNها و دامنه‌ی ادعا

شش Event-SNN پیش از benchmark به‌صورت محدود روی همان قطعه‌ی DSEC آموزش داده شدند تا فعالیت اسپایکی نهایی حاصل از وزن‌ها کاملا تصادفی نباشد. این مرحله با یک بودجه‌ی ثابت پنج epoch انجام شد و هدف آن بهینه‌سازی نهایی mAP یا ایجاد benchmark دقت نبود.

تنظیمات اصلی آموزش در جدول ۴-۲ آمده است. برای طبقه‌بندی از Focal Loss استفاده شد [۴۰] و برای رگرسیون مختصات کادر محصورکننده از Smooth L1 بهره گرفته شد. پردازش نمونه‌ها به ترتیب زمانی انجام شد و حالت غشایی بین پنجره‌های متوالی حفظ شد. در پنجره‌های فاقد برچسب، عبور رو به جلو[۴] همچنان برای به‌روزرسانی حالت زمانی اجرا می‌شد، ولی تابع loss اعمال نمی‌شد. برای جلوگیری از رشد نامحدود گراف محاسباتی، اتصال گرادیانی پس از حداکثر پنج گام بدون برچسب قطع می‌شد.

جدول ۴-۲: تنظیمات آموزش محدود Event-SNNها

پارامتر	مقدار
تعداد epoch	۵
optimizer	Adam
initial learning rate	۵×۱۰^{-۳}
weight decay	$۱۰^{-۲}$
learning rate schedule	cosine annealing
gradient clipping	۱/۰
maximum unlabeled steps before detach	۵
number of activity-probe windows	۲۴
PLIF $\tau_۰$	۲
آستانه‌ی شلیک	۱
گرادیان جانشین	ATan، شیب ۲

دو نسخه‌ی ANN در این مطالعه آموزش داده نشدند و به‌عنوان خط مبناهای ساختاری و اجرایی با وزن‌های اولیه استفاده شدند. از این رو، نتایج فصل حاضر برای مقایسه‌ی معماری، پیچیدگی، زمان اجرا و انرژی معتبرند، اما به‌عنوان مقایسه‌ی دقت تشخیص SNN و ANN تفسیر نمی‌شوند. ادعای mAP مقایسه‌ای به آموزش و ارزیابی مستقل روی داده‌ی held-out نیاز دارد و خارج از دامنه‌ی این آزمایش است.

[۴] Warm-up

۴-۶ محیط نرم‌افزاری و معیارهای ارزیابی

پیاده‌سازی اصلی در Python و PyTorch انجام شد [۴۱]. اجرای benchmark روی NVIDIA Tesla T4 در محیط Google Colab صورت گرفت. تمام مدل‌ها با ورودی ۶۴۰ × ۴۸۰ و همان توالی ۳۹ پنجره ارزیابی شدند.

۴-۶-۱ پارامتر، حافظه و شمارش محاسبات

برای هر مدل تعداد پارامترها، حافظه‌ی گرفته‌شده وزن‌ها در FP32 و FP16، اندازه‌ی state dictionary و تعداد MACهای پیچشی محاسبه شد. شمارش MAC برای یک کانولوشن به‌صورت

$$\mathrm{MAC} = B\, C_{out}\, H_{out}\, W_{out} \left(\frac{C_{in}}{g}\right) K_h K_w \tag{۴-۷}$$

انجام شد. در Event-SNN ابتدا هزینه‌ی dense-equivalent محاسبه شد؛ سپس با استفاده از فعالیت واقعی مشاهده‌شده، MACهای موثر رویدادمحور برآورد شدند. علاوه بر وزن‌ها، حافظه‌ی لازم برای نگهداری پتانسیل‌های غشایی تمام لایه‌های PLIF نیز محاسبه شد.

۴-۶-۲ فعالیت اسپایکی

نرخ اسپایک[۵] کلی شبکه از نسبت تعداد اسپایک‌ها به تعداد توابع فعال‌سازی قابل مشاهده به‌دست آمد و تنکی فعالیت برابر $r - ۱$ تعریف شد. علاوه بر میانگین کلی، نرخ اسپایک هر لایه PLIF جداگانه ثبت شد. در این پروژه، آستانه‌های تشخیصی به‌صورت $r \leq ۱۰^{-۵}$ برای dead، $۱۰^{-۵} < r < ۱۰^{-۳}$ برای near-dead و $r \geq ۰٫۸$ برای saturated در نظر گرفته شدند. این آستانه‌ها صرفا برای گزارش وضعیت فعالیت لایه‌ها استفاده شدند و در فرآیند آموزش دخالتی نداشتند.

۴-۶-۳ تاخیر، throughput و حافظه‌ی GPU

برای بررسی رفتار واقعی پیاده‌سازی روی سخت‌افزار متعارف، هر ۱۸ مدل روی NVIDIA Tesla T4 و با اجرای PyTorch/CUDA ارزیابی شدند. اندازه‌گیری تاخیر بر روی اجرای کامل سامانه‌ی تشخیص

[۵] Spike Rate

(full-detector) انجام شد؛ بنابراین زمان گزارش‌شده شامل عبور رو به جلوی ستون استخراج ویژگی، لایه‌های اضافی SSD و بخش خروجی تشخیص است. در مقابل، زمان مربوط به دریافت داده از حسگر، خواندن فایل از دیسک، ساخت پنجره‌ی رویدادی و سایر مراحل خارج از اجرای شبکه در این مقدار وارد نشده‌اند. از این رو، نتایج این بخش بیانگر زمان اجرای شبکه هستند و نباید به‌عنوان تاخیر کامل sensing-to-decision سامانه تفسیر شوند.

پیش از ثبت زمان، چند اجرای گرم‌سازی[۶] انجام شد تا هزینه‌های اولیه‌ی CUDA مانند ایجاد کرنل‌ها، تخصیص‌های اولیه‌ی حافظه و گرم‌شدن مسیر اجرایی تا حد ممکن از اندازه‌گیری اصلی حذف شوند. همچنین برای جلوگیری از خطای ناشی از اجرای غیرهمگام CUDA، زمان‌گیری پس از همگام‌سازی GPU انجام شد.

برای Event-SNN حالت اصلی اندازه‌گیری، استنتاج جریانی[۷] بود. در ابتدای توالی، حالت‌های غشایی نورون‌های PLIF یک‌بار صفر شدند و سپس در طول پنجره‌های زمانی متوالی حفظ شدند. بنابراین هر ورودی جدید با استفاده از حالت زمانی باقی‌مانده از پنجره‌ی قبلی پردازش شد. در دو مدل ANN حالت غشایی متناظری وجود ندارد و هر تصویر به‌صورت یک عبور رو به جلوی متراکم پردازش می‌شود.

برای هر مدل، میانگین زمان اجرا، صدک ۹۵ [۸] و نرخ پردازش (FPS[۹]) ثبت شد. مقدار صدک ۹۵ برای نشان دادن رفتار انتهای توزیع تاخیر استفاده شد و نسبت به میانگین، حساسیت بیشتری به اجراهای کندتر دارد. نرخ پردازش نیز از زمان متوسط هر پنجره به‌دست آمد و بیانگر تعداد تقریبی پنجره‌هایی است که شبکه می‌تواند در یک ثانیه پردازش کند.

علاوه بر زمان اجرا، بیشینه‌ی حافظه‌ی اختصاص‌یافته روی GPU در حین استنتاج به‌صورت peak VRAM ثبت شد. این مقدار تنها شامل حافظه‌ی پارامترهای مدل نیست و حافظه‌ی تنسورهای میانی، بافرهای اجرایی و در Event-SNN حافظه‌ی موردنیاز برای نگهداری حالت‌های غشایی را نیز در بر می‌گیرد.

در نمودارها، مقدار ۵۰ میلی‌ثانیه تنها به‌عنوان مرجع طول هر پنجره‌ی ورودی نشان داده می‌شود. قرار گرفتن زمان اجرای شبکه زیر این مقدار نشان می‌دهد که عبور رو به جلوی مدل از فاصله‌ی زمانی ورود پنجره‌ها کوتاه‌تر است، اما به‌تنهایی اثبات‌کننده‌ی عملکرد بی‌درنگ کل سامانه‌ی end-to-end نیست.

[۶] Warm-up
[۷] Streaming Inference
[۸] 95th Percentile (p95)
[۹] Frames Per Second

۴-۶-۴ اندازه‌گیری انرژی GPU

برای اندازه‌گیری مستقیم رفتار انرژی اجرای مدل‌ها روی Tesla T4، توان برد GPU با استفاده از NVIDIA Management Library (NVML) ثبت شد [۴۲]. این اندازه‌گیری، توان و انرژی اجرای واقعی PyTorch/CUDA روی برد NVIDIA را نشان می‌دهد و نباید به‌عنوان انرژی یک معماری نورومورفیک یا انرژی صرفا مربوط به عملیات سیناپسی تفسیر شود.

برای کاهش اثر نوسانات اولیه، پیش از اندازه‌گیری انرژی هر مدل سه پیمایش گرم‌سازی[۱۰] انجام شد. پس از آن، پنج تکرار مستقل برای هر مدل اجرا شد. هر تکرار شامل سه مرحله بود: یک بازه‌ی ۲ ثانیه‌ای idle پیش از اجرای مدل، حداقل ۱۰ ثانیه اجرای فعال، و یک بازه‌ی ۲ ثانیه‌ای idle پس از آن. توان برد در طول این فرآیند با فاصله‌ی زمانی ۲۰ میلی‌ثانیه نمونه‌برداری شد.

برای هر تکرار، توان پایه از اندازه‌گیری‌های idle قبل و بعد از اجرای فعال تعیین شد. سپس انرژی دینامیکی با انتگرال‌گیری از توان در بازه‌ی فعال و حذف سهم متناظر با خط پایه محاسبه شد. به‌صورت مفهومی، انرژی دینامیکی از رابطه‌ی

$$E_{\mathrm{dyn}} = \int_{t_0}^{t_1} \left[P_{\mathrm{active}}(t) - P_{\mathrm{idle}}\right] dt \tag{۸-۴}$$

به‌دست می‌آید. انرژی حاصل بر تعداد پنجره‌های پردازش‌شده در همان بازه تقسیم شد تا مقدار mJ/window به‌دست آید.

اگر توان اجرای فعال به‌طور قابل تفکیک از خط پایه‌ی idle بالاتر نبود، آن تکرار نامعتبر علامت‌گذاری می‌شد و به‌جای جایگزینی با مقدار صفر، از محاسبه‌ی میانگین کنار گذاشته می‌شد. برای هر مدل، میانگین انرژی دینامیکی تکرارهای معتبر به‌عنوان مقدار اصلی گزارش شد. همچنین انحراف معیار و بازه‌ی اطمینان ۹۵ درصد مبتنی بر توزیع Student-t برای نشان دادن تغییرپذیری بین تکرارها محاسبه شدند.

علاوه بر انرژی، توان دینامیکی متوسط نیز از اختلاف توان متوسط اجرای فعال و خط پایه به‌دست آمد:

$$P_{\mathrm{dyn}} = P_{\mathrm{active}} - P_{\mathrm{idle}}. \tag{۹-۴}$$

[۱۰] Warm-up Traversal

گزارش هم‌زمان توان و انرژی اهمیت دارد، زیرا انرژی تنها تابع توان لحظه‌ای نیست و به مدت اجرای مدل نیز وابسته است. در نتیجه، مدلی می‌تواند توان دینامیکی کمتری داشته باشد اما به دلیل زمان اجرای طولانی‌تر، انرژی بیشتری به ازای هر پنجره مصرف کند. برای بررسی هم‌زمان این دو جنبه، حاصل‌ضرب انرژی و تاخیر (Energy-Delay Product; EDP) نیز محاسبه شد:

$$\mathrm{EDP} = E_{\mathrm{dyn}} \times T_{\mathrm{latency}}. \tag{۱۰-۴}$$

در محاسبه‌ی EDP، T_{latency} زمان اجرای متناظر با همان تکرار اندازه‌گیری انرژی است؛ از این رو، انرژی و زمان مورد استفاده در این معیار از یک اجرای مشترک به‌دست می‌آیند.

۴-۶-۵ مدل تحلیلی انرژی

برای ایجاد یک چارچوب تحلیلی مستقل از یک پردازنده یا شتاب‌دهنده‌ی مشخص، از مدل انرژی ارائه‌شده توسط Lemaire و همکاران استفاده شد [۶]. این مدل به یک تراشه‌ی تجاری مشخص، مانند GPU یا پردازنده‌ی نورومورفیک خاص، وابسته نیست؛ با این حال، برای تبدیل تعداد عملیات و دسترسی‌های حافظه به انرژی، از پارامترهای مرجع فناوری 45 nm CMOS استفاده می‌کند.

در این چارچوب، انرژی کل اجرای شبکه از سه مولفه‌ی اصلی تشکیل می‌شود:

$$E_{\mathrm{total}} = E_{\mathrm{memory}} + E_{\mathrm{synaptic}} + E_{\mathrm{address}}, \tag{۱۱-۴}$$

که در آن E_{memory} انرژی مربوط به خواندن و نوشتن داده‌ها در حافظه، E_{synaptic} انرژی عملیات سیناپسی، و E_{address} هزینه‌ی عملیات مربوط به آدرس‌دهی و مدیریت فعالیت تنک است.

برای محاسبه‌ی هزینه‌ی عملیات حسابی، مدل مرجع Lemaire از مقادیر گزارش‌شده برای فناوری 45 nm CMOS و محاسبات صحیح ۳۲بیتی استفاده می‌کند. هزینه‌ی یک عملیات جمع و یک عملیات ضرب به‌ترتیب برابر با

$$E_{\mathrm{ADD}} = ۰٫۱\ \mathrm{pJ}, \qquad E_{\mathrm{MUL}} = ۳٫۱\ \mathrm{pJ} \tag{۱۲-۴}$$

در نظر گرفته می‌شوند [۶]. بنابراین، برای مجموعه‌ای از عملیات MAC و ACC، هزینه‌ی حسابی

به‌صورت کلی از رابطه‌ی

$$E_{\text{ops}} = N_{\text{MAC}} \left(E_{\text{MUL}} + E_{\text{ADD}} \right) + N_{\text{ACC}} E_{\text{ADD}} \qquad (۱۳-۴)$$

محاسبه می‌شود. در شبکه‌های متراکم ANN، بخش عمده‌ی عملیات پیچشی به‌صورت MAC انجام می‌شود، در حالی که در بخش‌های اسپایکی SNN تعداد عملیات وابسته به فعالیت واقعی نورون‌ها و تعداد اسپایک‌های منتقل‌شده میان لایه‌ها است. در نتیجه، نرخ اسپایک پایین می‌تواند تعداد عملیات سیناپسی فعال را به‌طور قابل توجهی کاهش دهد.

مدل Lemaire علاوه بر عملیات حسابی، هزینه‌ی دسترسی حافظه را نیز به‌صورت صریح در نظر می‌گیرد. برای حافظه‌ی SRAM، هزینه‌ی هر دسترسی تابعی از اندازه‌ی حافظه در نظر گرفته می‌شود. مقادیر مرجع مورد استفاده عبارت‌اند از ۱۰ pJ برای ۸ kB، ۲۰ pJ برای ۳۲ kB و ۱۰۰ pJ برای ۱ MB. برای اندازه‌های بین این نقاط، هزینه‌ی دسترسی با درون‌یابی خطی محاسبه می‌شود. همچنین، مطابق فرض مرجع، هزینه‌ی خواندن و نوشتن SRAM برابر در نظر گرفته شده است [۶].

یکی از تفاوت‌های مهم در مدل‌سازی حافظه‌ی دو نوع شبکه، نحوه‌ی نگهداری داده‌های بین لایه‌ها است. در مدل مرجع برای ANN، نقشه‌های ویژگی کامل در SRAM ذخیره می‌شوند. در مقابل، برای SNN انتقال اسپایک میان لایه‌ها با یک بافر FIFO با ظرفیت ۱۰۰۰ کلمه‌ی ۳۲بیتی مدل می‌شود [۶]. با این حال، شبکه‌ی اسپایکی برای نگهداری و به‌روزرسانی پتانسیل غشایی نورون‌ها نیز به خواندن و نوشتن حافظه نیاز دارد. بنابراین کاهش انرژی SNN تنها ناشی از کاهش عملیات حسابی نیست و تعادل میان کاهش دسترسی به وزن‌ها و ورودی‌ها و هزینه‌ی اضافی نگهداری حالت نورون‌ها نیز در نتیجه نقش دارد.

در پیاده‌سازی این پروژه، همین پارامترهای مرجع فناوری 45 nm شامل داده‌های ۳۲بیتی، هزینه‌های ۰٫۱ pJ برای جمع و ۳٫۱ pJ برای ضرب، مدل وابسته به اندازه‌ی SRAM و بافر FIFO با ظرفیت ۱۰۰۰ کلمه برای بخش اسپایکی استفاده شدند. همچنین در محاسبه‌ی انرژی استنتاج، Batch Normalization در لایه‌ی کانولوشن مجاور ادغام‌شده فرض شد و برای نورون‌های PLIF هزینه‌ی خواندن و نوشتن حالت غشایی نیز در شمارش دسترسی حافظه لحاظ شد. هر پنجره‌ی ۵۰ میلی‌ثانیه‌ای داده‌ی رویدادی در این محاسبات متناظر با یک گام زمانی SNN در نظر گرفته شد.

لازم به تاکید است که فناوری 45 nm CMOS در اینجا یک مرجع انرژی برای مدل تحلیلی است

و به معنی اجرای شبکه روی یک تراشه‌ی فیزیکی 45 nm نیست. همچنین مقادیر ۳۲بیتی این مدل نباید با نوع داده و دستورهای واقعی اجراشده توسط PyTorch/CUDA روی Tesla T4 یکسان در نظر گرفته شوند. خروجی این مدل تنها برآورد انرژی حاصل از شمارش عملیات و دسترسی‌های حافظه تحت مجموعه‌ای ثابت از مفروضات فناوری است.

در این پروژه، شمارش عملیات و دسترسی‌های حافظه برای هر ۱۸ مدل از ساختار واقعی شبکه استخراج شد و برای Event-SNN مقادیر فعالیت اسپایکی اندازه‌گیری‌شده در ۳۹ پنجره‌ی متوالی نیز وارد محاسبات شدند. از این رو، مقایسه‌های ANN/SNN در بخش نتایج تحت یک مجموعه‌ی یکسان از ضرایب انرژی انجام می‌شوند. این تفکیک اهمیت دارد، زیرا مدل Lemaire [۶] یک برآورد تحلیلی مبتنی بر فناوری مرجع است و اندازه‌گیری NVML رفتار فیزیکی اجرای PyTorch/CUDA روی Tesla T4 را ثبت می‌کند.

۴-۶-۶ نگاشت نورومورفیک با SANA-FE

برای بررسی این موضوع که رفتار تنک مدل‌های Event-SNN در یک بستر نورومورفیک چگونه می‌تواند به هزینه‌ی اجرایی متفاوتی منجر شود، از شبیه‌ساز SANA-FE 2.2.9 استفاده شد [۷]. برخلاف اندازه‌گیری‌های GPU که اجرای واقعی PyTorch/CUDA روی NVIDIA Tesla T4 را اندازه‌گیری می‌کنند، SANA-FE یک چارچوب شبیه‌سازی معماری است که امکان مدل‌سازی اجرای شبکه‌های عصبی اسپایکی روی سخت‌افزارهای نورومورفیک چندهسته‌ای را فراهم می‌کند. بنابراین نتایج این بخش باید به‌عنوان یک برآورد معماری‌محور تفسیر شوند.

در این پروژه، هدف نگاشت یک معماری Loihi-1-like در نظر گرفته شد. پردازنده‌ی Loihi نمونه‌ای از معماری‌های نورومورفیک چندهسته‌ای است که در آن نورون‌ها، سیناپس‌ها و ارتباطات اسپایکی به‌صورت توزیع‌شده روی هسته‌های محاسباتی اجرا می‌شوند [۲۳]. مزیت اصلی چنین معماری‌هایی در مقایسه با پردازنده‌های متراکم متعارف این است که فعالیت محاسباتی می‌تواند مستقیماً به وقوع اسپایک وابسته باشد؛ در نتیجه، نبود اسپایک در یک مسیر می‌تواند به کاهش عملیات و ترافیک ارتباطی منجر شود.

مدل زمان‌بندی SANA-FE در حالت detailed تنظیم شد و برای هر شش نگاشت، افق شبیه‌سازی ثابت برابر با ۵۰ timestep در نظر گرفته شد. استفاده از افق یکسان باعث می‌شود مقایسه‌ی داخلی شش زیرشبکه تحت شرایط شبیه‌سازی یکسان انجام شود. مقادیر نهایی در بخش نتایج به‌صورت نرمال‌شده بر پنجره‌ی ورودی گزارش می‌شوند.

ورودی شبیه‌ساز مستقیماً از نتایج benchmark مدل‌های آموزش‌دیده استخراج شد. برای هر لایه‌ی اسپایکی، اطلاعات زیر وارد نگاشت شد:

- تعداد نورون‌های واقعی لایه،
- ساختار اتصال میان لایه‌ها،
- نرخ اسپایک اندازه‌گیری‌شده روی توالی DSEC،
- تعداد رخدادهای سیناپسی ناشی از این فعالیت،
- و ترتیب وابستگی لایه‌ها در گراف محاسباتی.

به‌دلیل اندازه‌ی شبکه‌ها، انجام نگاشت کامل neuron-by-neuron و synapse-by-synapse به‌صورت مستقیم بسیار پرهزینه بود. از این رو، در این پروژه از یک نگاشت تجمیع‌شده در سطح لایه استفاده شد. در این روش، هر لایه به‌جای نگاشت تک‌تک نورون‌ها و سیناپس‌ها، با مشخصات تجمعی خود شامل تعداد نورون‌ها، نرخ فعالیت و بار سیناپسی نمایش داده می‌شود. سپس این مشخصات به منابع معادل در معماری Loihi-1-like نگاشت می‌شوند. بنابراین این روش قادر است اثر نسبی اندازه‌ی لایه، فعالیت اسپایکی و ارتباطات را در برآورد انرژی و تاخیر لحاظ کند، اما جایگزین یک نگاشت فیزیکی دقیق روی تراشه نیست.

مدل ارتباطی نیز به‌صورت بسته‌های اسپایکی بین هسته‌ها در نظر گرفته شد. برای هر لایه، تعداد بسته‌های ارتباطی موردنیاز با توجه به تعداد اسپایک‌ها و fan-out متوسط سیناپسی برآورد شد. این بخش اهمیت دارد، زیرا در معماری‌های نورومورفیک انرژی و تاخیر تنها ناشی از به‌روزرسانی نورون‌ها نیست و انتقال اسپایک میان هسته‌ها نیز می‌تواند بخش قابل توجهی از هزینه‌ی سیستم را تشکیل دهد.

دامنه‌ی نگاشت SANA-FE محدود به زیرشبکه‌ی اسپایکی بود. این زیرشبکه شامل ستون اصلی استخراج ویژگی و لایه‌های اضافی اسپایکی SSD است. در مقابل، لایه‌های متراکم نهایی مربوط به طبقه‌بندی و رگرسیون جعبه‌های محدودکننده در نگاشت SANA-FE وارد نشدند. در نتیجه، مقادیر انرژی و تاخیر SANA-FE را نباید با مقادیر full-detector در GPU یا مدل تحلیلی Lemaire [۶] به‌صورت عددی مستقیم مقایسه کرد.

از شبیه‌سازی SANA-FE برای هر یک از شش مدل Event-SNN چند کمیت اصلی استخراج شد: انرژی برآوردشده‌ی زیرشبکه‌ی اسپایکی، تاخیر اجرایی برآوردشده، طول مسیر بحرانی گراف[۱۱] و شاخص‌های تقریبی ظرفیت سخت‌افزاری نظیر تعداد معادل هسته‌ها و تراشه‌ها. منظور از طول مسیر بحرانی گراف در این پروژه، طولانی‌ترین مسیر محاسباتی در گراف زمان‌بندی نگاشت‌شده است. این کمیت یک شاخص حاصل از مدل زمان‌بندی SANA-FE است و الزاماً معادل زمان اجرای واقعی یک تراشه‌ی فیزیکی نیست. مقادیر هسته و تراشه در این پروژه صرفا برآورد ظرفیت بر اساس تعداد نورون‌ها و محدودیت‌های معماری هستند و نباید به‌عنوان نتیجه‌ی یک فرآیند واقعی جایگذاری[۱۲] و مسیریابی[۱۳] تفسیر شوند.

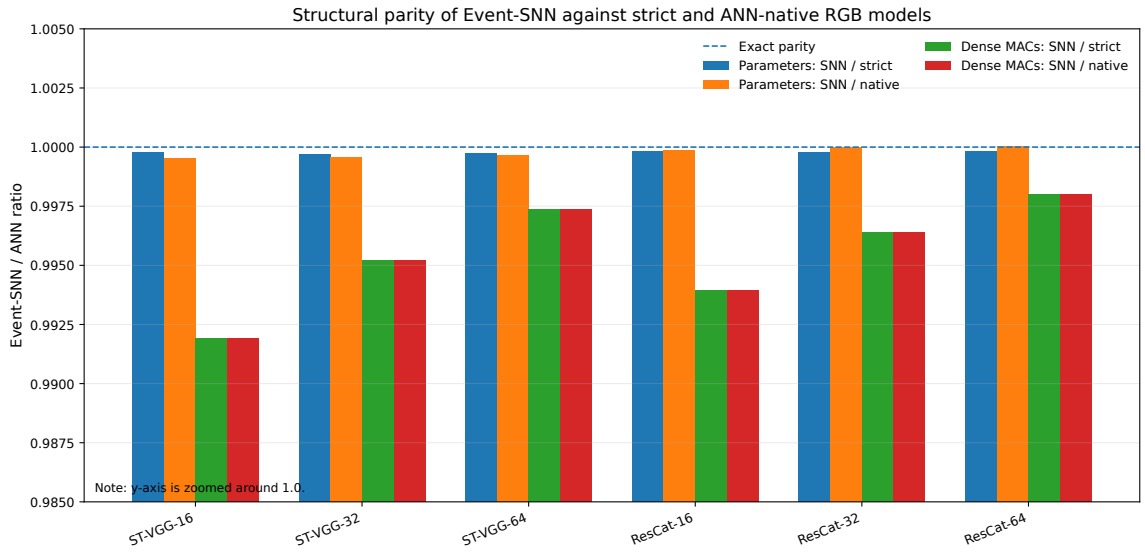
۴-۷ نتایج

۴-۷-۱ برابری ساختاری، اندازه‌ی مدل و هزینه‌ی حافظه

پیش از مقایسه‌ی رفتار اجرایی، لازم است اطمینان حاصل شود که اختلاف نتایج ناشی از کوچک‌تر بودن یکی از مدل‌ها یا تفاوت محسوس در ظرفیت پارامتری آن‌ها نیست. برای این منظور، تعداد پارامترها و بار محاسباتی متراکم هر سه نسخه‌ی Event-SNN، Strict RGB-ANN و ANN-native RGB برای هر شش معماری با یکدیگر مقایسه شدند. همان‌طور که در شکل ۴-۵ دیده می‌شود، نسبت تعداد پارامترهای Event-SNN به دو مرجع ANN در تمام پیکربندی‌ها تقریبا برابر یک است؛ این نسبت در بازه‌ی ۰٫۹۹۹۵ تا ۱٫۰۰۰۳ قرار گرفت. نسبت MAC متراکم نیز بین ۰٫۹۹۱۹ و ۰٫۹۹۸۰ بود. بنابراین اختلاف‌های مشاهده‌شده در بخش‌های بعدی را نمی‌توان به تفاوت عمده در اندازه‌ی شبکه یا بار محاسباتی متراکم نسبت داد.

برای تکمیل مقایسه‌ی نسبی، مقادیر مطلق مربوط به هر ۱۸ مدل در جدول ۴-۳ گزارش شده‌اند. تعداد پارامترها در مجموعه‌ی کامل مدل‌ها از حدود ۱٫۰۹۰ میلیون تا ۵٫۶۳۰ میلیون متغیر است. حافظه‌ی ایده‌آل موردنیاز پارامترهای FP32 از حدود ۴٫۱۶ تا ۲۱٫۴۸ MiB و اندازه‌ی state dictionary از حدود ۴٫۲۰ تا ۲۱٫۵۴ MiB تغییر می‌کند. همان‌گونه که انتظار می‌رود، سه نسخه‌ی هر معماری از

[۱۱] Critical Graph Path
[۱۲] Placement
[۱۳] Routing



شکل ۴-۵: برابری ساختاری Event-SNN با Strict RGB-ANN و ANN-native RGB. نسبت تعداد پارامترها و MAC متراکم در هر شش معماری نزدیک به یک است.

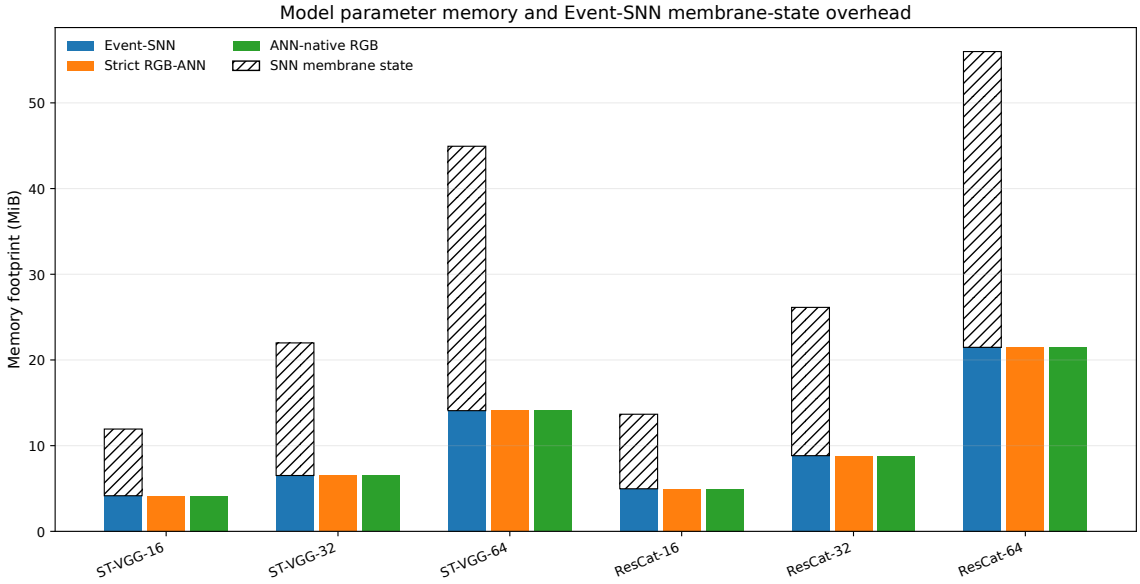
نظر حافظه‌ی پارامترها نیز تقریبا هم‌اندازه هستند. در صورت ذخیره‌ی پارامترها با دقت FP16، حافظه‌ی ایده‌آل وزن‌ها تقریبا نصف شده و در کل مجموعه در بازه‌ی ۲٫۰۸ تا ۱۰٫۷۴ MiB قرار می‌گیرد.

جدول ۴-۳: پارامترها، حافظه و بار محاسباتی متراکم برای هر ۱۸ مدل

معماری	زنجیره	پارامتر (میلیون)	پارامتر FP32 (MiB)	state dict (MiB)	حالت غشایی (MiB)	GMAC متراکم	Peak VRAM (MiB)
ST-VGG-16	Event-SNN	۱٫۰۹۰	۴٫۱۶	۴٫۲۰	۷٫۷۸	۰٫۶۰۳	۲۵۷٫۶۹
ST-VGG-16	Strict RGB-ANN	۱٫۰۹۰	۴٫۱۶	۴٫۲۰	۰٫۰۰	۰٫۶۰۸	۲۴۷٫۳۴
ST-VGG-16	ANN-native RGB	۱٫۰۹۰	۴٫۱۶	۴٫۲۰	۰٫۰۰	۰٫۶۰۸	۲۴۷٫۳۴
ST-VGG-32	Event-SNN	۱٫۷۰۸	۶٫۵۲	۶٫۵۶	۱۵٫۴۷	۲٫۰۴۷	۲۸۲٫۰۶
ST-VGG-32	Strict RGB-ANN	۱٫۷۰۹	۶٫۵۲	۶٫۵۶	۰٫۰۰	۲٫۰۵۷	۲۶۲٫۱۴
ST-VGG-32	ANN-native RGB	۱٫۷۰۹	۶٫۵۲	۶٫۵۶	۰٫۰۰	۲٫۰۵۷	۲۶۲٫۱۴
ST-VGG-64	Event-SNN	۳٫۶۹۲	۱۴٫۰۸	۱۴٫۱۳	۳۰٫۸۵	۷٫۵۲۳	۳۳۸٫۱۷
ST-VGG-64	Strict RGB-ANN	۳٫۶۹۳	۱۴٫۰۹	۱۴٫۱۳	۰٫۰۰	۷٫۵۴۳	۲۹۸٫۱۲
ST-VGG-64	ANN-native RGB	۳٫۶۹۴	۱۴٫۰۹	۱۴٫۱۴	۰٫۰۰	۷٫۵۴۳	۲۹۸٫۲۰
ResCat-16	Event-SNN	۱٫۳۰۳	۴٫۹۷	۵٫۰۲	۸٫۷۰	۰٫۸۰۶	۲۵۹٫۲۱
ResCat-16	Strict RGB-ANN	۱٫۳۰۳	۴٫۹۷	۵٫۰۲	۰٫۰۰	۰٫۸۱۰	۲۴۸٫۹۸
ResCat-16	ANN-native RGB	۱٫۳۰۳	۴٫۹۷	۵٫۰۲	۰٫۰۰	۰٫۸۱۰	۲۴۸٫۹۸
ResCat-32	Event-SNN	۲٫۳۱۵	۸٫۸۳	۸٫۸۹	۱۷٫۳۰	۲٫۷۱۷	۲۸۷٫۷۲
ResCat-32	Strict RGB-ANN	۲٫۳۱۶	۸٫۸۳	۸٫۸۹	۰٫۰۰	۲٫۷۲۷	۲۶۷٫۹۵
ResCat-32	ANN-native RGB	۲٫۳۱۵	۸٫۸۳	۸٫۸۸	۰٫۰۰	۲٫۷۲۷	۲۶۷٫۹۴
ResCat-64	Event-SNN	۵٫۶۲۹	۲۱٫۴۷	۲۱٫۵۴	۳۴٫۵۱	۹٫۹۲۲	۳۵۲٫۸۵
ResCat-64	Strict RGB-ANN	۵٫۶۳۰	۲۱٫۴۸	۲۱٫۵۴	۰٫۰۰	۹٫۹۴۲	۳۱۳٫۶۴
ResCat-64	ANN-native RGB	۵٫۶۲۹	۲۱٫۴۷	۲۱٫۵۳	۰٫۰۰	۹٫۹۴۲	۳۱۳٫۰۷

تفاوت اصلی حافظه‌ای میان دو رویکرد در حافظه‌ی حالت زمانی ظاهر می‌شود. نسخه‌های ANN حالت غشایی پایدار میان پنجره‌های زمانی ندارند، در حالی که Event-SNN برای نگهداری پتانسیل غشایی نورون‌های اسپایکی به حافظه‌ی اضافی نیاز دارد. این حافظه در FP32 از ۷٫۷۸ MiB برای ST-VGG-16 تا ۳۴٫۵۱ MiB برای ResCat-64 افزایش یافت و در تمام شش مدل اسپایکی از حافظه‌ی

صرف پارامترهای همان مدل بزرگ‌تر بود. شکل ۴-۶ حافظه‌ی پارامترهای هر سه زنجیره را برای هر شش معماری، به‌همراه سربار حالت غشایی مختص Event-SNN، به‌صورت مستقیم نشان می‌دهد.



شکل ۴-۶: مقایسه‌ی حافظه‌ی پارامترهای FP32 در هر ۱۸ مدل. برای هر معماری، حافظه‌ی Event-SNN، Strict RGB-ANN و ANN-native RGB به‌صورت جداگانه نمایش داده شده و سربار حافظه‌ی حالت غشایی تنها برای Event-SNN افزوده شده است.

علاوه بر حافظه‌ی ایده‌آل مدل، بیشینه‌ی حافظه‌ی GPU در زمان اجرای معیار نیز ثبت شد. مقدار peak VRAM در Event-SNN بین ۲۵۷٫۶۹ و ۳۵۲٫۸۵ MiB، در Strict RGB-ANN بین ۲۴۷٫۳۴ و ۳۱۳٫۶۴ MiB و در ANN-native RGB بین ۲۴۷٫۳۴ و ۳۱۳٫۰۷ MiB بود. در مقایسه‌ی زوجی با Strict RGB-ANN، سربار peak VRAM نسخه‌ی اسپایکی بسته به معماری حدود ۴٫۱ تا ۱۳٫۴ درصد بود. این مقدار فقط حافظه‌ی وزن‌ها را نشان نمی‌دهد، بلکه حافظه‌ی موردنیاز اجرای PyTorch/CUDA و نگهداری حالت‌ها و تنسورهای میانی را نیز در بر می‌گیرد.

۴-۷-۲ فعالیت اسپایکی و محاسبات موثر

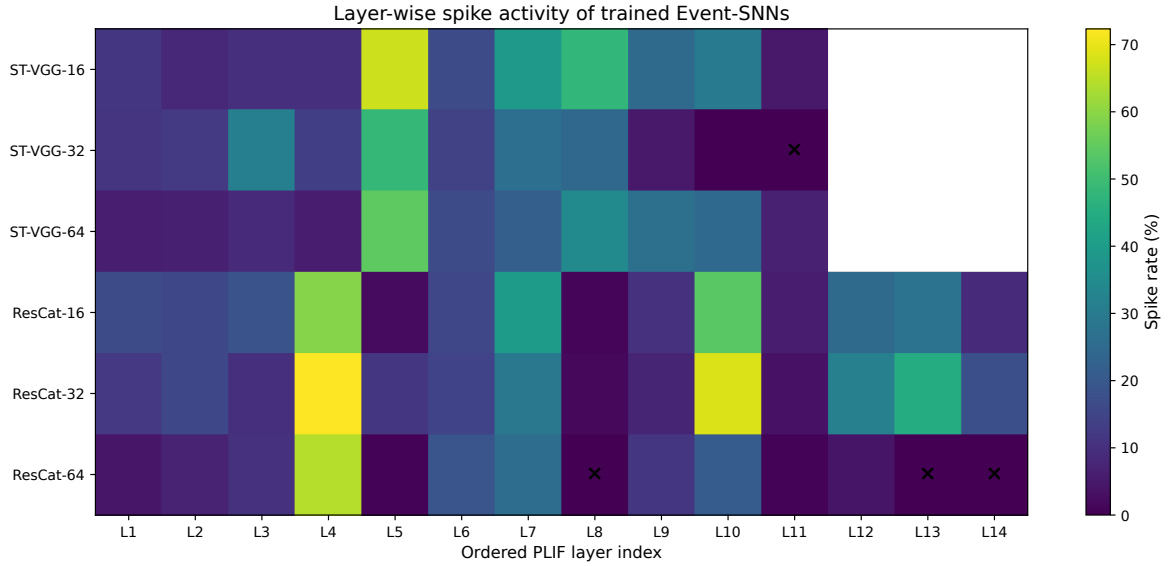
پس از آموزش محدود، نرخ اسپایک کلی شش مدل Event-SNN بین ۱۱٫۹۵ تا ۲۱٫۳۳ درصد قرار گرفت؛ در نتیجه تنکی فعالیت کلی بین ۷۸٫۶۷ تا ۸۸٫۰۵ درصد بود. این فعالیت به‌طور مصنوعی هدف‌گذاری نشده است و از رفتار مدل‌های آموزش‌دیده روی ۳۹ پنجره‌ی زمانی متوالی به‌دست آمده است. جدول ۴-۴ مقادیر دقیق نرخ اسپایک و تنکی را نشان می‌دهد. با در نظر گرفتن تعریف غیرهم‌پوشان لایه‌های dead و near-dead، در ST-VGG-32 یک لایه‌ی dead ثبت شد. در ResCat-64 دو لایه‌ی dead و یک لایه‌ی

near-dead مشاهده شد، در حالی که در چهار مدل دیگر لایه‌ی dead یا near-dead بر اساس آستانه‌های تعریف‌شده مشاهده نشد. از این رو، نتایج ResCat-64 باید با توجه به وجود چند لایه‌ی بسیار کم‌فعال تفسیر شوند.

جدول ۴-۴: خلاصه‌ی فعالیت اسپایکی Event-SNNها روی ۳۹ پنجره‌ی متوالی

مدل	نرخ اسپایک (٪)	تنکی فعالیت (٪)	dead	near-dead	saturated
ST-VGG-16	۱۵٫۷۸	۸۴٫۲۲	۰	۰	۰
ST-VGG-32	۲۱٫۳۳	۷۸٫۶۷	۱	۰	۰
ST-VGG-64	۱۱٫۹۵	۸۸٫۰۵	۰	۰	۰
ResCat-16	۱۹٫۷۰	۸۰٫۳۰	۰	۰	۰
ResCat-32	۱۷٫۴۱	۸۲٫۵۹	۰	۰	۰
ResCat-64	۱۳٫۲۵	۸۶٫۷۵	۲	۱	۰

برای مشاهده‌ی توزیع فعالیت در سطح لایه، نرخ اسپایک هر لایه و لایه‌های بسیار کم‌فعال در شکل ۴-۷ نمایش داده شده‌اند.



شکل ۴-۷: نرخ اسپایک لایه‌به‌لایه‌ی شش مدل Event-SNN. علامت ضربدر لایه‌های dead یا near-dead را بر اساس آستانه‌های تشخیصی این پروژه نشان می‌دهد.

برای بررسی اثر مستقیم این تنکی بر بار محاسباتی، علاوه بر MAC متراکم معماری، یک معیار محاسبات موثر وابسته به فعالیت نیز محاسبه شد. در مدل‌های ANN، کانولوشن‌ها به‌صورت متراکم اجرا می‌شوند و بنابراین بار مرجع تقریبا همان MAC متراکم است. در Event-SNN، بخش‌های اسپایکی بر اساس فعالیت واقعی ثبت‌شده در هر لایه محاسبه می‌شوند؛ در مقابل، لایه‌های متراکم بخش خروجی

SSD همچنان با هزینه‌ی کامل در محاسبه باقی می‌مانند. به همین دلیل، کاهش محاسبات موثر نه فقط به نرخ اسپایک کلی، بلکه به محل وقوع اسپایک‌ها در شبکه و هزینه‌ی محاسباتی لایه‌های متناظر نیز وابسته است.

در پیاده‌سازی معیار، مقدار گزارش‌شده‌ی MAC موثر برای Event-SNN از مجموع MAC متراکم بخش‌های غیراسپایکی و مولفه‌ی MAC بخش‌های اسپایکی به‌دست می‌آید. بنابراین GMAC موثر در این بخش یک معیار معادل و فعالیت‌محور برای مقایسه‌ی بار محاسباتی است و نباید به‌عنوان تعداد واقعی دستورهای CUDA اجراشده روی GPU تفسیر شود.

جدول ۴-۵ مقادیر مطلق و نسبی این مقایسه را نشان می‌دهد. برای هر شش معماری، بار متراکم دو مرجع Strict RGB-ANN و ANN-native RGB یکسان بود؛ از این رو در جدول در یک ستون مشترک گزارش شده‌اند. در ST-VGG-16، بار موثر Event-SNN برابر ۰٫۱۴۵ GMAC در برابر ۰٫۶۰۸ GMAC برای مرجع ANN بود که نشان‌دهنده ۲۳٫۹۰ درصد بار متراکم مرجع و معادل ۷۶٫۱۰ درصد کاهش است. برای ST-VGG-32 این مقدار به ۰٫۲۷۶ در برابر ۲٫۰۵۷ GMAC رسید که متناظر با ۸۶٫۵۹ درصد کاهش است. بیشترین کاهش در خانواده‌ی ST-VGG برای ST-VGG-64 مشاهده شد؛ ۰٫۵۳۷ GMAC موثر در برابر ۷٫۵۴۳ GMAC متراکم و این به معنی فقط ۷٫۱۲ درصد بار مرجع و ۹۲٫۸۸ درصد کاهش است.

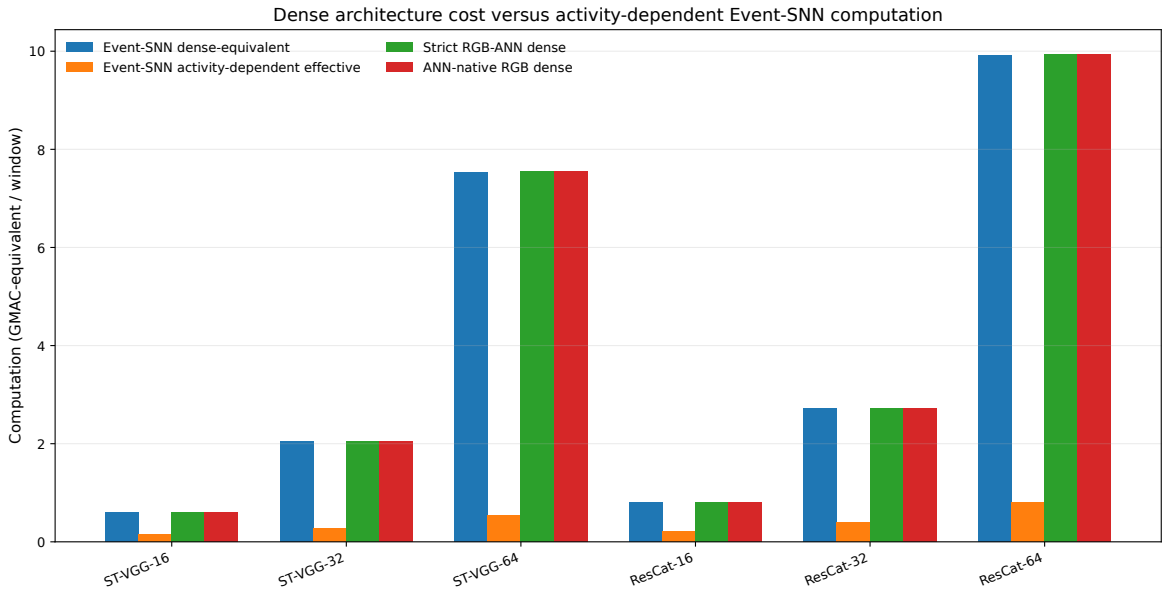
جدول ۴-۵: مقایسه‌ی بار محاسباتی موثر Event-SNN با بار متراکم دو مرجع ANN. دو مرجع Strict RGB-ANN و ANN-native RGB در شمارش متراکم MAC مقادیر یکسانی دارند.

مدل	نرخ اسپایک (٪)	(GMAC) Dense SNN	(GMAC) Effective SNN	(GMAC) Dense ANN	نسبت موثر به ANN (٪)	کاهش نسبت به ANN (٪)
ST-VGG-16	۱۵٫۷۸	۰٫۶۰۳	۰٫۱۴۵	۰٫۶۰۸	۲۳٫۹۰	۷۶٫۱۰
ST-VGG-32	۲۱٫۳۳	۲٫۰۴۷	۰٫۲۷۶	۲٫۰۵۷	۱۳٫۴۱	۸۶٫۵۹
ST-VGG-64	۱۱٫۹۵	۷٫۵۲۳	۰٫۵۳۷	۷٫۵۴۳	۷٫۱۲	۹۲٫۸۸
ResCat-16	۱۹٫۷۰	۰٫۸۰۶	۰٫۲۱۰	۰٫۸۱۰	۲۵٫۸۹	۷۴٫۱۱
ResCat-32	۱۷٫۴۱	۲٫۷۱۷	۰٫۴۰۵	۲٫۷۲۷	۱۴٫۸۵	۸۵٫۱۵
ResCat-64	۱۳٫۲۵	۹٫۹۲۲	۰٫۷۹۵	۹٫۹۴۲	۸٫۰۰	۹۲٫۰۰

رفتار مشابهی در خانواده‌ی ResCat مشاهده شد. مقدار موثر برای ResCat-16، ResCat-32 و ResCat-64 به‌ترتیب ۰٫۲۱۰، ۰٫۴۰۵ و ۰٫۷۹۵ GMAC بود، در حالی که بار متراکم مدل‌های ANN متناظر به‌ترتیب ۰٫۸۱۰، ۲٫۷۲۷ و ۹٫۹۴۲ GMAC اندازه‌گیری شد. بنابراین بار موثر Event-SNN در این سه مدل تنها ۲۵٫۸۹، ۱۴٫۸۵ و ۸٫۰۰ درصد بار متراکم مرجع بود که به‌ترتیب به کاهش ۷۴٫۱۱، ۸۵٫۱۵ و ۹۲٫۰۰ درصدی منجر می‌شود.

نتایج فوق اثر فعالیت اسپایکی را به‌صورت کمی روشن می‌کنند. اگر شبکه صرفا بر اساس ساختار

متراکم مقایسه شود، سه نسخه‌ی هر معماری تقریبا هزینه‌ی یکسانی دارند؛ اما با وارد کردن فعالیت واقعی نورون‌های اسپایکی، بار موثر شش Event-SNN به حدود ۷٫۱۲ تا ۲۵٫۸۹ درصد بار متراکم ANN کاهش می‌یابد. به بیان دیگر، معیار فعالیت‌محور کاهش ۷۴٫۱۱ تا ۹۲٫۸۸ درصدی را نشان می‌دهد. با این حال، رابطه‌ی میان نرخ اسپایک کلی و کاهش محاسبات کاملا خطی نیست. برای مثال، ST-VGG-32 با نرخ اسپایک ۲۱٫۳۳ درصد هنوز تنها ۱۳٫۴۱ درصد بار متراکم مرجع را دارد، زیرا فعالیت لایه‌ها با وزن محاسباتی متفاوت در این معیار اثر می‌گذارد. همچنین بخش خروجی متراکم SSD در هر سه زنجیره فعال باقی می‌ماند. از این رو، نرخ اسپایک کلی به‌تنهایی برای پیش‌بینی بار محاسباتی کافی نیست و توزیع لایه‌به‌لایه‌ی فعالیت نیز اهمیت دارد. مقایسه‌ی تصویری بار متراکم هر ۱۸ مدل و بار موثر شش مدل اسپایکی در شکل ۴-۸ ارائه شده است.



شکل ۴-۸: مقایسه‌ی بار محاسباتی متراکم هر ۱۸ مدل و بار محاسباتی موثر وابسته به فعالیت برای شش Event-SNN. با وجود برابری تقریبی بار متراکم سه زنجیره، فعالیت اسپایکی باعث کاهش قابل توجه معیار محاسبات موثر Event-SNN می‌شود.

این کاهش محاسبات موثر به‌تنهایی به معنی کاهش زمان اجرا یا انرژی روی GPU نیست. در اجرای PyTorch/CUDA روی NVIDIA Tesla T4، تنسورهای اسپایکی همچنان از طریق کرنل‌های متعارف GPU پردازش می‌شوند و سخت‌افزار الزاما عملیات مربوط به ورودی‌های صفر را به‌صورت رویدادمحور حذف نمی‌کند. به همین دلیل، اثر این تنکی در ادامه به‌طور مستقل در اندازه‌گیری تاخیر و انرژی GPU و نیز در مدل تحلیلی انرژی بررسی می‌شود.

۴-۷-۳ تاخیر full-detector روی GPU

در این بخش، تاخیر اجرای full-detector برای هر ۱۸ مدل روی NVIDIA Tesla T4 اندازه‌گیری و مقایسه شد. بنابراین، مقایسه‌ی این بخش صرفا میان مدل‌هایی انجام می‌شود که با روش اندازه‌گیری یکسان و روی سخت‌افزار یکسان اجرا شده‌اند.

نتایج جدول ۴-۶ نشان می‌دهند که در هر شش معماری، نسخه‌ی Event-SNN زمان اجرای بیشتری نسبت به هر دو مرجع ANN داشت. میانگین تاخیر جریانی Event-SNN در بازه‌ی ۸٫۹۶ تا ۱۶٫۹۸ ms/window قرار گرفت، در حالی که این بازه برای Strict RGB-ANN برابر ۵٫۶۰ تا ۱۱٫۰۷ ms/window و برای ANN-native RGB برابر ۴٫۹۴ تا ۱۰٫۲۱ ms/window بود.

جدول ۴-۶: مقایسه‌ی تاخیر جریانی، صدک ۹۵ و نرخ پردازش هر سه زنجیره روی NVIDIA Tesla T4.

مدل	میانگین تاخیر (ms)			صدک ۹۵ (ms)			FPS		
	SNN	Strict	Native	SNN	Strict	Native	SNN	Strict	Native
ST-VGG-16	۸٫۹۶	۵٫۶۰	۴٫۹۴	۱۰٫۴۹	۶٫۳۶	۵٫۲۶	۱۱۱٫۵۶	۱۷۸٫۶۱	۲۰۲٫۵۵
ST-VGG-32	۱۲٫۲۲	۶٫۲۲	۵٫۴۵	۱۵٫۵۰	۷٫۰۱	۶٫۰۸	۸۱٫۸۵	۱۶۰٫۸۰	۱۸۳٫۴۲
ST-VGG-64	۱۳٫۸۱	۸٫۸۱	۸٫۶۱	۱۴٫۷۰	۹٫۸۲	۱۰٫۸۱	۷۲٫۴۱	۱۱۳٫۵۲	۱۱۶٫۱۶
ResCat-16	۱۰٫۹۳	۶٫۶۴	۶٫۱۵	۱۲٫۱۸	۷٫۱۰	۶٫۵۹	۹۱٫۴۸	۱۵۰٫۷۰	۱۶۲٫۶۰
ResCat-32	۱۲٫۲۶	۷٫۴۳	۶٫۶۷	۱۳٫۵۲	۸٫۶۸	۷٫۶۶	۸۱٫۶۰	۱۳۴٫۵۳	۱۴۹٫۸۵
ResCat-64	۱۶٫۹۸	۱۱٫۰۷	۱۰٫۲۱	۱۸٫۵۸	۱۱٫۷۷	۱۱٫۲۷	۵۸٫۹۰	۹۰٫۳۶	۹۷٫۹۴

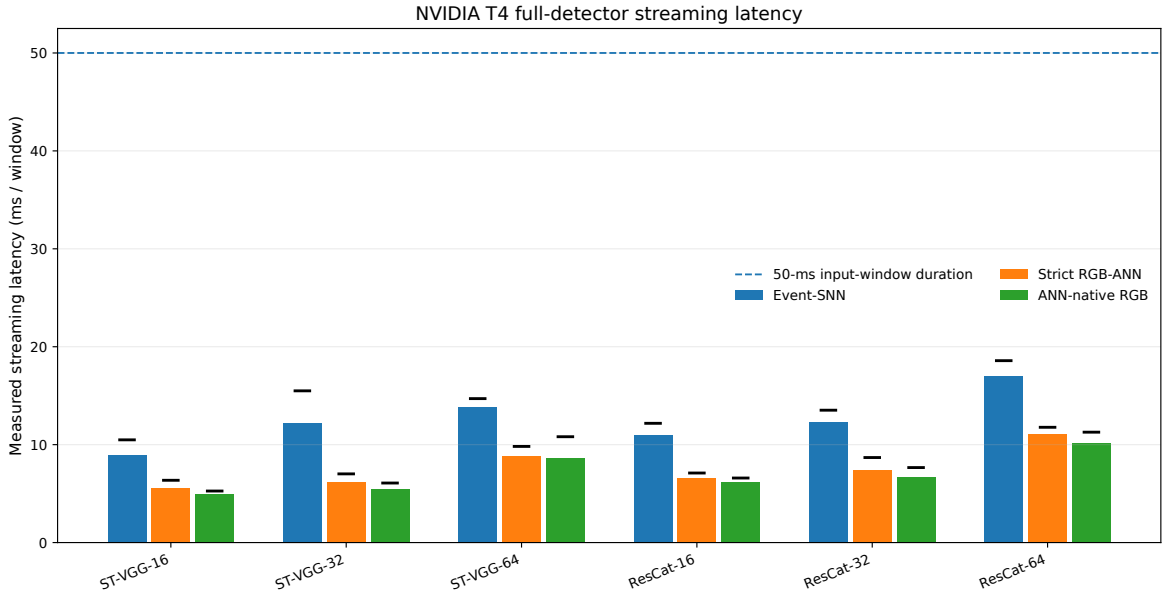
در مقایسه‌ی زوجی، نسبت میانگین تاخیر Event-SNN به Strict RGB-ANN بین ۱٫۵۳ تا ۱٫۹۶ و نسبت آن به ANN-native RGB بین ۱٫۶۰ تا ۲٫۲۴ قرار گرفت. به بیان دیگر، اجرای Event-SNN روی Tesla T4 بسته به معماری حدود ۵۳٫۴ تا ۹۶٫۵ درصد کندتر از Strict RGB-ANN و حدود ۶۰٫۴ تا ۱۲۴٫۱ درصد کندتر از ANN-native RGB بود.

برای مثال، در ST-VGG-32 میانگین تاخیر Event-SNN برابر ۱۲٫۲۲ ms بود، در حالی که Strict RGB-ANN و ANN-native RGB به‌ترتیب به ۶٫۲۲ و ۵٫۴۵ ms رسیدند. در ResCat-64 نیز زمان اجرای Event-SNN برابر ۱۶٫۹۸ ms بود، در مقایسه با ۱۱٫۰۷ ms برای Strict RGB-ANN و ۱۰٫۲۱ ms برای ANN-native RGB. بنابراین افزایش تاخیر نسخه‌ی اسپایکی به یک معماری منفرد محدود نبود و در تمام شش مقایسه مشاهده شد.

همین روند در صدک ۹۵ تاخیر و نرخ پردازش نیز دیده می‌شود. بیشترین مقدار صدک ۹۵ در میان مدل‌های Event-SNN مربوط به ResCat-64 و برابر ۱۸٫۵۸ ms بود، در حالی که مقدار متناظر برای

Strict RGB-ANN و ANN-native RGB به‌ترتیب ۱۱٫۷۷ و ۱۱٫۲۷ ms بود. همچنین نرخ پردازش Event-SNN بین ۵۸٫۹۰ تا ۱۱۱٫۵۶ FPS قرار داشت، در حالی که Strict RGB-ANN به ۹۰٫۳۶ تا ۱۷۸٫۶۱ FPS و ANN-native RGB به ۹۷٫۹۴ تا ۲۰۲٫۵۵ FPS رسید.

نمایش تصویری همین مقایسه برای هر ۱۸ مدل در شکل ۴-۹ آمده است و موقعیت هر مدل را نسبت به خط مرجع ۵۰ میلی‌ثانیه‌ای نشان می‌دهد.



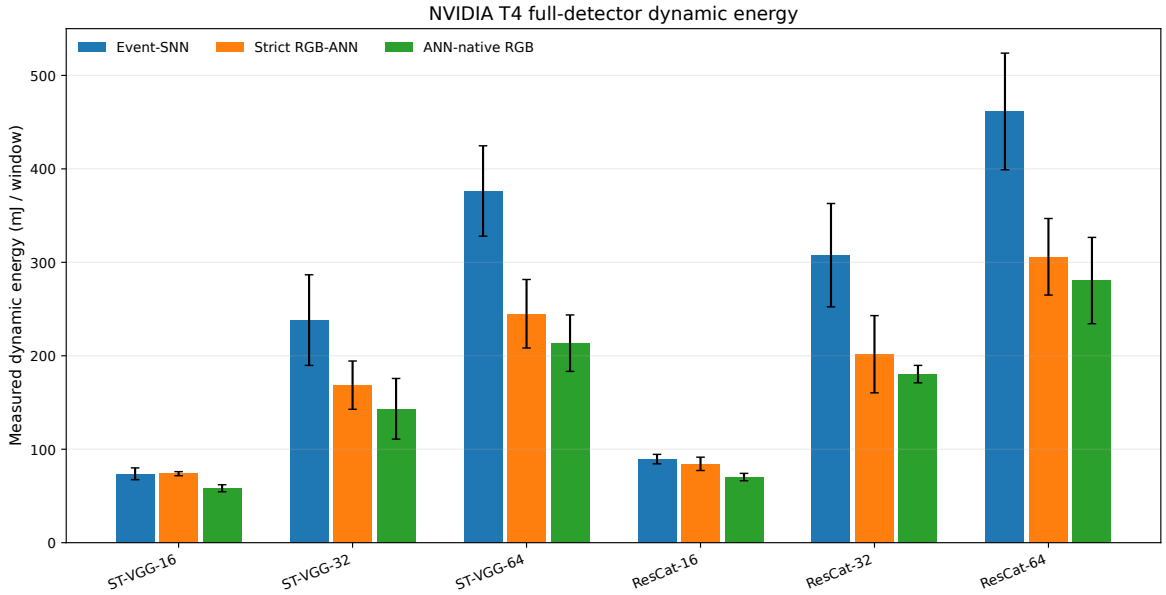
شکل ۴-۹: مقایسه‌ی تاخیر استنتاج جریانی full-detector برای هر ۱۸ مدل روی NVIDIA Tesla T4. در تمام شش معماری، تاخیر Event-SNN از هر دو مرجع ANN بیشتر است. خط مرجع ۵۰ میلی‌ثانیه‌ای طول پنجره‌ی ورودی را نشان می‌دهد.

با وجود تاخیر بیشتر Event-SNN نسبت به مدل‌های ANN، مقادیر صدک ۹۵ هر ۱۸ مدل کمتر از طول ۵۰ ms پنجره‌ی ورودی بودند. با این حال، این نتیجه تنها نشان می‌دهد که اجرای عبور رو به جلوی شبکه در شرایط معیار از فاصله‌ی زمانی ورودی کوتاه‌تر بوده است و نباید به‌عنوان اثبات تاخیر انتهابه‌انتهای sensing-to-decision تفسیر شود.

نکته‌ی مهم این است که نتیجه‌ی این بخش با کاهش شدید محاسبات موثر گزارش‌شده برای Event-SNN در بخش قبل تناقض ندارد. کاهش MAC موثر یک معیار فعالیت‌محور است، اما Tesla T4 یک GPU متراکم متعارف است و اجرای PyTorch/CUDA الزاما از صفر بودن تعداد زیادی از اسپایک‌ها برای حذف عملیات استفاده نمی‌کند. همچنین، نگهداری و به‌روزرسانی حالت غشایی، اعمال آستانه و بازنشانی نورون‌های PLIF نیز عملیات اضافی ایجاد می‌کنند. در نتیجه، در این بستر اجرایی، تنکی اسپایکی به کاهش زمان اجرا تبدیل نشد و Event-SNN عملا در تمام معماری‌ها کندتر از دو مرجع ANN بود.

۴-۷-۴ انرژی و توان اندازه‌گیری‌شده‌ی full-detector روی GPU

اندازه‌گیری NVML برای همان دامنه‌ی full-detector انجام شد و در تمام ۱۸ مدل هر پنج تکرار انرژی معتبر بودند. برای نشان دادن عدم‌قطعیت بین تکرارها، علاوه بر میانگین، بازه‌ی اطمینان ۹۵ درصد مبتنی بر توزیع Student-t نیز محاسبه شد. در مقابل، مقادیر توان دینامیکی جدول به‌صورت میانگین پنج تکرار گزارش می‌شوند. از آنجا که انرژی معیار اصلی این بخش است، عدم‌قطعیت عددی به‌صورت صریح برای انرژی ارائه شده و در شکل ۴-۱۰ نیز با میله‌های خطا نمایش داده می‌شود.



شکل ۴-۱۰: میانگین انرژی دینامیکی اندازه‌گیری‌شده‌ی full-detector برای هر ۱۸ مدل روی NVIDIA Tesla T4. هر ستون میانگین پنج تکرار معتبر را نشان می‌دهد و میله‌های خطا بازه‌ی اطمینان ۹۵ درصد مبتنی بر Student-t هستند.

جدول ۴-۷ نتایج هر سه زنجیره را به‌صورت مستقیم مقایسه می‌کند. میانگین توان دینامیکی Event-SNN در شش معماری بین ۷٫۴۳ تا ۲۵٫۲۸ وات بود. این بازه برای Strict RGB-ANN برابر ۱۰٫۸۴ تا ۲۵٫۵۸ وات و برای ANN-native RGB برابر ۱۰٫۰۴ تا ۲۵٫۳۲ وات بود. نکته‌ی مهم آن است که توان دینامیکی Event-SNN در تمام شش معماری از Strict RGB-ANN کمتر بود؛ نسبت SNN/Strict برای توان بین ۰٫۶۳ تا ۰٫۹۹ قرار گرفت. در مقایسه با ANN-native RGB نیز توان Event-SNN در پنج معماری کمتر بود و تنها در ST-VGG-64 تقریبا برابر و اندکی بیشتر بود (۱٫۰۰۸ برابر).

با وجود این، روند انرژی متفاوت است. میانگین انرژی دینامیکی Event-SNN از ۷۳٫۷۲ تا ۴۶۱٫۴۵ mJ/window تغییر کرد، در حالی که این بازه برای Strict RGB-ANN برابر ۷۳٫۸۴ تا ۳۰۵٫۹۳ mJ/window

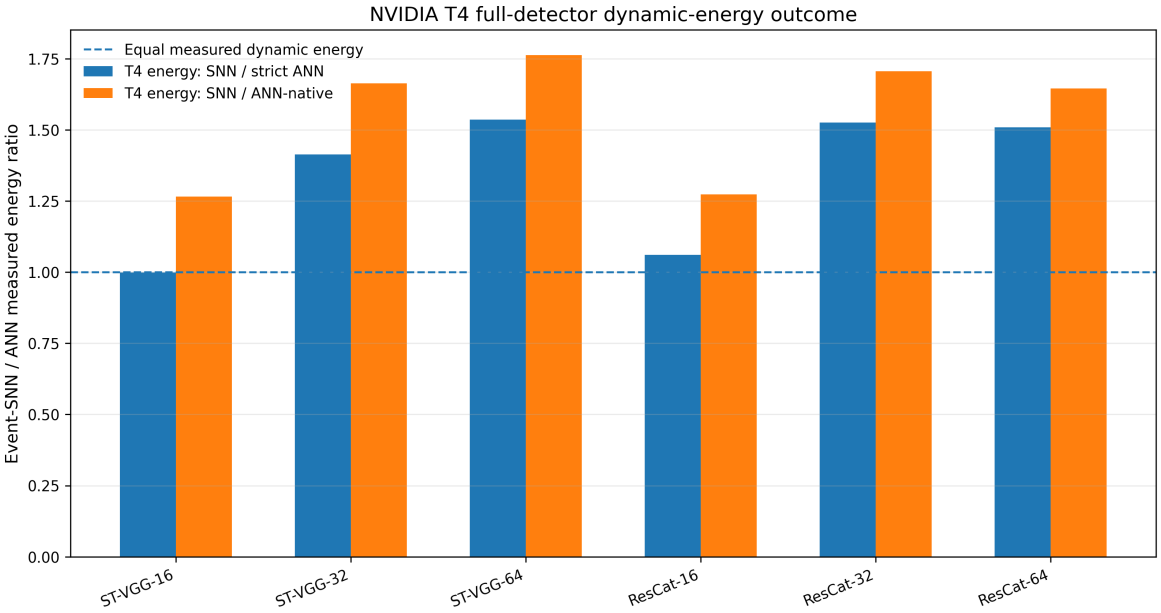
جدول ۴-۷: مقایسه‌ی توان دینامیکی، انرژی دینامیکی و EDP برای هر سه زنجیره در اجرای full-detector روی NVIDIA Tesla T4. توان‌ها میانگین پنج تکرار معتبر هستند. مقادیر انرژی به‌صورت میانگین [بازه‌ی اطمینان ۹۵٪] گزارش شده‌اند. مقادیر EDP نیز از همان تکرارهای اندازه‌گیری انرژی و زمان اجرای متناظر آن‌ها محاسبه شده‌اند.

مدل	توان دینامیکی میانگین (W)			انرژی دینامیکی (mJ/window)			EDP (mJ·ms)		
	Event-SNN	Strict ANN	ANN-native	Event-SNN	Strict ANN	ANN-native	Event-SNN	Strict ANN	ANN-native
ST-VGG-16	۷٫۵۱	۱۱٫۸۷	۱۰٫۵۰	۷۳٫۷۲ [۶۷٫۴۲, ۸۰٫۰۲]	۷۳٫۸۴ [۷۱٫۶۷, ۷۶٫۰۱]	۵۸٫۲۴ [۵۴٫۴۸, ۶۲٫۰۱]	۷۲۴٫۳۱	۴۵۹٫۷۸	۳۲۳٫۳۶
ST-VGG-32	۲۱٫۷۴	۲۴٫۵۶	۲۳٫۴۳	۲۳۸٫۱۹ [۱۸۹٫۷۱, ۲۸۶٫۶۷]	۱۶۸٫۵۸ [۱۴۲٫۷۵, ۱۹۴٫۴۱]	۱۴۳٫۲۶ [۱۱۰٫۷۹, ۱۷۵٫۷۳]	۲۶۱۰٫۵۹	۱۱۵۷٫۳۶	۸۷۶٫۲۲
ST-VGG-64	۲۵٫۲۸	۲۵٫۵۸	۲۵٫۰۷	۳۷۶٫۳۶ [۳۲۸٫۰۰, ۴۲۴٫۷۲]	۲۴۴٫۹۹ [۲۰۸٫۳۳, ۲۸۱٫۶۴]	۲۱۳٫۵۰ [۱۸۳٫۲۹, ۲۴۳٫۷۱]	۵۶۰۴٫۶۳	۲۳۴۶٫۰۸	۱۸۱۹٫۴۹
ResCat-16	۷٫۴۳	۱۰٫۸۴	۱۰٫۰۴	۸۹٫۴۵ [۸۴٫۴۱, ۹۴٫۵۰]	۸۴٫۳۷ [۷۷٫۲۴, ۹۱٫۵۰]	۷۰٫۲۳ [۶۶٫۲۸, ۷۴٫۱۸]	۱۰۷۷٫۰۴	۶۵۹٫۹۱	۴۹۲٫۸۳
ResCat-32	۲۳٫۱۵	۲۴٫۷۳	۲۴٫۳۹	۳۰۷٫۶۶ [۲۵۲٫۳۶, ۳۶۲٫۹۵]	۲۰۱٫۶۶ [۱۶۰٫۳۲, ۲۴۲٫۹۹]	۱۸۰٫۴۰ [۱۷۱٫۰۶, ۱۸۹٫۷۳]	۴۰۹۰٫۲۳	۱۶۴۴٫۹۱	۱۳۳۴٫۸۷
ResCat-64	۲۵٫۲۳	۲۵٫۵۳	۲۵٫۳۲	۴۶۱٫۴۵ [۳۹۹٫۰۵, ۵۲۳٫۸۶]	۳۰۵٫۹۳ [۲۶۴٫۹۶, ۳۴۶٫۹۰]	۲۸۰٫۵۲ [۲۳۴٫۳۵, ۳۲۶٫۶۹]	۸۴۴۱٫۷۴	۳۶۶۶٫۰۸	۳۱۰۹٫۶۳

و برای ANN-native RGB برابر ۵۸٫۲۴ تا ۲۸۰٫۵۲ mJ/window بود. نسبت میانگین انرژی SNN/Strict برای شش معماری به‌ترتیب ۰٫۹۹۸، ۱٫۴۱۳، ۱٫۵۳۶، ۱٫۰۶۰، ۱٫۵۲۶ و ۱٫۵۰۸ بود. بنابراین ST-VGG-16 از نظر میانگین تقریبا هم‌انرژی با مرجع Strict بود، اما در پنج معماری دیگر میانگین انرژی نسخه‌ی اسپایکی بیشتر بود. در مقایسه با ANN-native این نسبت در تمام شش مدل بیشتر از یک و در بازه‌ی ۱٫۲۶۶ تا ۱٫۷۶۳ قرار گرفت.

بازه‌های اطمینان نشان می‌دهند که میزان عدم‌قطعیت بین تکرارها برای همه‌ی مدل‌ها یکسان نیست. برای نمونه، در ST-VGG-16 بازه‌های اطمینان انرژی Event-SNN و Strict RGB-ANN هم‌پوشانی قابل توجهی دارند و بنابراین اختلاف بسیار کوچک میان میانگین‌های ۷۳٫۷۲ و ۷۳٫۸۴ mJ/window نباید به‌عنوان یک برتری واقعی برای یکی از دو مدل تفسیر شود. در ST-VGG-32 نیز بازه‌های این دو مدل اندکی هم‌پوشانی دارند و در ResCat-16 هم‌پوشانی مشاهده می‌شود. در مقابل، برای ST-VGG-64، ResCat-32 و ResCat-64 بازه‌های اطمینان ۹۵ درصد Event-SNN و Strict RGB-ANN از یکدیگر جدا هستند. همچنین در مقایسه با ANN-native RGB، بازه‌های اطمینان انرژی Event-SNN در هر شش معماری بالاتر و بدون هم‌پوشانی هستند. این مشاهده یک توصیف از عدم‌قطعیت اندازه‌گیری است و به‌تنهایی جایگزین آزمون آماری فرضیه نیست. نسبت میانگین انرژی هر مدل اسپایکی به دو مرجع متناظر به‌صورت مستقیم در شکل ۴-۱۱ نمایش داده شده است.

ترکیب نتایج توان و انرژی نکته‌ی مهمی را نشان می‌دهد: افزایش انرژی Event-SNN ناشی از افزایش عمومی توان دینامیکی نیست. در واقع، توان دینامیکی نسخه‌ی اسپایکی در بیشتر مقایسه‌ها کمتر از دو مرجع ANN بود، اما همان‌طور که در بخش تاخیر مشاهده شد، زمان اجرای آن روی Tesla T4 بیشتر بود. از آنجا که انرژی تقریبا حاصل انتگرال توان در طول زمان است، این زمان اجرای طولانی‌تر می‌تواند حتی در حضور توان لحظه‌ای کمتر، انرژی بیشتری به ازای هر پنجره ایجاد کند. این نتیجه در معیار



شکل ۴-۱۱: نسبت میانگین انرژی دینامیکی Event-SNN به دو مرجع Strict RGB-ANN و ANN-native RGB در اجرای full-detector روی NVIDIA Tesla T4. هر نسبت از میانگین پنج تکرار معتبر دو مدل متناظر محاسبه شده است. مقدار بزرگ‌تر از یک نشان‌دهنده‌ی انرژی بیشتر Event-SNN در همین بستر اندازه‌گیری است.

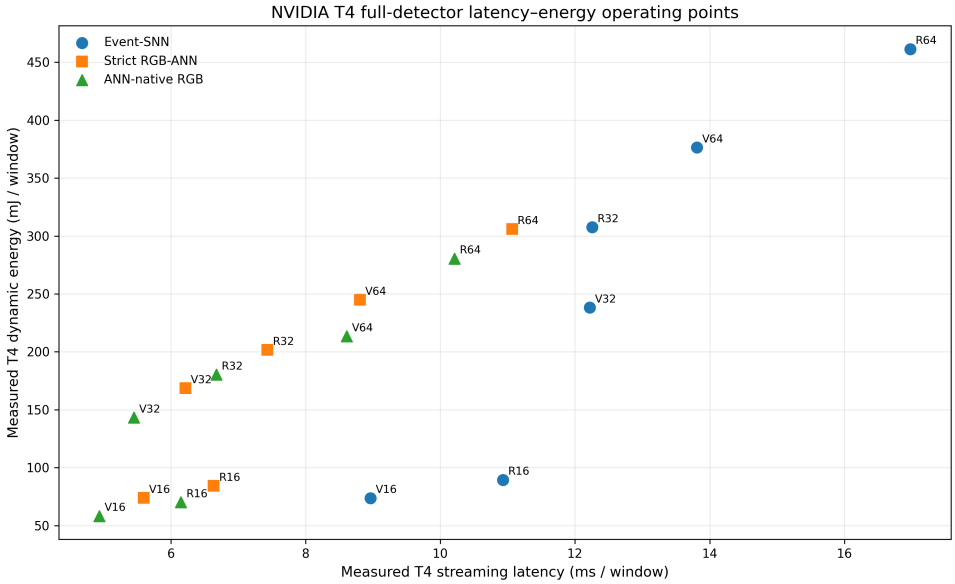
EDP نیز مشاهده می‌شود. مقادیر عددی EDP برای هر ۱۸ مدل در جدول ۴-۷ گزارش شده‌اند. این کمیت با استفاده از انرژی دینامیکی و زمان اجرای متناظر در همان تکرارهای اندازه‌گیری انرژی محاسبه شد؛ بنابراین انرژی و زمان مورد استفاده در محاسبه‌ی EDP به یک اجرای مشترک تعلق دارند.

نسبت میانگین EDP مدل Event-SNN به Strict RGB-ANN برای شش معماری به‌ترتیب ۱٫۵۸، ۲٫۲۶، ۲٫۳۹، ۱٫۶۳، ۲٫۴۹ و ۲٫۳۰ بود. نسبت متناظر به ANN-native RGB نیز به‌ترتیب ۲٫۲۴، ۲٫۹۸، ۳٫۰۸، ۲٫۱۹، ۳٫۰۶ و ۲٫۷۱ به‌دست آمد. بنابراین در تمام شش معماری، Event-SNN از نظر ترکیب هم‌زمان انرژی و زمان اجرا روی Tesla T4 مقدار EDP بالاتری نسبت به هر دو مرجع متراکم داشت.

در نتیجه، کاهش بار محاسباتی موثر ناشی از تنکی اسپایکی در اجرای متراکم PyTorch/CUDA روی Tesla T4 به مزیت تاخیر یا انرژی تبدیل نشد. این نتیجه باید به‌عنوان مشخصه‌ی اجرای این پیاده‌سازی روی یک GPU متعارف تفسیر شود و به‌عنوان نتیجه‌ای کلی درباره‌ی مصرف انرژی SNN روی سخت‌افزارهای رویدادمحور یا نورومورفیک استفاده نشود.

برای مشاهده‌ی هم‌زمان دو معیار اصلی اجرایی، نقاط عملکرد تاخیر–انرژی هر ۱۸ مدل در شکل ۴-۱۲ رسم شده‌اند. محور افقی میانگین تاخیر جریانی و محور عمودی میانگین انرژی دینامیکی به ازای هر پنجره را نشان می‌دهد؛ بنابراین حرکت به سمت پایین و چپ مطلوب‌تر است. در شکل، برچسب‌های

V16/V32/V64 به خانواده‌ی ST-VGG و برچسب‌های R16/R32/R64 به خانواده‌ی ResCat اشاره دارند. در هر شش معماری، نقطه‌ی Event-SNN نسبت به دو مرجع متراکم به سمت راست قرار گرفته است که تاخیر بیشتر آن روی Tesla T4 را نشان می‌دهد؛ همچنین به‌جز نزدیکی انرژی ST-VGG-16 به Strict RGB-ANN، نقاط اسپایکی عموماً در ارتفاع بیشتری قرار دارند. از این رو، در این بستر متعارف، هیچ مزیت مشترک تاخیر–انرژی برای Event-SNN مشاهده نشد.



شکل ۴-۱۲: نقاط عملکرد تاخیر–انرژی full-detector برای هر ۱۸ مدل روی NVIDIA Tesla T4. محور افقی میانگین تاخیر جریانی و محور عمودی میانگین انرژی دینامیکی اندازه‌گیری‌شده به ازای هر پنجره را نشان می‌دهد. قرار گرفتن نقطه در ناحیه‌ی پایین‌تر و چپ‌تر بیانگر ترکیب مطلوب‌تر تاخیر و انرژی در همین بستر اندازه‌گیری است.

۴-۷-۵ برآورد تحلیلی انرژی برای full-detector

مدل تحلیلی Lemaire که روش محاسبه، مفروضات فناوری و مولفه‌های انرژی آن در بخش ۴-۶-۵ شرح داده شد [۶]، با مجموعه‌ای یکسان از ضرایب انرژی به هر ۱۸ مدل full-detector اعمال شد. مطابق این مدل تحلیلی، انرژی کل از مجموع انرژی مربوط به حافظه، عملیات سیناپسی و آدرس‌دهی به‌دست می‌آید [۶]. در این بخش ابتدا انرژی کل سه زنجیره مقایسه شده و سپس منشا اختلاف‌ها از طریق بررسی جداگانه‌ی این سه مولفه تحلیل می‌شود.

مقادیر کل در جدول ۴-۸ نشان می‌دهند که انرژی تحلیلی Event-SNN در شش معماری از ۴۱٫۵۸ تا ۷۴۷٫۲۶ mJ/window تغییر می‌کند. برای Strict RGB-ANN این بازه برابر ۶۲٫۹۰ تا ۱۷۴۸٫۱۷

mJ/window و برای ANN-native RGB برابر ۶۲٫۵۱ تا ۱۷۴۷٫۵۲ mJ/window است.

جدول ۴-۸: انرژی تحلیلی full-detector برای هر سه زنجیره در چارچوب مدل Lemaire [۶].

مدل	(mJ) Event-SNN	(mJ) Strict ANN	(mJ) ANN-native
ST-VGG-16	۴۱٫۵۸	۶۲٫۹۰	۶۲٫۵۱
ST-VGG-32	۱۷۸٫۳۹	۲۶۵٫۸۲	۲۶۴٫۴۵
ST-VGG-64	۵۶۰٫۲۳	۱۲۵۱٫۰۰	۱۲۵۰٫۶۹
ResCat-16	۶۴٫۰۵	۹۱٫۱۵	۹۰٫۰۱
ResCat-32	۲۲۳٫۲۹	۳۹۵٫۲۰	۳۹۰٫۹۷
ResCat-64	۷۴۷٫۲۶	۱۷۴۸٫۱۷	۱۷۴۷٫۵۲

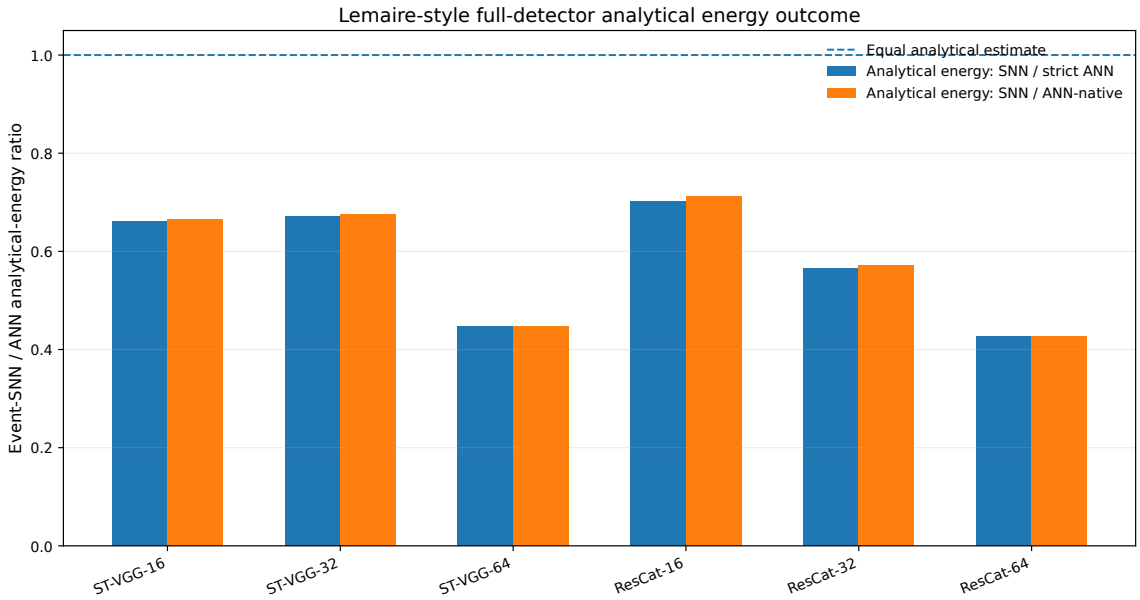
در مقایسه‌ی زوجی، نسبت انرژی تحلیلی Event-SNN به Strict RGB-ANN بین ۰٫۴۲۷ و ۰٫۷۰۳ و نسبت آن به ANN-native RGB بین ۰٫۴۲۸ و ۰٫۷۱۲ قرار گرفت. بنابراین، در چارچوب همین مدل تحلیلی، انرژی Event-SNN نسبت به مرجع متراکم متناظر حدود ۲۹ تا ۵۷ درصد کمتر برآورد می‌شود.

بیشترین کاهش نسبی در ResCat-64 و ST-VGG-64 مشاهده شد. این دو معماری در بخش ۴-۷-۲ نیز از پایین‌ترین نسبت‌های محاسبات موثر Event-SNN به بار متراکم ANN برخوردار بودند. این هم‌جهتی نشان می‌دهد که در مدل تحلیلی مورد استفاده، کاهش فعالیت اسپایکی و تعداد عملیات فعال در کاهش انرژی برآوردشده نقش مستقیم دارد؛ هرچند همان‌طور که در ادامه نشان داده می‌شود، عملیات سیناپسی تنها مولفه‌ی غالب در انرژی کل نیست.

نسبت‌های انرژی تحلیلی هر شش معماری در شکل ۴-۱۳ نمایش داده شده‌اند. در تمام پیکربندی‌ها مقدار این نسبت کمتر از یک است، یعنی مدل تحلیلی برای Event-SNN انرژی کمتری نسبت به هر دو مرجع ANN پیش‌بینی می‌کند.

برای مشخص شدن منشا اختلاف انرژی کل، سه مولفه‌ی تعریف‌شده به‌صورت جداگانه بررسی شدند. مقادیر عددی جدول ۴-۹ نشان می‌دهند که در هر ۱۸ مدل، انرژی مربوط به دسترسی و جابه‌جایی حافظه مولفه‌ی غالب برآورد تحلیلی است. سهم این مولفه از انرژی کل بین ۹۶٫۸۹ تا ۹۹٫۶۴ درصد قرار دارد. در شش مدل Event-SNN این سهم بین ۹۸٫۸۰ تا ۹۹٫۶۴ درصد است.

در مدل‌های Event-SNN، انرژی حافظه از ۴۱٫۰۸ تا ۷۴۴٫۳۰ mJ/window تغییر کرد. این مقدار برای Strict RGB-ANN بین ۶۰٫۹۶ تا ۱۷۱۶٫۳۶ mJ/window و برای ANN-native RGB بین ۶۰٫۵۷ تا ۱۷۱۵٫۷۰ mJ/window بود. در مقایسه‌ی زوجی با Strict RGB-ANN، مولفه‌ی حافظه‌ی Event-SNN حدود ۲۸٫۵ تا ۵۶٫۶ درصد کمتر برآورد شد. از آنجا که مولفه‌ی حافظه در تمامی مدل‌ها بیش از



شکل ۴-۱۳: نسبت انرژی Event-SNN به دو مرجع ANN برای full-detector صرفا در چارچوب مدل تحلیلی Lemaire [۶]. مقدار کمتر از یک نشان‌دهنده‌ی انرژی تحلیلی کمتر برای Event-SNN در همان مدل است.

جدول ۴-۹: تفکیک عددی مولفه‌های انرژی تحلیلی در مدل Lemaire [۶]. ستون آخر سهم انرژی حافظه از انرژی کل را نشان می‌دهد.

مدل	زنجیره	حافظه (mJ)	سیناپسی (mJ)	آدرس‌دهی (mJ)	سهم حافظه (٪)
ST-VGG-16	Event-SNN	۴۱٫۰۸	۰٫۴۷۵	۰٫۰۲۳۹	۹۸٫۸۰
ST-VGG-16	Strict ANN	۶۰٫۹۶	۱٫۹۴۶	۰٫۰۰۰۶	۹۶٫۹۱
ST-VGG-16	ANN-native	۶۰٫۵۷	۱٫۹۴۶	۰٫۰۰۰۶	۹۶٫۸۹
ST-VGG-32	Event-SNN	۱۷۷٫۳۹	۰٫۹۲۲	۰٫۰۷۹۹	۹۹٫۴۴
ST-VGG-32	Strict ANN	۲۵۹٫۲۴	۶٫۵۸۳	۰٫۰۰۱۱	۹۷٫۵۲
ST-VGG-32	ANN-native	۲۵۷٫۸۶	۶٫۵۸۳	۰٫۰۰۱۰	۹۷٫۵۱
ST-VGG-64	Event-SNN	۵۵۸٫۲۲	۱٫۸۱۸	۰٫۱۹۲۸	۹۹٫۶۴
ST-VGG-64	Strict ANN	۱۲۲۶٫۸۶	۲۴٫۱۳۸	۰٫۰۰۲۰	۹۸٫۰۷
ST-VGG-64	ANN-native	۱۲۲۶٫۵۵	۲۴٫۱۳۸	۰٫۰۰۲۰	۹۸٫۰۷
ResCat-16	Event-SNN	۶۳٫۳۳	۰٫۶۸۸	۰٫۰۳۱۰	۹۸٫۸۸
ResCat-16	Strict ANN	۸۸٫۵۵	۲٫۵۹۴	۰٫۰۰۰۸	۹۷٫۱۵
ResCat-16	ANN-native	۸۷٫۴۲	۲٫۵۹۴	۰٫۰۰۰۸	۹۷٫۱۲
ResCat-32	Event-SNN	۲۲۱٫۸۵	۱٫۳۵۴	۰٫۰۸۹۰	۹۹٫۳۵
ResCat-32	Strict ANN	۳۸۶٫۴۷	۸٫۷۲۷	۰٫۰۰۱۴	۹۷٫۷۹
ResCat-32	ANN-native	۳۸۲٫۲۴	۸٫۷۲۷	۰٫۰۰۱۴	۹۷٫۷۷
ResCat-64	Event-SNN	۷۴۴٫۳۰	۲٫۷۱۴	۰٫۲۴۶۸	۹۹٫۶۰
ResCat-64	Strict ANN	۱۷۱۶٫۳۶	۳۱٫۸۱۵	۰٫۰۰۲۷	۹۸٫۱۸
ResCat-64	ANN-native	۱۷۱۵٫۷۰	۳۱٫۸۱۵	۰٫۰۰۲۶	۹۸٫۱۸

۹۶ درصد انرژی کل را تشکیل می‌دهد، بخش عمده‌ی اختلاف انرژی کل میان Event-SNN و مدل‌های ANN نیز به همین مولفه مربوط است.

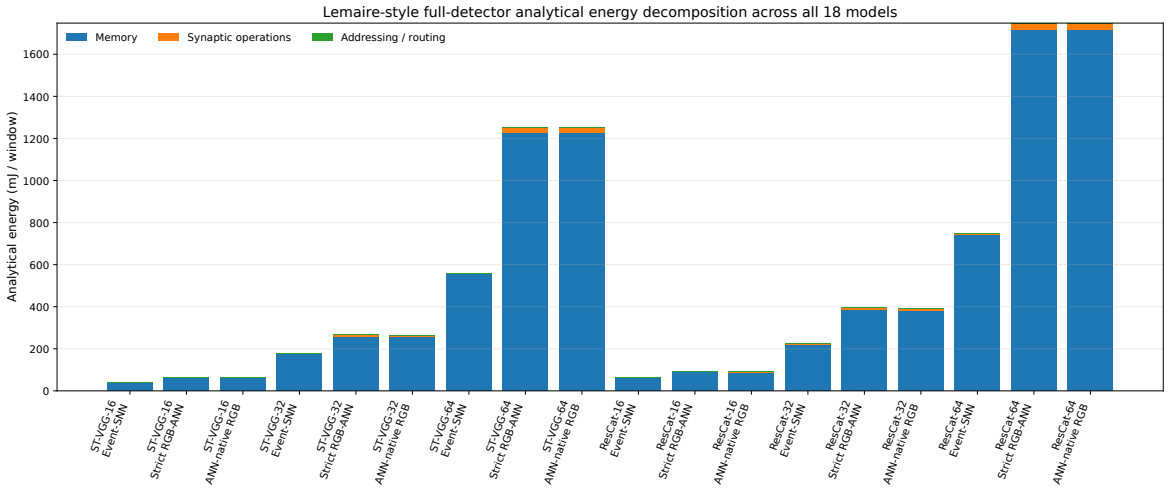
اثر تنکی اسپایکی در مولفه‌ی عملیات سیناپسی شدیدتر است. انرژی سیناپسی شش مدل Event-SNN بین ۰٫۴۷۵ تا ۲٫۷۱۴ mJ/window بود، در حالی که برای دو مرجع ANN این مقدار تقریبا از ۱٫۹۴۶ تا ۳۱٫۸۱۵ mJ/window تغییر کرد. در نتیجه، انرژی سیناپسی Event-SNN در مقایسه با Strict RGB-ANN حدود ۷۳٫۵ تا ۹۲٫۵ درصد کاهش یافت. این کاهش با نتیجه‌ی بخش محاسبات موثر سازگار است و نشان می‌دهد که فعالیت تنک واقعاً هزینه‌ی مربوط به عملیات سیناپسی را در مدل تحلیلی کاهش می‌دهد.

با این حال، کاهش ۷۳٫۵ تا ۹۲٫۵ درصدی انرژی سیناپسی مستقیماً به همان میزان کاهش انرژی کل منجر نمی‌شود. علت آن است که سهم عملیات سیناپسی از انرژی کل بسیار کوچک‌تر از سهم حافظه است. برای نمونه، در مدل‌های Event-SNN سهم انرژی سیناپسی تنها حدود ۰٫۳۲ تا ۱٫۱۴ درصد انرژی کل را تشکیل می‌دهد. بنابراین، حتی کاهش بسیار شدید تعداد عملیات فعال نمی‌تواند به‌تنهایی رفتار انرژی کل را تعیین کند.

در مقابل، مولفه‌ی آدرس‌دهی تنک در Event-SNN از نظر نسبی بزرگ‌تر از مدل‌های ANN است. انرژی آدرس‌دهی مدل‌های اسپایکی بین ۰٫۰۲۳۹ تا ۰٫۲۴۶۸ mJ/window قرار داشت، در حالی که مقدار متناظر در Strict RGB-ANN تنها حدود ۰٫۰۰۰۶ تا ۰٫۰۰۲۷ mJ/window بود. از نظر نسبی، این مولفه در Event-SNN حدود ۴۰ تا ۹۷ برابر بزرگ‌تر است. با این حال، سهم آن از انرژی کل مدل‌های اسپایکی تنها حدود ۰٫۰۳۳ تا ۰٫۰۵۷ درصد است؛ بنابراین سربار آدرس‌دهی فعالیت تنک، با وجود قابل مشاهده بودن در شمارش عملیات، اثر تعیین‌کننده‌ای بر انرژی کل این مدل تحلیلی ندارد.

تفکیک نسبی سه مولفه‌ی انرژی برای هر ۱۸ مدل در شکل ۴-۱۴ نمایش داده شده است. همان‌طور که در این شکل نیز مشاهده می‌شود، سهم حافظه در هر سه زنجیره به‌وضوح غالب است و سهم عملیات سیناپسی و آدرس‌دهی در مقایسه با آن بسیار کوچک‌تر باقی می‌ماند.

نتایج این بخش نشان می‌دهند که در چارچوب مدل Lemaire [۶]، تنکی فعالیت Event-SNN مزیت محاسباتی قابل توجهی ایجاد می‌کند و هزینه‌ی عملیات سیناپسی را به‌شدت کاهش می‌دهد؛ اما انرژی کل عمدتاً توسط هزینه‌ی دسترسی و جابه‌جایی حافظه تعیین می‌شود. به همین دلیل، کاهش ۷۴٫۱۱ تا ۹۲٫۸۸ درصدی بار محاسباتی موثر گزارش‌شده در بخش قبل، به کاهش کوچک‌تر ۲۹ تا ۵۷ درصدی انرژی تحلیلی کل منجر می‌شود.



شکل ۴-۱۴: تفکیک انرژی تحلیلی full-detector به سهم حافظه، عملیات سیناپسی و آدرس‌دهی در چارچوب مدل Lemaire [۶]. در هر ۱۸ مدل، انرژی مربوط به دسترسی و جابه‌جایی حافظه مولفه‌ی غالب است.

این نتیجه همچنین اهمیت تفکیک سطوح مختلف ارزیابی را نشان می‌دهد. مقادیر این بخش بر اساس ضرایب فناوری و فرضیات مدل تحلیلی Lemaire [۶] محاسبه شده‌اند و اندازه‌گیری مستقیم یک سخت‌افزار واقعی نیستند.

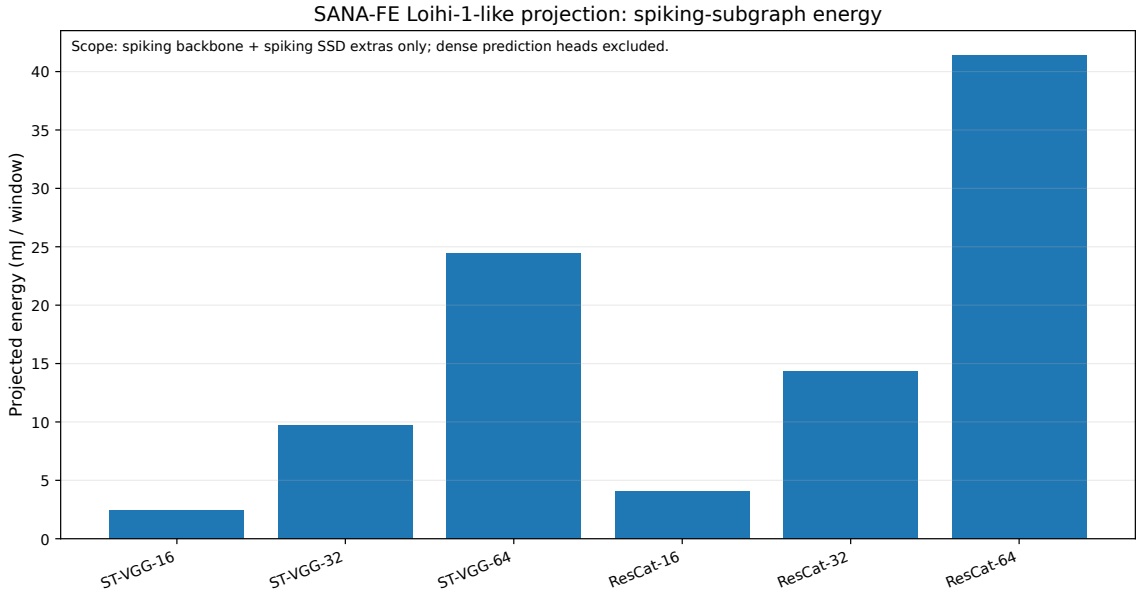
۴-۷-۶ نتایج نگاشت نورومورفیک SANA-FE

نتایج SANA-FE دامنه‌ای متفاوت از دو بخش قبلی دارند و فقط شامل بخش استخراج ویژگی اسپایکی و سه لایه‌ی اضافی اسپایکی SSD هستند؛ بخش‌های خروجی متراکم طبقه‌بندی و رگرسیون در این نگاشت حضور ندارند. بنابراین در این زیربخش فقط شش Event-SNN درون همان نگاشت layer-aggregated Loihi-1-like با یکدیگر مقایسه می‌شوند و هیچ نسبت SANA-FE/GPU یا SANA-FE/analytical گزارش نمی‌شود.

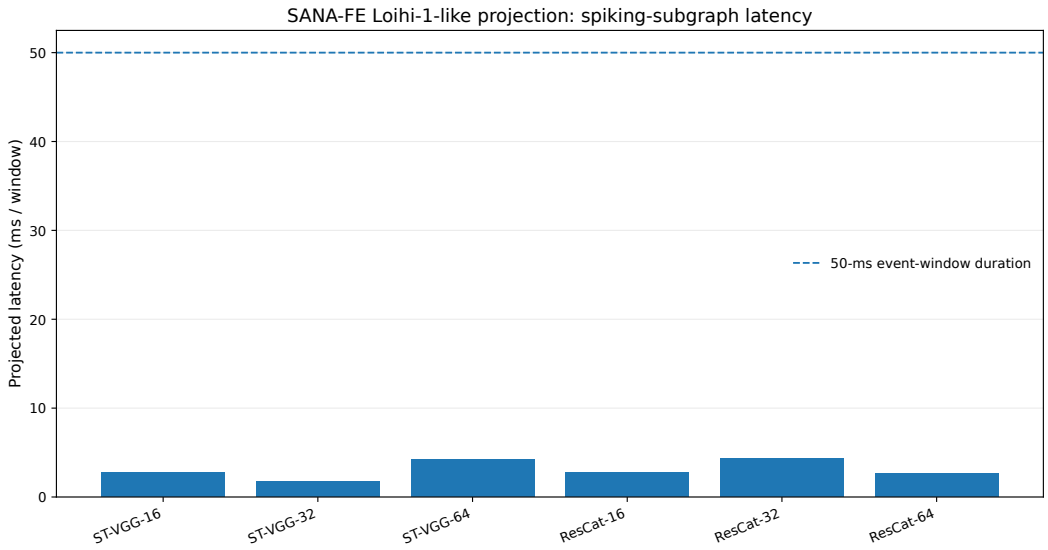
مقادیر نتیجه نگاشت در جدول ۴-۱۰ نشان می‌دهند که انرژی نگاشت‌شده‌ی زیرشبکه‌ی اسپایکی از ۲٫۴۷ mJ/window برای ST-VGG-16 تا ۴۱٫۴۴ mJ/window برای ResCat-64 تغییر می‌کند. تاخیر proxy بین ۱٫۷۴ تا ۴٫۳۲ ms/window و طول مسیر بحرانی گراف بین ۷٫۷۲ تا ۱۷٫۱۳ ms بود. این مقادیر یک نگاشت هستند و اندازه‌گیری یک تراشه‌ی واقعی Loihi محسوب نمی‌شوند. انرژی و تاخیر نگاشت‌شده‌ی شش زیرشبکه به‌ترتیب در شکل ۴-۱۵ و شکل ۴-۱۶ نمایش داده شده‌اند؛ مقادیر عددی کامل نیز در جدول ۴-۱۰ خلاصه شده‌اند.

جدول ۴-۱۰: خلاصه‌ی نتایج SANA-FE برای زیرشبکه‌ی اسپایکی شش Event-SNN

مدل	انرژی (mJ)	تاخیر proxy (ms)	طول مسیر بحرانی گراف (ms)	هسته‌ی معادل	تراشه‌ی معادل
ST-VGG-16	۲٫۴۷	۲٫۷۷	۸٫۸۰	۱۹۹۳	۱۶
ST-VGG-32	۹٫۷۴	۱٫۷۴	۷٫۷۲	۳۹۶۲	۳۱
ST-VGG-64	۲۴٫۴۶	۴٫۲۲	۱۷٫۱۳	۷۸۹۹	۶۲
ResCat-16	۴٫۱۰	۲٫۷۶	۸٫۶۱	۲۲۲۹	۱۸
ResCat-32	۱۴٫۳۵	۴٫۳۲	۱۴٫۶۰	۴۴۳۱	۳۵
ResCat-64	۴۱٫۴۴	۲٫۶۴	۱۲٫۷۳	۸۸۳۷	۷۰



شکل ۴-۱۵: انرژی نگاشت‌شده‌ی SANA-FE برای زیرشبکه‌ی اسپایکی شش Event-SNN. دامنه شامل بخش استخراج ویژگی و لایه‌های اضافی اسپایکی SSD است و بخش‌های خروجی متراکم را شامل نمی‌شود.



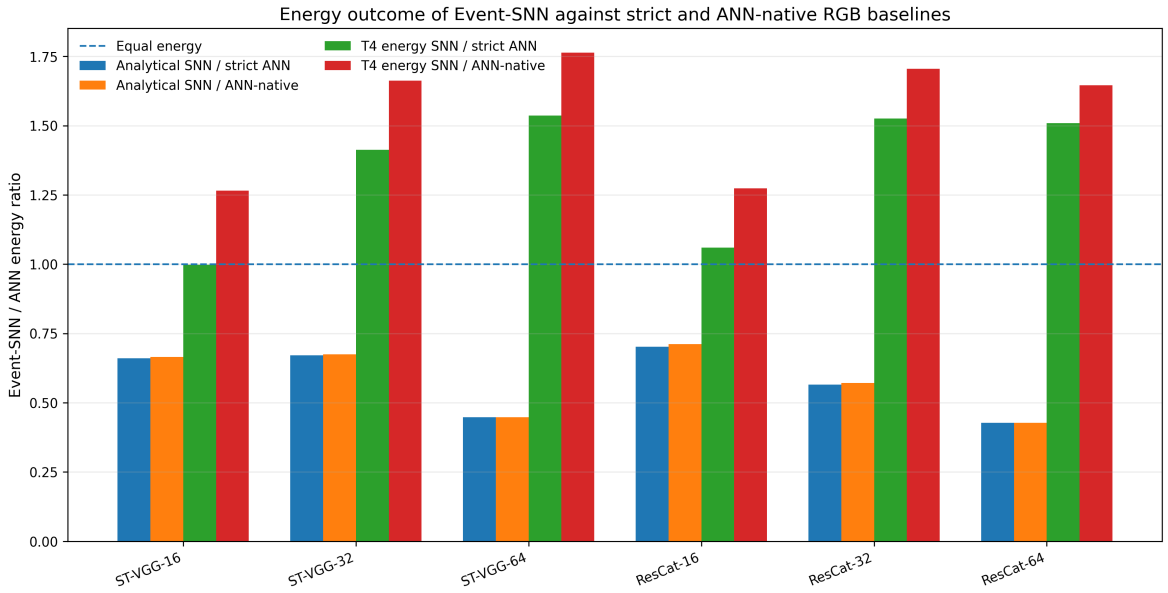
شکل ۴-۱۶: تاخیر نگاشت‌شده‌ی زیرشبکه‌ی اسپایکی در نگاشت SANA-FE و خط مرجع ۵۰ میلی‌ثانیه‌ای پنجره‌ی ورودی. این مقدار اندازه‌گیری سخت‌افزار واقعی نیست.

برای درک مقیاس مدل، یک تخمین ساده‌ی ظرفیت نیز با فرض ۱۰۲۴ نورون در هر هسته و ۱۲۸ هسته در هر تراشه‌ی Loihi-1 محاسبه شد [۲۳]. بر این اساس، تعداد هسته‌ی معادل از ۱۹۹۳ برای ST-VGG-16 تا ۸۸۳۷ برای ResCat-64 و تعداد تراشه‌ی معادل از ۱۶ تا ۷۰ به‌دست آمد. این مقادیر صرفا تخمین ظرفیت حالت نورون هستند و نتیجه‌ی جایگذاری و مسیریابی واقعی شبکه روی سخت‌افزار نیستند.

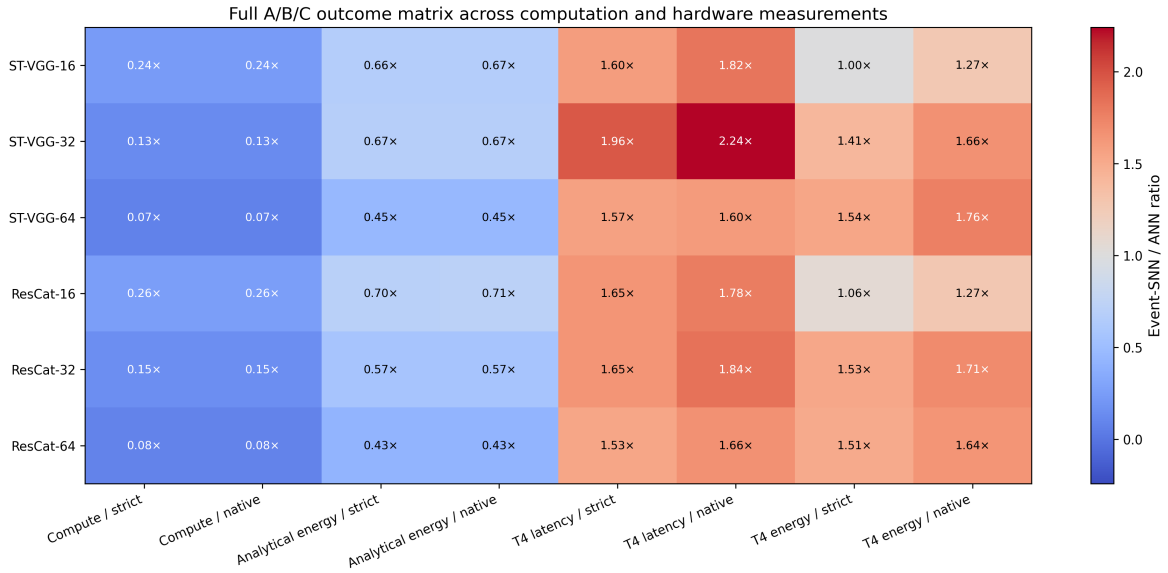
۴-۷-۷ جمع‌بندی کمی نتایج در سطوح ارزیابی

برای جمع‌بندی روندهای مشاهده‌شده، شکل ۴-۱۷ و شکل ۴-۱۸ رفتار نسبی Event-SNN را در برابر دو مرجع ANN نشان می‌دهند. این شکل‌ها نسبت‌های بدون بعد را در هر معیار نمایش می‌دهند و برای مقایسه‌ی مستقیم مقادیر مطلق حاصل از مدل‌های انرژی متفاوت استفاده نمی‌شوند.

از نظر محاسبات موثر، نسبت Event-SNN/Strict ANN از ۰٫۰۷۱ تا ۰٫۲۵۹ است. در مدل تحلیلی این نسبت انرژی در بازه‌ی ۰٫۴۲۸ تا ۰٫۷۰۳ قرار می‌گیرد، اما در اندازه‌گیری T4 نسبت تاخیر ۱٫۵۳ تا ۱٫۹۶ و نسبت انرژی دینامیکی تقریبا ۱٫۰۰ تا ۱٫۵۴ است. در مقایسه با ANN-native همین الگو حتی واضح‌تر است: نسبت تاخیر ۱٫۶۰ تا ۲٫۲۴ و نسبت انرژی ۱٫۲۷ تا ۱٫۷۶ بود. در نتیجه، داده‌های عددی این پروژه نشان می‌دهند که کاهش شدید عملیات فعال در سطح مدل به‌تنهایی تضمین نمی‌کند که یک بستر متراکم عمومی همان مزیت را در زمان و انرژی تحقق دهد.



شکل ۴-۱۷: مقایسه‌ی نسبت انرژی Event-SNN/ANN در دو سطح ارزیابی: مدل تحلیلی Lemaire [۶] و اندازه‌گیری انرژی دینامیکی روی NVIDIA Tesla T4. هر مجموعه‌ی میله تنها در چارچوب روش مربوط به خود تفسیر می‌شود.



شکل ۴-۱۸: خلاصه‌ی نرمال‌شده‌ی نتایج Event-SNN نسبت به دو مرجع ANN در معیارهای محاسبات موثر، انرژی تحلیلی، تاخیر T4 و انرژی T4. هر مقدار نسبت Event-SNN/ANN را در همان معیار نشان می‌دهد و به معنی نسبت‌گیری مستقیم میان مقادیر مطلق حاصل از روش‌های متفاوت نیست.

در سطح SANA-FE این جمع‌بندی فقط به مقایسه‌ی داخلی شش زیرشبکه‌ی اسپایکی محدود است و از مقادیر آن برای ادعای برتری نسبت به ANN استفاده نمی‌شود. به‌طور کلی، نتایج عددی فصل حاضر سه مشاهده‌ی اصلی را پشتیبانی می‌کنند: نخست، تنکی فعالیت و کاهش عملیات فعال در Event-SNN واقعی و قابل اندازه‌گیری است؛ دوم، این کاهش در مدل تحلیلی به انرژی کمتر منجر می‌شود؛ و سوم، اجرای متراکم روی Tesla T4 این مزیت را به‌صورت خودکار به کاهش تاخیر یا انرژی تبدیل نمی‌کند.

۸-۴ محدودیت‌های روش و نتایج

نتایج این فصل باید در محدوده‌ی طراحی آزمایش تفسیر شوند. نخست، شش SNN تنها پنج epoch روی یک قطعه‌ی کوچک DSEC آموزش دیده‌اند و ANNها آموزش متناظر ندارند؛ بنابراین مقایسه‌ی mAP یا کیفیت تشخیص از این نتایج قابل استنتاج نیست. دوم، نرخ‌های اسپایک روی همان مجموعه کار محدود اندازه‌گیری شده‌اند و برای نتیجه‌گیری درباره‌ی توزیع فعالیت در کل DSEC باید توالی‌های مستقل بیشتری بررسی شود. سوم، اندازه‌گیری NVML مربوط به رفتار سطح برد اجرای PyTorch روی T4 است و یک اندازه‌گیری تراشه‌ی نورومورفیک نیست. چهارم، مدل Lemaire [۶] یک برآورد تحلیلی وابسته به مفروضات هزینه‌ی محاسبه و حافظه است و نباید به‌عنوان اندازه‌گیری همان سخت‌افزار T4 تلقی شود. پنجم، SANA-FE در این پروژه یک نگاشت layer-aggregated از زیرشبکه‌ی اسپایکی است و جایگذاری و مسیریابی کامل شبکه روی تراشه‌ی Loihi واقعی انجام نشده است. در نهایت، عدد ۵۰ میلی‌ثانیه مرجع طول پنجره‌ی ورودی است و تاخیر کامل sensing-to-decision سامانه را نشان نمی‌دهد.

با وجود این محدودیت‌ها، طراحی ۱۸ مدل همتا اجازه می‌دهد برابری ساختاری، فعالیت اسپایکی و رفتار اجرایی مدل تشخیص اشیا روی یک GPU متعارف به‌صورت کنترل‌شده بررسی شوند. مدل تحلیلی نیز یک مقایسه‌ی داخلی A/B/C تحت مفروضات ثابت فراهم می‌کند، در حالی که SANA-FE فقط برای بررسی رفتار نسبی شش زیرشبکه‌ی اسپایکی در یک نگاشت نورومورفیک به‌کار می‌رود.

فصل پنجم

نتیجه‌گیری و پیشنهاد کارهای آینده

۵-۱ نتیجه‌گیری

این پروژه با هدف بررسی این پرسش انجام شد که آیا مزیت نظری محاسبات رویدادمحور و اسپایکی، یعنی کاهش تعداد عملیات مؤثر از طریق تولید داده و انجام محاسبه تنها در صورت وقوع تغییر، در عمل نیز به کاهش حافظه، تأخیر و انرژی مصرفی منجر می‌شود یا خیر. برای پاسخ به این پرسش، ۱۸ مدل همتا از دو خانواده‌ی معماری ST-VGG و ResCat، در سه مقیاس کانالی و سه نسخه‌ی متناظر ورودی و محاسباتی (Event-SNN، Strict RGB-ANN و ANN-native RGB) طراحی و از چند جنبه‌ی مکمل ارزیابی شدند. این ارزیابی شامل پیچیدگی ساختاری و فعالیت محاسباتی مستقل از سخت‌افزار، اجرای اندازه‌گیری‌شده روی یک GPU متعارف (NVIDIA Tesla T4)، مدل تحلیلی انرژی Lemaire [۶] و برآورد نگاشت نوروموروفیک با استفاده از SANA-FE [۷] بود.

نتایج نشان دادند که تنکی فعالیت در مدل‌های Event-SNN به کاهش قابل توجه تعداد عملیات فعال منجر می‌شود. نسبت محاسبات مؤثر Event-SNN به Strict ANN در مدل‌های بررسی‌شده در بازه‌ی ۰٫۰۷۱ تا ۰٫۲۵۹ قرار داشت. به بیان دیگر، مدل‌های اسپایکی بسته به معماری و مقیاس، تنها حدود ۷ تا ۲۶ درصد از عملیات فعال مدل متناظر خود را انجام می‌دهند. این نتیجه نشان می‌دهد که مزیت محاسباتی ناشی از پردازش رویدادمحور در سطح فعالیت واقعی شبکه قابل مشاهده و اندازه‌گیری است.

این کاهش تعداد عملیات در مدل تحلیلی انرژی نیز به کاهش مصرف انرژی منجر شد. نسبت انرژی Event-SNN به Strict ANN در مدل Lemaire [۶] بین ۰٫۴۲۸ و ۰٫۷۰۳ به‌دست آمد. بنابراین، هنگامی که هزینه‌ی محاسبات بر اساس تعداد واقعی عملیات فعال برآورد می‌شود، شبکه‌های اسپایکی مورد بررسی انرژی کمتری نسبت به شبکه‌های متناظر خود نیاز دارند.

با این حال، اندازه‌گیری مستقیم روی NVIDIA Tesla T4 نشان داد که کاهش تعداد عملیات فعال لزوماً به کاهش تأخیر یا انرژی روی یک GPU متعارف منجر نمی‌شود. نسبت تأخیر Event-SNN به Strict ANN در بازه‌ی ۱٫۵۳ تا ۱٫۹۶ قرار گرفت و نسبت انرژی دینامیکی آن‌ها تقریباً بین ۱٫۰۰ و ۱٫۵۴ بود. بنابراین، مدل‌های اسپایکی با وجود انجام محاسبات فعال بسیار کمتر، روی این سکوی محاسباتی کندتر اجرا شدند و در بسیاری از موارد انرژی برابر یا بیشتری مصرف کردند. این اختلاف در مقایسه با ANN-native RGB نیز مشاهده شد؛ به‌طوری که نسبت تأخیر در بازه‌ی ۱٫۶۰ تا ۲٫۲۴ و نسبت انرژی در بازه‌ی ۱٫۲۷ تا ۱٫۷۶ قرار داشت.

مجموع این نتایج نشان می‌دهد که کاهش تعداد عملیات، به‌تنهایی تضمین‌کننده‌ی بهبود کارایی اجرایی نیست. بهره‌برداری عملی از تنکی و ماهیت رویدادمحور شبکه‌های اسپایکی به میزان زیادی به معماری سخت‌افزار و نحوه‌ی اجرای محاسبات وابسته است. سخت‌افزارهای همه‌منظوره‌ای مانند GPU که عمدتاً برای اجرای محاسبات متراکم و موازی طراحی شده‌اند، الزاماً قادر به تبدیل کاهش فعالیت اسپایکی به کاهش متناظر در تأخیر و انرژی نیستند. در مقابل، نتایج حاصل از مدل تحلیلی انرژی و بررسی نگاشت نورومورفیک نشان می‌دهند که استفاده از سکوهایی که به‌طور اختصاصی برای بهره‌برداری از تنکی و پردازش رویدادمحور طراحی شده‌اند، می‌تواند زمینه‌ی مناسب‌تری برای تحقق مزیت محاسباتی شبکه‌های اسپایکی فراهم کند.

در نتیجه، یافته‌ی اصلی این پروژه آن است که مزیت محاسبات رویدادمحور و اسپایکی را نمی‌توان صرفاً بر اساس تعداد کمتر عملیات ارزیابی کرد. کارایی نهایی این رویکرد حاصل تعامل میان الگوریتم، ساختار فعالیت شبکه و سکوی سخت‌افزاری اجرا است. از این رو، ارزیابی منصفانه‌ی سامانه‌های اسپایکی نیازمند بررسی هم‌زمان پیچیدگی الگوریتمی و رفتار آن‌ها روی سخت‌افزارهای متناسب با ماهیت رویدادمحور محاسبات است.

۵-۲ پیشنهاد کارهای آینده

با توجه به نتایج بیان‌شده، مسیرهای زیر برای ادامه‌ی این پروژه پیشنهاد می‌شوند:

- **آموزش کامل و مقایسه‌ی دقت تشخیص:** آموزش هر ۱۸ مدل تا همگرایی، با تنظیم دقیق ابرپارامترها و ارزیابی روی داده‌ی نگه‌داشته‌شده‌ی مستقل، به‌گونه‌ای که بتوان علاوه بر ابعاد محاسباتی این پروژه، پارامترهای دقت تشخیص را نیز به‌صورت معتبر میان زنجیره‌ها مقایسه کرد.
- **گسترش دامنه‌ی داده‌ی ارزیابی:** تکرار اندازه‌گیری فعالیت اسپایکی و معیارهای اجرایی روی توالی‌های مستقل و متنوع‌تری از DSEC، به‌جای یک قطعه‌ی کوچک، برای اینکه پایداری نتایج در برابر تغییر صحنه بررسی شود.
- **اندازه‌گیری مستقیم روی سخت‌افزار نورومورفیک واقعی:** تکمیل برآورد layer-aggregated این پروژه با اجرای جایگذاری و مسیریابی کامل زیرشبکه‌ی اسپایکی روی یک تراشه‌ی نورومورفیک واقعی مانند Loihi، تا بتوان به طور مستقیم توان مصرفی آن به‌جای برآورد شبیه‌سازی‌شده با

SANA-FE، اندازه‌گیری شود.

- **بررسی سکوهای محاسباتی نهفته‌ی بیشتر:** تکرار اندازه‌گیری‌های سطح GPU این پروژه روی سکوهای نهفته‌تر و با بهره‌وری انرژی بالاتر (برای نمونه، NVIDIA Jetson) و شتاب‌دهنده‌های تنک‌آگاه[۱] Eyeriss)، تا مشخص شود آیا الگوی مشاهده‌شده روی Tesla T4 به سکوهای دیگر نیز تعمیم می‌یابد یا خاص یک نوع سخت‌افزار است.

- **بررسی حساسیت به پنجره‌ی زمانی و کدگذاری اسپایک:** اثر طول پنجره‌ی ورودی (غیر از عدد ۵۰ میلی‌ثانیه‌ی مرجع این پروژه) و شیوه‌های جایگزین کدگذاری رویداد به اسپایک بر تعادل میان دقت، تنکی فعالیت و انرژی سنجیده شود.

- **اندازه‌گیری تاخیر کامل sensing-to-decision:** معیار تاخیر این پروژه، که تنها به اجرای مدل محدود بود، به کل زنجیره از دریافت رویداد یا فریم خام تا تصمیم نهایی، شامل زمان ساخت ورودی و پردازش پیش از شبکه گسترش یابد.

- **بررسی معماری‌های دیگر بهینه‌شده برای سامانه‌های نهفته‌ی بی‌درنگ:** تکرار همین چارچوب مقایسه‌ای روی خانواده‌های معماری دیگری که به‌طور خاص برای اجرا روی سامانه‌های نهفته و بی‌درنگ طراحی شده‌اند (برای نمونه MobileNet، EfficientNet یا معماری‌های ترانسفورمری سبک‌وزن)، تا مشخص شود آیا الگوهای مشاهده‌شده در این پروژه به انتخاب خانواده‌ی معماری ResCat/ST-VGG وابسته‌اند یا فراتر از آن تعمیم می‌یابند.

- **بررسی اثر روش‌های TinyML:** سنجش اینکه روش‌های فشرده‌سازی مدل رایج در حوزه‌ی TinyML، از جمله هرس[۲]، کوانتیزه کردن[۳] و تقطیر دانش[۴]، چگونه بر نتایج مقایسه‌ی محاسباتی و انرژی این پروژه میان مدل‌های اسپایکی و مصنوعی اثر می‌گذارند؛ برای نمونه، آیا فشرده‌سازی شکاف مشاهده‌شده میان دو رویکرد را کاهش می‌دهد یا آن را حفظ می‌کند.

---

[۱] Sparsity-Aware
[۲] Pruning
[۳] Quantization
[۴] Knowledge Distillation

منابع و مراجع

[1] Szeliski, Richard. Computer Vision: Algorithms and Applications. Springer, 2010.

[2] Gallego, Guillermo, Delbruck, Tobi, Orchard, Garrick, Bartolozzi, Chiara, Taba, Brian, Censi, Andrea, Leutenegger, Stefan, Davison, Andrew J., Conradt, Joerg, Daniilidis, Kostas, and Scaramuzza, Davide. Event-based vision: A survey. IEEE Transactions on Pattern Analysis and Machine Intelligence, 44(1):154–180, 2022.

[3] Davidson, Simon and Furber, Steve B. Comparison of artificial and spiking neural networks on digital hardware. Frontiers in Neuroscience, 15:651141, 2021.

[4] Dampfhoffer, Manon, Mesquida, Thomas, Valentian, Alexandre, and Anghel, Lorena. Are SNNs really more energy-efficient than ANNs? an in-depth hardware-aware study. IEEE Transactions on Emerging Topics in Computational Intelligence, 7(3):731–741, 2023.

[5] Yan, Zhanglu, Bai, Zhenyu, and Wong, Weng-Fai. Reconsidering the energy efficiency of spiking neural networks. in Proceedings of the AAAI Conference on Artificial Intelligence, 2025. arXiv:2409.08290.

[6] Lemaire, Edgar, Cordone, Loic, Castagnetti, Andrea, Novac, Pierre-Emmanuel, Courtois, Jonathan, and Miramond, Benoit. An analytical estimation of spiking neural networks energy efficiency. in International Conference on Neural Information Processing (ICONIP), pp. 574–587, 2022.

[7] Boyle, James A., Plagge, Mark, Cardwell, Suma George, Chance, Frances S., and Gerstlauer, Andreas. Sana-fe: Simulating advanced neuromorphic architectures for fast exploration. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 44(8):3165–3178, 2025.

[8] LeCun, Yann, Bottou, Leon, Bengio, Yoshua, and Haffner, Patrick. Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11):2278–2324, 1998.

[9] Krizhevsky, Alex, Sutskever, Ilya, and Hinton, Geoffrey E. Imagenet classification with deep convolutional neural networks. in Advances in Neural Information Processing Systems (NeurIPS), vol. 25, pp. 1097–1105, 2012.

[10] Simonyan, Karen and Zisserman, Andrew. Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556, 2014.

[11] Szegedy, Christian, Liu, Wei, Jia, Yangqing, Sermanet, Pierre, Reed, Scott, Anguelov, Dragomir, Erhan, Dumitru, Vanhoucke, Vincent, and Rabinovich, Andrew. Going deeper with convolutions. in IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 1–9, 2015.

[12] He, Kaiming, Zhang, Xiangyu, Ren, Shaoqing, and Sun, Jian. Deep residual learning for image recognition. in IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778, 2016.

[13] Girshick, Ross, Donahue, Jeff, Darrell, Trevor, and Malik, Jitendra. Rich feature hierarchies for accurate object detection and semantic segmentation. in IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 580–587, 2014.

[14] Redmon, Joseph, Divvala, Santosh, Girshick, Ross, and Farhadi, Ali. You only look once: Unified, real-time object detection. in IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 779–788, 2016.

[15] Liu, Wei, Anguelov, Dragomir, Erhan, Dumitru, Szegedy, Christian, Reed, Scott, Fu, Cheng-Yang, and Berg, Alexander C. Ssd: Single shot multibox detector. in European Conference on Computer Vision (ECCV), pp. 21–37, 2016.

[16] Lichtsteiner, Patrick, Posch, Christoph, and Delbruck, Tobi. A 128×128 120 db 15 μs latency asynchronous temporal contrast vision sensor. IEEE Journal of Solid-State Circuits, 43(2):566–576, 2008.

[17] Gehrig, Daniel and Scaramuzza, Davide. Low-latency automotive vision with event cameras. Nature, 629:1034–1040, 2024.

[18] Maass, Wolfgang. Networks of spiking neurons: The third generation of neural network models. Neural Networks, 10(9):1659–1671, 1997.

[19] Neftci, Emre O., Mostafa, Hesham, and Zenke, Friedemann. Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. IEEE Signal Processing Magazine, 36(6):51–63, 2019.

[20] Sze, Vivienne, Chen, Yu-Hsin, Yang, Tien-Ju, and Emer, Joel S. Efficient processing of deep neural networks: A tutorial and survey. Proceedings of the IEEE, 105(12):2295–2329, 2017.

[21] Roy, Kaushik, Jaiswal, Akhilesh, and Panda, Priyadarshini. Towards spike-based machine intelligence with neuromorphic computing. Nature, 575(7784):607–617, 2019.

[22] Merolla, Paul A., Arthur, John V., Alvarez-Icaza, Rodrigo, Cassidy, Andrew S., Sawada, Jun, Akopyan, Filipp, Jackson, Bryan L., Imam, Nabil, Guo, Chen, Nakamura, Yutaka, Brezzo, Bernard, Vo, Ivan, Esser, Steven K., Appuswamy, Rathinaku-

mar, Taba, Brian, Amir, Arnon, Flickner, Myron D., Risk, William P., Manohar, Rajit, and Modha, Dharmendra S. A million spiking-neuron integrated circuit with a scalable communication network and interface. Science, 345(6197):668–673, 2014.

[23] Davies, Mike, Srinivasa, Narayan, Lin, Tsung-Han, Chinya, Gautham, Cao, Yongqiang, Choday, Sri Harsha, Dimou, Georgios, Joshi, Prasad, Imam, Nabil, Jain, Shweta, Liao, Yuyun, Lin, Chit-Kwan, Lines, Andrew, Liu, Ruokun, Mathaikutty, Deepak, McCoy, Steven, Paul, Arnab, Tse, Jonathan, Venkataramanan, Guruguhanathan, Weng, Yi-Hsin, Wild, Andreas, Yang, Yoonseok, and Wang, Hong. Loihi: A neuromorphic manycore processor with on-chip learning. IEEE Micro, 38(1):82–99, 2018.

[24] Cordone, Loic, Miramond, Benoit, and Thierion, Philippe. Object detection with spiking neural networks on automotive event data. in 2022 International Joint Conference on Neural Networks (IJCNN), pp. 1–8, 2022.

[25] Cordone, Loic. Performance of Spiking Neural Networks on Event Data for Embedded Automotive Applications. Ph.D. thesis, Universite Cote d'Azur, Nice, France, 2022.

[26] Perot, Etienne, de Tournemire, Pierre, Nitti, Davide, Masci, Jonathan, and Sironi, Amos. Learning to detect objects with a 1 megapixel event camera. in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 16639–16652, 2020.

[27] Gehrig, Mathias and Scaramuzza, Davide. Recurrent vision transformers for object detection with event cameras. in IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 13884–13893, 2023.

[28] Kugele, Alexander, Pfeil, Thomas, Pfeiffer, Michael, and Chicca, Elisabetta. Hybrid SNN-ANN: Energy-efficient classification and object detection for event-based vision. in Pattern Recognition (43rd DAGM German Conference, GCPR 2021), vol. 13024 of Lecture Notes in Computer Science, pp. 297–312, 2021.

[29] Su, Qiaoyi, Chou, Yuhong, Hu, Yifan, Li, Jianing, Mei, Shijie, Zhang, Ziyang, and Li, Guoqi. Deep directly-trained spiking neural networks for object detection. in Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 6555–6565, 2023. arXiv:2307.11411.

[30] Patel, Kinjal, Hunsberger, Eric, Batir, Sean, and Eliasmith, Chris. A spiking neural network for image segmentation. arXiv preprint arXiv:2106.08921, 2021.

[31] Henkes, Alexander, Eshraghian, Jason K., and Wessels, Henning. Spiking neural networks for nonlinear regression. Royal Society Open Science, 11:231606, 2024.

[32] Lunghi, Paolo, Silvestrini, Stefano, Dold, Dominik, Meoni, Gabriele, Hadjiivanov, Alexander, and Izzo, Dario. Energy efficiency analysis of spiking neural networks for space applications. Astrodynamics, 2025.

[33] Dalgaty, Thomas, Mesquida, Thomas, Joubert, Damien, Sironi, Amos, Soubeyrat, C., Vivet, P., and Posch, Christoph. The CNN vs. SNN event-camera dichotomy and perspectives for event-graph neural networks. in Design, Automation & Test in Europe Conference & Exhibition (DATE), pp. 1–6, 2023.

[34] Furber, Steve B., Lester, David R., Plana, Luis A., Garside, Jim D., Painkras, Eustace, Temple, Steve, and Brown, Andrew D. Overview of the SpiNNaker system architecture. IEEE Transactions on Computers, 62(12):2454–2467, 2013.

[35] Arfa, Sirine, Vogginger, Bernhard, and Mayr, Christian. Hardware-aware fine-tuning of spiking q-networks on the SpiNNaker2 neuromorphic platform. in International Conference on Neuromorphic Systems (ICONS), pp. 74–81, 2025.

[36] Amir, Arnon, Taba, Brian, Berg, David, Melano, Timothy, McKinstry, Jeffrey, Nolfo, Carmelo Di, Nayak, Tapan, Andreopoulos, Alexander, Garreau, Guillaume, Mendoza,

Marcela, Kusnitz, Jeff, Debole, Michael, Esser, Steve, Delbruck, Tobi, Flickner, Myron, and Modha, Dharmendra. A low power, fully event-based gesture recognition system. in IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 7388–7397, 2017.

[37] Yik, Jason, den Berghe, Korneel Van, den Blanken, Douwe, Bouhadjar, Younes, Fabre, Maxime, Hueber, Paul, et al. The NeuroBench framework for benchmarking neuromorphic computing algorithms and systems. Nature Communications, 16:1545, 2025.

[38] Gehrig, Mathias, Aarents, Willem, Gehrig, Daniel, and Scaramuzza, Davide. Dsec: A stereo event camera dataset for driving scenarios. IEEE Robotics and Automation Letters, 6(3):4947–4954, 2021.

[39] Fang, Wei, Yu, Zhaofei, Chen, Yanqi, Masquelier, Timothee, Huang, Tiejun, and Tian, Yonghong. Incorporating learnable membrane time constant to enhance learning of spiking neural networks. in 2021 IEEE/CVF International Conference on Computer Vision (ICCV), pp. 2661–2671, 2021.

[40] Lin, Tsung-Yi, Goyal, Priya, Girshick, Ross, He, Kaiming, and Dollar, Piotr. Focal loss for dense object detection. in 2017 IEEE International Conference on Computer Vision (ICCV), pp. 2980–2988, 2017.

[41] Paszke, Adam, Gross, Sam, Massa, Francisco, Lerer, Adam, Bradbury, James, Chanan, Gregory, Killeen, Trevor, Lin, Zeming, Gimelshein, Natalia, Antiga, Luca, Desmaison, Alban, Kopf, Andreas, Yang, Edward, DeVito, Zachary, Raison, Martin, Tejani, Alykhan, Chilamkurthy, Sasank, Steiner, Benoit, Fang, Lu, Bai, Junjie, and Chintala, Soumith. Pytorch: An imperative style, high-performance deep learning library. in Advances in Neural Information Processing Systems, vol. 32, 2019.

[42] NVIDIA Corporation. NVIDIA Management Library (NVML) API Reference Guide. NVIDIA Corporation. Device power query interface, nvmlDeviceGetPowerUsage.

Abstract

Computer vision is now integral to many real-time, embedded autonomous systems, yet conventional convolutional neural networks impose a roughly constant computational cost per frame regardless of how much a scene actually changes. Event cameras and spiking neural networks (SNNs) offer a conceptually different approach, generating data and computing only when change or activity occurs. Prior work shows that this theoretical reduction is necessary but not sufficient for practical efficiency, since the outcome depends heavily on the underlying hardware's ability to exploit sparsity. This project is motivated by this gap, asking how much of that theoretical efficiency an event-based, spiking computer-vision pipeline actually delivers, in memory, latency, and energy, when implemented on real computing platforms for object detection, in place of a conventional frame-based ANN pipeline. Rather than comparing detection accuracy, already addressed in prior work, this study builds and evaluates a matrix of 18 paired models from two architecture families, ST-VGG and ResCat, at three channel scales and three input variants (Event-SNN, Strict RGB-ANN, and ANN-native RGB), under a structural-parity principle, on identical scenes from the DSEC dataset, from four complementary angles: theoretical operation counts, measured GPU execution, the Lemaire analytical energy model, and a neuromorphic-mapping estimate via SANA-FE. The results show this reduction does translate into genuine energy savings under the analytical model, but when the same models run densely on a conventional GPU, the advantage does not automatically carry over to lower latency or energy. The central message of the project is that the practical efficiency of event-based and spiking computation is not an inherent property of the algorithm alone, but the result of co-designing algorithm and hardware together.

Keywords:

event-based computer vision, spiking neural networks, real-time embedded systems, energy efficiency, neuromorphic hardware



Amirkabir University of Technology
(Tehran Polytechnic)

Faculty of Computer Engineering

B.Sc. Thesis

Comparison of Frame-Based and Event-Based Computer Vision Models on Real-Time Embedded Systems

By

Seyed Sina Negahban

Supervisor

Dr. Hamed Farbeh

September 2026